



王道计算机教育

2026^年
计算机专业基础综合考试
冲刺模拟题参考答案

前言

由王道论坛（cskaoyan.com）组织名校状元级选手编写的“王道考研系列”辅导书，不仅参考了国内外的优秀教辅资料，而且结合了高分选手的独特复习经验，包括对考点的讲解以及对习题的选择和解析。“王道考研系列”单科辅导书共有如下四本：

- 《2026 年数据结构考研复习指导》
- 《2026 年计算机组成原理考研复习指导》
- 《2026 年操作系统考研复习指导》
- 《2026 年计算机网络考研复习指导》

我们还围绕这套书开发了一系列赢得众多读者好评的计算机考研课程，包括考点精讲、习题详解、暑期强化训练营、冲刺串讲、伴学督学和全程答疑服务等，读者可扫描封底的二维码加客服微信咨询。对于基础较为薄弱的读者，我们相信这些课程和服务能助你一臂之力。此外，我们在 B 站免费公开了本书配套的基础课程，读者可凭兑换码获取课程的课件及部分选择题的讲解视频。基础课程升华了单科辅导书中的考点讲解，强烈建议读者结合使用。

在冲刺阶段，我们还将出版如下两本冲刺用书：

- 《2026 年计算机专业基础综合考试冲刺模拟题》
- 《2026 年计算机专业基础综合考试历年真题解析》

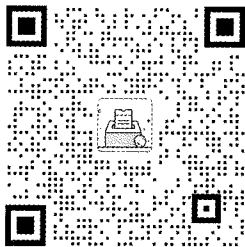
深入掌握专业课的内容没有捷径，考生也不应抱有任何侥幸心理。只有扎实打好基础，踏实做题巩固，最后灵活致用，才能在考研时取得高分。我们希望本套辅导书能够指导读者复习，但是学习仍然要靠自己，高分无法建立在空中楼阁之上。对想要继续在计算机领域深造的读者来说，认真学习和扎实掌握计算机专业的四门重要专业课是最基本的前提。

“王道考研系列”是计算机考研学子口碑相传的辅导书，自 2011 版首次推出以来，始终占据同类书销量的榜首，这就是口碑的力量。有这么多学长的成功经验，相信只要读者合理地利用辅导书，并且采用科学的复习方法，就一定能够收获属于自己的那份回报。

“不包就业，不包推荐，培养有态度的程序员。”王道训练营是王道团队打造的线下魔鬼式编程训练营。打下编程功底，增强项目经验，彻底转行入行，不再迷茫，期待有梦想的你！

参与本书编写的主要有赵霖、罗乐、徐秀瑛、赵淑芬、赵淑芳、罗庆学、赵晓宇、喻云珍、余勇、刘政学、严汉、范佳玉等。予人玫瑰，手有余香，王道论坛伴你一路同行！

若对本书有任何建议，或发现有错误，欢迎扫码联系我们，以便及时改正优化。



目 录

计算机专业基础综合考试模拟试卷（一）参考答案	1
计算机专业基础综合考试模拟试卷（二）参考答案	13
计算机专业基础综合考试模拟试卷（三）参考答案	24
计算机专业基础综合考试模拟试卷（四）参考答案	37
计算机专业基础综合考试模拟试卷（五）参考答案	48
计算机专业基础综合考试模拟试卷（六）参考答案	59
计算机专业基础综合考试模拟试卷（七）参考答案	70
计算机专业基础综合考试模拟试卷（八）参考答案	83



关注公众号【考研小舟】
免费考研资料&无水印PDF



黑白&彩印都是5分/页，满4.9全国包邮
下单备注：考研小舟，送草稿纸

全国硕士研究生入学统一考试
计算机科学与技术学科联考
计算机专业基础综合考试模拟试卷（一）参考答案

一、单项选择题（第 01~40 题）

01. D 02. B 03. D 04. A 05. B 06. B 07. D 08. D
09. C 10. D 11. D 12. A 13. C 14. A 15. C 16. C
17. B 18. C 19. A 20. B 21. A 22. C 23. A 24. B
25. D 26. D 27. B 28. B 29. D 30. B 31. C 32. C
33. A 34. D 35. B 36. C 37. A 38. D 39. D 40. B

01. D. 【解析】本题考查算法的基本性质。

算法的主要特征包括：有穷性（有限步骤内结束）、确定性（每一步都明确无歧义）、可行性（每一步都可执行），以及输入/输出。可读性是代码实现的优化目标（如变量命名、注释等），并非算法本身的必备特性。算法可用数学语言描述，即使不易读，仍有效。

02. B. 【解析】本题考查时间复杂度。

可以将双重 for 循环拆分成两个单层循环，其中外层循环的变量 i 从 1 到 n ，内层循环的变量 j 从 $2i$ 到 n 。需要注意，当 $i > n/2$ 后，内层循环的条件不再满足，不会执行，每个单层循环的执行情况如下表所示。

外层循环 i	内层循环 j	基本语句执行次数
1	$2 \sim n$	$n-1$
2	$4 \sim n$	$n-3$
...	$2i \sim n$	$n-2i+1$
$n/2$	n	1

基本语句的总执行次数为 $(n-1) + (n-3) + (n-5) + \cdots + 1 = n^2/4$ ，因此时间复杂度为 $O(n^2)$ 。

03. D. 【解析】本题考查各种遍历算法的特点。

前序、中序和后序遍历算法访问叶结点的顺序都一样，而对于层序遍历算法，当二叉树的叶结点不在同一层上时，可能先遍历后面的叶结点。本题叶结点前序遍历的顺序为 2, 6, 5，与另一种遍历算法叶结点的遍历顺序 6, 5, 2 并不一样，因此只可能选择选项 D。

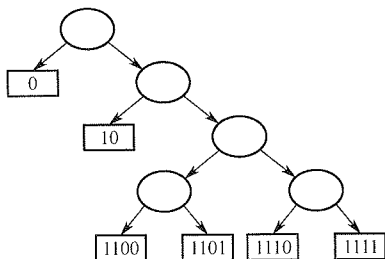
04. A. 【解析】本题考查树和二叉树的转换。

有 7 个结点的树转换为二叉树后，二叉树的根结点必定只有左孩子而没有右孩子，因此本题等价于求有 6 个结点的二叉树的形态数量，即参数为 6 的卡特兰数 $= \frac{1}{7}C_{12}^6 = 132$ 。



05. B. 【解析】本题考查哈夫曼编码。

如下图所示，可以构造出如下的哈夫曼树，在两个字符编码为 0 和 10 的情况下，构造出叶结点最多且编码长度小于或等于 4 的情况，最多还能对 4 个字符进行编码，编码分别为 1100, 1101, 1110, 1111，因此选择选项 B。



06. B. 【解析】本题考查强连通图的性质。

强连通图一定是有向图， n 个顶点至少有 n 条边才能构成强连通图，因此邻接矩阵中至少有 n 个非零元素。

07. D. 【解析】本题考查强连通分量的性质。

对于选项 A，若存在一条从 u 到 u' 的路径，假设还存在一条从 v' 到 v 的路径，则 C 和 C' 就变成一个强连通分量，与题意不符，选项 A 错误。对于选项 B，若在图 G 的某个强连通分量中加入一条新边，则不会影响该图的连通性，选项 B 错误。对于选项 C，若原先 A 和 A' 之间没有路径，则增加边 $A \rightarrow A'$ 后， A' 所属的强连通分量中的所有顶点仍然无法到达 A 所属的连通分量，选项 C 错误。对于选项 D，若原先有 A' 到 A 的路径，则增加边 $A \rightarrow A'$ 之后，两个连通分量中的顶点都能互相到达，两个连通分量变成了一个连通分量，选项 D 正确。

08. D. 【解析】本题考查排序二叉树的性质。

题意中的排序二叉树的中序遍历序列一定是升序序列 2, 3, 4, 5, 6, 7，根据排序二叉树的前序序列和中序序列能够唯一地确定一棵二叉树，选项 A、B、C 中的前序序列都能和中序序列 2, 3, 4, 5, 6, 7 构造出一棵完整的二叉树，只有选项 D 无法构造。

09. C. 【解析】本题考查 AVL 树的性质。

每个非叶结点的平衡因子不是 1 就是 -1，说明这棵有 232 个结点的 AVL 树的高度达到了最大值，树高为 1 的 AVL 树最少有 1 个结点，树高为 2 的 AVL 树最少有 2 个结点，树高为 3 的 AVL 树最少有 $1+1+2=4$ 个结点，树高为 4 的 AVL 树最少有 $1+2+4=7$ 个结点，以此类推，可知树高为 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, ... 的 AVL 树最少有 1, 2, 4, 7, 12, 20, 33, 54, 88, 143, 232, ... 个结点，因此 232 个结点代表的是树高为 11 的拥有最少结点数的 AVL 树，选项 C 正确。

10. D. 【解析】本题考查 B 树和 B+树的性质。

选项 I 和 IV 显然正确。B+树的叶结点之间存在指针链接，这有利于进行范围查询，在数据库索引中，范围查询是非常常见的操作，而 B+树的结构使得它更适合这种操作。

11. D. 【解析】本题考查快速排序的应用。

在最好的情况下，经过第一趟快速排序，即 7 次比较，确定了一个枢轴的最终位置，并且划分了两个序列，一个序列中有 3 个元素（设为序列 A），另一个序列中有 4 个元素（设为序



列 B)。第二趟快速排序对序列 A 和 B 各进行一次划分,在最好的情况下,序列 A 经过一次划分,即 2 次比较,确定了所有元素的最终位置,再对序列 B 进行一次划分,即 3 次比较,划分了两个序列,一个序列中有 1 个元素(设为序列 C),另一个序列中有 2 个元素(设为序列 D)。最后对序列 D 进行一趟快排,只需要 1 次比较就确定了所有元素的最终位置。因此,总比较次数为 $7+2+3+1=13$ 。

12. A. 【解析】本题考查指令的执行时间。

假设程序中的指令总条数为 N , 时钟周期为 T , 则 CPU 执行程序的时间为 $(20 \times 10\% + 5 \times 90\%) \times N \times T$, 其中指令 I 所用的时间为 $20 \times 10\% \times N \times T$, 因此, CPU 执行指令 I 所用的时间占整个 CPU 时间的比例是 $(20 \times 10\% \times N \times T) / ((20 \times 10\% + 5 \times 90\%) \times N \times T) = 30.77\%$ 。采取改进措施后, 指令 I 的执行时间缩短为 10 个时钟周期, 时钟周期延长 10%; 于是, 改进后 CPU 执行程序的时间为 $(10 \times 10\% + 5 \times 90\%) \times N \times T \times 1.1 = 6.05 \times N \times T$; 改进前 CPU 执行程序的时间为 $(20 \times 10\% + 5 \times 90\%) \times N \times T = 6.5 \times N \times T$, 改进后的性能是改进前的 $6.5/6.05 \approx 1.07$ 倍, 因此值得采取这种措施。

13. C. 【解析】本题考查有符号整数和无符号整数的表示。

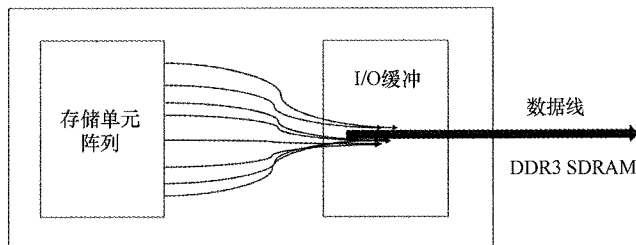
存储器访问异常是由访问数组 a 时产生的越界或越权错误造成的。循环变量 i 是 int 型, 而变量 len 是 unsigned 型, 当 int 型与 unsigned 型参与运算或比较大小时, int 型均转换为 unsigned 型。当 len 为 0 时, len-1 的运算结果为可表示的最大的 32 位无符号数, 任何无符号数都比它小, 这就使得循环体被不断执行, 进而使得数组访问越界或越权, 发生存储器访问异常。因此, 应将变量 len 定义为 int 型。

14. A. 【解析】本题考查浮点数的对阶。

float 型采用 IEEE 754 单精度浮点数格式表示, 因此最多有 24 位二进制有效位(23 位尾数 + 1 位隐藏位)。2.5 的数量级为 2^1 , 在进行 $2.5 + 2^{33}$ 运算时, 要先对阶, 对阶时, 两个数的阶码的差为 32, 则 2.5 的尾数要右移 32 位, 从而使得 24 位有效位全部丢失, 尾数全部变为 0, 再与 2^{33} 的尾数相加的结果就是 2^{33} 的尾数, 因此 $f = 2.5 + 2^{33}$ 的运算结果仍为 2^{33} , 这样再执行 $f = f - 2^{33}$ 时结果就为 0。这个例子就是典型的“大数吃小数”的例子。

15. C. 【解析】本题考查 SDRAM 芯片的性质。

DDR3 SDRAM 芯片内部的核心频率是 133.25MHz, 即存储单元阵列每秒向 I/O 缓冲传输 133.25M 次数据, 每次传输 8 个数据, 即采用了 8 位预取技术, 选项 A 正确。存储器总线每次只能传输一个数据, 因此存储器总线每秒传输数据的次数为 $133.25M \times 8 \approx 1066M$, 总线带宽为 $1066M \times 8B/s \approx 8.5GB/s$, 选项 B 和选项 D 正确。由于存储器总线在每个时钟周期内传输两次数据, 因此其时钟频率为 $1066MHz/2 = 533MHz$, 选项 C 错误。下图中显示了 DDR3 SDRAM 芯片的数据预取。



16. C. 【解析】本题考查存储器的扩展。

对于此类题，首先应确定芯片的扩展方式，计算地址时不用考虑位扩展的方向，然后列出各组芯片的地址分配，确定给定地址所在的地址范围。当用 $8K \times 8$ 位的芯片组成一个 $32K \times 32$ 位的存储器时，每行所需的芯片数为 4，每列所需的芯片数为 4， $32K$ 按字编址，地址位数为 15 位。共四组，开头两位表示组数。于是，地址划分如下：第一组为 $000\ 0000\ 0000\ 0000 \sim 001\ 1111\ 1111\ 1111$ ，即 $0000H \sim 1FFFH$ （4 位十六进制数总共不是 16 位地址，而是 15 位地址），其他芯片同理。各行芯片的地址分配如下：

第一行（4 个芯片并联）： $0000H \sim 1FFFH$ 。

第二行（4 个芯片并联）： $2000H \sim 3FFFH$ 。

第三行（4 个芯片并联）： $4000H \sim 5FFFH$ 。

第四行（4 个芯片并联）： $6000H \sim 7FFFH$ 。

因此，地址 $41F0H$ 所在芯片的最大地址为 $5FFFH$ 。

17. B. 【解析】本题考查访问存储器的具体过程。

TLB 缺失和 Cache 缺失都是在访问存储器的过程中发生的，因此这两种缺失在相同的周期内进行检测。与存储器访问相关的周期有每个指令的取指（IF）周期、lw 指令和 sw 指令的存储器访问（Mem）周期，因此①和④正确。

▲注意：在五段式指令流水线中，考生需要熟练掌握各种常见的指令在各个流水段的具体操作。本题中的 lw 指令的 IF 段与 ID 段和其他指令的没有什么不同，在执行阶段计算有效的取数地址，在 Mem 段执行从主存取数的操作，在 WB 段将取到的数写回相应的寄存器。sw 指令的 IF 段和 ID 段与其他指令的没有什么不同，在执行阶段计算有效的存数地址，在 Mem 段执行往主存写数的操作，WB 段为空操作。

18. C. 【解析】本题考查程序的机器级表示。

第一条汇编指令的含义是将变量 g 和 h 相加，并将得到的结果存放到寄存器 t0 中；第二条汇编指令的含义是将变量 i 和 j 相加，并将得到的结果存放到寄存器 t1 中；第三条汇编指令将寄存器 t0 中的值减去寄存器 t1 中的值，即将 $g+h$ 和 $i+j$ 的结果进行相减操作，因此选项 C 正确。

19. A. 【解析】本题考查多周期处理器和单周期处理器的区别。

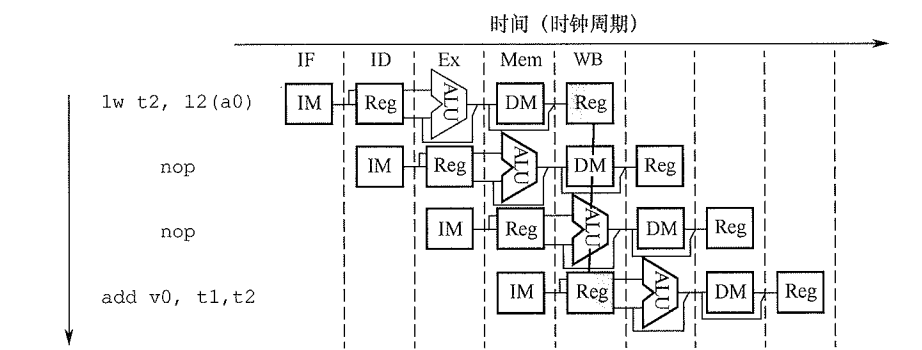
在多周期处理器的数据通路中，每条指令的 CPI 可能不同，复杂指令的 CPI 比简单指令的 CPI 更大，而单周期数据通路中每条指令的 CPI 都是 1，因此单周期处理器的 CPI 比多周期处理器的 CPI 小。

20. B. 【解析】本题考查数据冒险的处理方式。

如下图所示，在第 3 条指令前插入两条空操作指令后，当执行到第 3 条指令的 ID 段时，第一条指令已执行完成，寄存器 t1 中的值已更新，自然就消除了第 1 条和第 3 条指令之间的数据冒险；对于第 2 条和第 3 条指令之间的数据冒险，在第 3 条指令的 ID 段的前半周期，第 2 条指令已完成对寄存器 t2 的写操作，在第 3 条指令的 ID 段的后半周期，第 3 条指令可以从寄存器 t2 中取得正确的值，因此也消除了第 2 条和第 3 条指令之间的数据冒险。具体示意图



如下图所示。



21. A. 【解析】本题考查主机和外设之间的连接。

CPU 和主存通过 I/O 总线和 I/O 接口相连，I/O 接口通过通信总线（电缆）和外设相连。

22. C. 【解析】本题考查中断响应和中断处理的过程。

中断响应周期的工作由中断隐指令完成，即由硬件完成关中断、保存断点、引出中断服务程序，选项 A、选项 B 正确。当 CPU 正在执行中断源 A 的中断服务程序时，中断屏蔽寄存器中的内容由中断源 A 来设置，由于中断源 A 的处理优先级高于中断源 B 的处理优先级，因此中断源 A 会屏蔽中断源 B 的中断请求信号，选项 C 错误。若中断源 A 和 B 同时发出未被屏蔽的中断请求，则由响应优先级决定先响应中断源 B 的中断请求，当进入中断源 B 的中断服务程序，CPU 开中断后，由于中断源 A 的处理优先级高于中断源 B，于是中止中断源 B 的中断处理，转去执行中断源 A 的中断处理程序。因此，CPU 先完成中断源 A 的中断处理过程，选项 D 正确。

23. A. 【解析】本题考查资源虚拟化。

时间片轮转调度算法使得每个进程在执行过程中都像是独占 CPU，实现了对 CPU 资源的虚拟化。内存分区使得每个进程都像是拥有一整块内存，实现了对内存资源的虚拟化。SPOOLing 技术采用类似于脱机输入/输出的思想，实现了虚拟设备的功能。第二类虚拟机管理程序以用户态运行在宿主操作系统上，选项 IV 错误，其余说法均正确。

24. B. 【解析】本题考查引导程序。

引导扇区是在启动时用来引导操作系统的，没有操作系统，也就没有引导扇区的概念，选项 C、选项 D 错误。引导扇区中的内容是在安装操作系统时写入的，选项 B 正确。BIOS（基本输入/输出系统）存放在只读 ROM 中，其功能主要有：① 硬件通电自检；② 初始化，包括创建中断向量、设置寄存器及一些硬件参数；③ 引导程序，将磁盘 0 号磁道 0 号扇区上的引导程序装入内存，然后将控制权交给引导程序，由引导程序将操作系统装入内存。

25. D. 【解析】本题考查进程的创建和撤销。

引起进程创建的主要事件有用户登录、作业调度、提供用户需要的服务、应用请求等。引起进程撤销的主要事件有：① 正常结束；② 异常结束，在进程运行期间，可能出现某些错误迫使进程终止，包括越界错误、保护错、非法指令、算术运算错、I/O 故障等；③ 外界干预，

包括操作员或操作系统干预、父进程请求和父进程终止。因此，选项 D 错误。

26. D. 【解析】本题考查进程状态的转换。

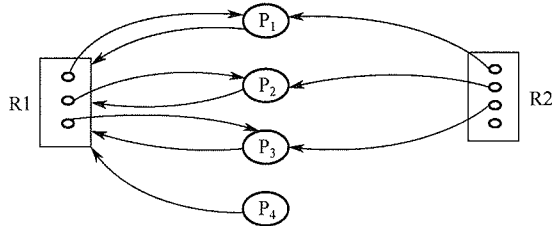
创建进程是一个很复杂的过程：首先由进程申请一个空白 PCB，并向 PCB 中填写用于控制和管理进程的信息；然后为该进程分配运行时所需的资源；最后，将该进程的状态转为就绪态，并插入就绪队列的队尾。然而，若进程所需的资源不能得到满足，例如，系统尚无足够的内存装入进程，即创建工作尚未完成，进程不能被调度运行，则此时进程所处的状态称为创建态。若进程创建成功，则进程状态就会转为就绪态，选项 D 错误。

27. B. 【解析】本题考查进程的同步互斥。

进程执行完成后，若系统中没有任何其他的就绪进程，则不会引起任何进程状态的变化，选项 A 错误。wait、signal 操作是用来解决进程的同步互斥的，可以解决一切互斥问题，但不能防止系统发生死锁，选项 C 错误。程序的顺序执行具有可再现性，即每次执行的结果一定都是相同的，而多道程序并发执行时，会失去可再现性，选项 D 错误。

28. B. 【解析】本题考查死锁的分析。

系统运行过程中有可能产生死锁。由题意可知，系统中只有 3 台 R1，它要被 4 个进程共享，且每个进程对它的最大需求均为 2，则当进程 P_1 、 P_2 、 P_3 各得到 1 台 R1 时，它们可继续运行，并且均可以顺利申请到一台 R2，但当它们第 2 次申请 R1 时，因系统已无空闲的 R1，它们将全部阻塞，并进入循环等待的死锁状态，此时的进程-资源图如下图所示。



29. D. 【解析】本题考查共享段的概念和性质。

整个操作系统中只有一张共享段表，共享段表包含共享进程计数变量 count、存取控制、段号等字段。其中，变量 count 记录有多少个进程正在共享该分段，只有当 count 值为 0 时，才由系统回收该段所占用的内存区。存取控制字段可为不同的进程赋予不同的存取权限，一个共享段在不同的进程中有不同的段号，每个进程可用自己进程的段号去访问该共享段。

30. B. 【解析】本题考查文件“簇”的概念。

为了适应磁盘容量不断增大的需要，当进行盘块分配时，不再以盘块而以簇为基本单位，一簇应包含的扇区数量与磁盘容量大小直接相关。以簇为基本分配单位的优点如下：能适应磁盘容量不断增大的情况，减少 FAT 表的表项数（在相同的磁盘容量下，FAT 表的表项数与簇的大小成反比），使 FAT 表占用更少的存储空间，减少访问 FAT 表的存取开销。

31. C. 【解析】本题考查设备驱动程序的特点。

设备驱动程序有以下特点：



- 1) 设备驱动程序主要是指在请求 I/O 的进程与设备控制器之间的一个通信和转换程序。
- 2) 设备驱动程序与设备控制器和 I/O 设备的硬件特性密切相关。
- 3) 设备驱动程序与 I/O 设备所采用的 I/O 控制方式紧密相关。
- 4) 由于设备驱动程序与硬件紧密相关, 因此其中的一部分须用汇编语言编写, 目前许多设备驱动程序的基本部分已经固化在 ROM 中。
- 5) 设备驱动程序应允许可重入, 相同的设备可以使用相同的设备驱动程序。
- 6) 设备驱动程序不允许系统调用, 设备驱动程序本身就运行在内核态, 而系统调用是用户态的程序请求使用内核态的服务。

因此, 选项 C 错误。

32. C. 【解析】本题考查磁盘的性能分析。

磁盘旋转用时为 20ms/转, 每个磁道存放 10 条记录, 故读出一条记录的时间为 $20\text{ms}/10 = 2\text{ms}$ 。

- 1) 对于第一种记录分布的情况, 读出并处理记录 A 需要 6ms, 此时读/写磁头已转到记录 D 的开始处, 因此为了读出记录 B, 必须 8 条记录 (从记录 D 到记录 B)。后续 8 条记录的读取及处理与此相同, 但最后一条记录的读取与处理只需 6ms。于是, 处理 10 条记录的总时间为 $9 \times (2 + 4 + 16)\text{ms} + (2 + 4)\text{ms} = 204\text{ms}$ 。
- 2) 对于第二种记录分布的情况, 读出并处理记录 A 后, 读/写磁头刚好转到记录 B 的开始处, 因此立即就可读出并处理, 后续记录的读取与处理的情况相同, 共转了 2.7 圈。最后一条记录的读取与处理只需 6ms。于是, 处理 10 条记录的总时间为 $20\text{ms} \times 2.7 + 6\text{ms} = 60\text{ms}$ 。

综上所述, 信息分布优化后, 处理时间缩短了 $204\text{ms} - 60\text{ms} = 144\text{ms}$ 。

33. A. 【解析】本题考查分组交换和电路交换的时间计算。

电路交换的时延由三部分组成: 建立连接的时间 c , 总传播时延 kd , 发送时延 x/b 。因此, 电路交换的总时延为 $c + kd + x/b$ 。分组交换的时延也要考虑三部分: 分组的首部长度不考虑, 因此源点的发送时延为 x/b ; 中间还要经过 $k-1$ 个结点的转发, 每个分组的转发时延为 p/b , 因此所有中间结点的总转发时延为 $(k-1)p/b$; 所有链路的传播时延为 kd 。分组交换的总时延为 $kd + x/b + (k-1)p/b$ 。于是, 若要求分组交换的总时延比电路交换的小, 则需满足 $kd + x/b + (k-1)p/b < c + kd + x/b$, 即 $(k-1)p/b < c$ 。

34. D. 【解析】本题考查香农定理和时延带宽积的计算。

根据香农定理, 信道的最大数据传输率为 $w \log_2(1 + S/N) \approx 160\text{M} \times 5\text{b/s} = 800\text{Mb/s}$, 信道的传播时延为 $1\text{km} \div (200000\text{km/s}) = 5\mu\text{s}$ 。因此, 时延带宽积 $= 5\mu\text{s} \times 800\text{Mb/s} = 4000\text{bit}$ 。由于文件长度大于这个时延带宽积, 因此链路上的比特数量的最大值为 4000bit。选项 D 正确。当文件比特数量小于时延带宽积时, 链路上比特数量的最大值就是该文件所包含的比特数量, 即若这个文件只有 2000bit, 则该链路上的比特数量的最大值就是 2000bit。

35. B. 【解析】本题考查 SR 协议的理解和计算。

采用 SR 协议传输数据, 使用 4bit 给帧编号, 则发送窗口+接收窗口 ≤ 16 , 接收窗口的尺寸取值为 5, 则发送窗口尺寸可取的最大值为 $16 - 5 = 11$, 即初始时发送窗口内包含 0~10 号帧, SR 采用的是逐帧确认, 已收到 1, 3, 5 号帧的确认, 而 0, 2, 4 号数据帧依次超时, 因此发送窗口不发生移动, 发送方最多能发送的数据帧数量为 $11 - 3$ (1, 3, 5 号帧) $- 1$ (6 号帧还未超时,



不能重发) = 7。

36. C. 【解析】本题考查 IP 首部标识字段的作用。

网络层重传时, IP 数据报首部的标识字段会有另一个标识符。仅当标识符相同时, IP 数据报片才能组装成一个 IP 数据报。前两个报片的标识符与后两个报片的标识符不同, 因此不能组装成一个 IP 数据报。

37. A. 【解析】本题考查 ARP 的工作机制和交换机的特性。

主机 A 向 E 发送的 ARP 请求报文是广播报文, 因此 A 之外的其余 5 台主机均能收到此报文, 且该报文在经由交换机转发时, 交换机会记录下 A 的 MAC 地址。当 E 收到该请求报文时, 会发出 ARP 单播响应报文, 交换机收到该报文时, 因为已记录过 A 的 MAC 地址, 所以不会向主机 D、F 转发, 但集线器没有寻址功能, 会向所有端口转发, 因此 A、B、C 均能收到该响应报文。

38. D. 【解析】本题考查 PDU 在对等层间的处理。

PDU 中装载的是哪一层的数据, 就由哪一层来处理该数据, 而 PDU 所在的层只负责传输该数据。IP 网络是分组交换网络, 每个分组的首部都包含完整的源地址和目的地址, 以便途经的路由器为每个 IP 分组进行路由, 即便是同一个源站点向同一个目的站点发出的多个 IP 分组也并不一定走同一条路径, 也就是说, 这些 IP 分组可能不一定按序到达目的站点, 目的站点的传输层必须进行排序, 传输层根据 TCP 首部中的“序号”字段来对接收到的分组进行排序, 以保证数据能有效地上交给应用层。一个较大的 IP 分组在传输过程中, 由于途经物理网络的 MTU 可能较小, 因此一个 IP 分组可能会分成若干分组, 每个分组都有完整的首部, 与普通的 IP 分组没有区别地传输。因为 IP 数据报是在网络层进行分片的, 所以接收站点的网络层必须对沿途被分片的分组进行重组, 还原成原来的 IP 分组, 因此重组工作是由网络层完成的。

39. D. 【解析】本题考查 TCP 的计时器。

TCP 共使用 4 种计时器: 重传计时器(超时计时器)、持续计时器、保活计时器和时间等待计时器。TCP 每发送一个报文段, 就对这个报文段设置一个重传计时器, 用来判断该报文段是否需要超时重传; 当发送方收到一个窗口大小为零的确认时, 为避免双方死锁等待, 就启动持续计时器; 当客户端突然出现故障时, 使用保活计时器可避免服务器在客户端突然出现故障时一直等待; 时间等待计时器在连接释放时使用, 其初始值设置为最长报文段寿命的 2 倍。

40. B 【解析】本题考查 HTTP。

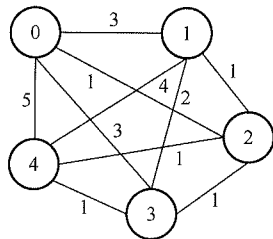
对于非持续的 HTTP, 需要使用 4 个 TCP 连接来分别传输这四个网页对象; 对于持续的 HTTP, 可在一个 TCP 连接上传输这四个网页对象, 当然, 不论 HTTP 采用的是持续连接还是非持续连接, 都需要发送 4 个请求报文才能收到 4 个响应报文, 选项 I 错误。x.com/1.html 和 x.com/2.html 在同一个服务器上, 若使用 HTTP/1.1 持续连接, 则可在同一个 TCP 连接上传输这两个网页, 选项 II 正确。当 HTTP 使用非持续连接时, 每个新 HTTP 请求都必须建立一个新 TCP 连接, 选项 III 错误。有些响应报文不用实体主体字段, 例如, 当服务器无法找到客户所请求的文件时, 服务器返回的响应报文就没有实体主体字段, 此时在响应报文的状态行中会返回一个状态码 404, 选项 IV 错误。



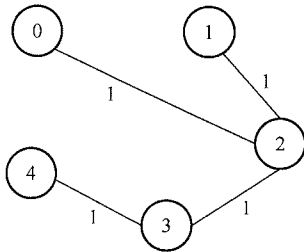
二、综合应用题

41. 【解析】

- 1) 不难发现, 该图是一个完全图, 本身就是一个强连通图, 因此只有一个强连通分量。
- 2) 函数 $f\{\}$ 的功能是求有向图 G 中顶点 v_i 的入度和出度之和, 因此函数 $f(\&G, 3)$ 返回的是邻接矩阵中第 3 行和第 3 列中非零元素的总数, 返回值为 8。
- 3) 该邻接矩阵对应的无向图是一个完全图, **Prim** 算法的时间复杂度为 $O(|V|^2)$, 它依赖于顶点数而非边数, 适用于求解边稠密的图的最小生成树, 因此应选用 **Prim** 算法。根据邻接矩阵画出该无向图, 求得其对应的最小生成树, 两图如下所示, 权值为 4。



无向图 G



对应的最小生成树

42. 【解析】

- 1) 本题可采用二叉树的后序遍历, 在函数中设置引用参数 `height`, 返回二叉树的高度。若一个结点的左、右子树的高度差为 1、0 或 -1, 则算法返回 1, 否则返回 0。
- 2) 算法的实现如下。

```
int HeightBalance (BiTNode *T, int height){
    //利用计算二叉树高度的算法判断二叉树的平衡性, 若平衡则返回 1, 否则返回 0
    if (T==NULL){          //空树高度为 0, 平衡
        height=0;
        return 1;
    }
    int lh,rh;
    if (HeightBalance (T->lchild,lh)==0) return 0; //左子树高度
    if (HeightBalance (T->rchild,rh)==0) return 0; //右子树高度
    height=(lh>rh)?1+lh:1+rh;          //取左、右子树高度大者再加 1
    if (lh-rh<=1&&lh-rh>=-1) return 1; //判平衡
    else return 0;
}
```

43. 【解析】

- 1) 数组 x 和 y 都按顺序访问, 空间局部性都较好, 但每个数组元素都只被访问一次, 所以没有时间局部性。
- 2) **Cache** 共有 $32B/16B = 2$ 行; 4 个数组元素占一个主存块 (现代计算机中 `float` 型一般为 32 位, 占 4B); 数组 x 的 8 个元素 (共 32B) 分别存放在以主存 40H 开始的 32 个单元中, 共占 2 个主存块, $x[0] \sim x[3]$ 在第 4 块 (00H~0FH 为第 0 块, 10H~1FH 为第 1 块, 以此类推, 40H~4FH 为第 4 块, 下同) 中, $x[4] \sim x[7]$ 在第 5 块中; 数组 y 的 8 个元素分别在主存的第 6 块和第 7 块中。因此, $x[0] \sim x[3]$ 和 $y[0] \sim y[3]$ 都映射到 **Cache** 的第 0 行; $x[4] \sim x[7]$ 和 $y[4] \sim y[7]$ 都映射到 **Cache** 的第 1 行, 如下表所示。因为 $x[i]$ 和 $y[i]$ ($0 \leq i \leq 7$) 总是映射到同一个 **Cache** 行, 相互淘汰对方, 所以每次都不命中, 命中率为 0。



Cache——主存地址	40H~5FH	60H~7FH
第 0 行	x[0]~x[3] (第 4 块)	y[0]~y[3] (第 6 块)
第 1 行	x[4]~x[7] (第 5 块)	y[4]~y[7] (第 7 块)

- 3) 若 Cache 改用 2 路组相联, 块的大小改为 8B, 则 Cache 共有 4 行, 每组 2 行, 共 2 组。两个数组元素占一个主存块。数组 x 占 4 个主存块, 数组元素 x[0]~x[1]、x[2]~x[3]、x[4]~x[5]、x[6]~x[7] 分别在第 8~11 块中 (与上题同理, 这里 00H~07H 为第 0 块, 08H~0FH 为第 1 块, 以此类推); 数组 y 占 4 个主存块, 数组元素 y[0]~y[1]、y[2]~y[3]、y[4]~y[5]、y[6]~y[7] 分别在第 12~15 块中, 映射关系如下表所示; 因为每组有两行, 所以 x[i] 和 y[i] ($0 \leq i \leq 7$) 虽然映射到同一个 Cache 组, 但可以存放到同一组的不同 Cache 行内, 因此不发生冲突。每调入一个主存块, 装入的 2 个数组元素中, 第 2 个数组元素总是命中, 故命中率为 50%。

Cache——主存地址	40H~4FH	50H~5FH	60H~6FH	70H~7FH
第 1 组	x[0]~x[1]	x[4]~x[5]	y[0]~y[1]	y[4]~y[5]
第 2 组	x[2]~x[3]	x[6]~x[7]	y[2]~y[3]	y[6]~y[7]

- 4) 数组 x 有 12 个元素, 则 x 共有 48B, 使得 y 从主存第 7 块开始存放, 即 x[0]~x[3] 在第 4 块中, x[4]~x[7] 在第 5 块中, x[8]~x[11] 在第 6 块中; y[0]~y[3] 在第 7 块中, y[4]~y[7] 在第 8 块中。因而, x[i] 和 y[i] ($0 \leq i \leq 7$) 就不会映射到同一个 Cache 行中, 映射关系如下表所示。每调入一个主存块, 装入 4 个数组元素, 第一个元素不命中, 后面 3 个总命中, 故命中率为 75%。

Cache——主存地址	40H~5FH	60H~7FH	80H~8FH
第 0 行	x[0]~x[3] (第 4 块)	x[8]~x[11] (第 6 块)	y[4]~y[7] (第 8 块)
第 1 行	x[4]~x[7] (第 5 块)	y[0]~y[3] (第 7 块)	—

44. 【解析】

- 1) 输入信号有 3 个:

- 标志信号 Zero, 用来说明条件转移指令的条件是否成立。
- 控制信号 Branch, 用来表示当前指令是否是分支跳转指令。
- 控制信号 Jump, 用来表示当前指令是否是无条件跳转指令。

- 2) 当顺序执行指令时, Zero 的值可能为 0 或 1, Branch=0, Jump=0。

当分支跳转指令执行且条件不满足时, Zero=0, Branch=1, Jump=0。

当分支跳转指令执行且条件满足时, Zero=1, Branch=1, Jump=0。

当无条件跳转指令执行时, Zero 的值可能为 0 或 1, Branch=0, Jump=1。

- 3) 因为是单周期 CPU, 每个时钟周期执行一条指令, 所以每个时钟周期 PC 值都会被更新一次, 因此 PC 无须写“使能”信号控制。

- 4) 跳转指令的功能是将本条指令中的 target 字段 (0~25 位) 拼接上旧 PC 值的高 4 位, 并送入 PC, 高 4 位不变, 相当于在某个固定范围内转移, 因为 target 字段占 26 位, 新 PC 值只有低 26 位是可以设置的, 因此在可转移的目标范围内共包含 2^{26} 条指令。

- 5) 输入一个 16 位立即数 imm16 后, 经过 SignExt 部件后变成了一个 30 位的数, 而且 imm16 表示的是条件转移指令的偏移量, 一定是一个有符号数。因此 SignExt 起到了符号扩展的作用。



45. 【解析】

本题的同步和互斥关系分析如下：① 生产者 A、B 和消费者 C 之间，不能同时将产品入库和出库，所以仓库是一个临界资源。② 两个生产者之间必须同步，当生产的产品 A 和 B 的件数之差大于或等于 m 时，生产者 A 必须等待；当件数之差小于或等于 $-n$ 时，生产者 B 必须等待。③ 生产者和销售者之间也必须同步，只有当生产者生产出产品并入库后，销售者才能进行销售。

为了互斥地入库和出库，需要为仓库设置一个初始值为 1 的互斥信号量 mutex ；为了使生产的产品件数满足 $-n \leq A \text{ 的件数} - B \text{ 的件数} \leq m$ ，需设置两个信号量，其中 SAB 表示当前允许 A 生产的产品数量，其初始值为 m ， SBA 表示当前允许 B 生产的产品数量，其初始值为 n ；此外，还需设置一个初始值为 0 的资源信号量 S ，它对应仓库中的产品数量。

具体的同步算法如下：

```
semaphore SAB=m;           //限制产品 A 的生产数量
semaphore SBA=n;           //限制产品 B 的生产数量
semaphore S=0;             //仓库中的产品量
semaphore mutex=1;         //互斥访问仓库
PA(){
    while(1){
        wait(SAB);
        produce a product A
        signal(SBA);
        wait(mutex);
        add the product A to the storehouse
        signal(mutex);
        signal(S);
    }
}
PB(){
    while(1){
        wait(SBA);
        produce a product B
        signal(SAB);
        wait(mutex);
        add the product B to the storehouse
        signal(mutex);
        signal(S);
    }
}
PC(){
    while(1){
        wait(S);
        wait(mutex);
        take a product A or B from storehouse
        signal(mutex);
        sell the product
    }
}
```

46. 【解析】

1) 013F F35AH 的页目录号为 4H，对应的页表项中 Page_Size 为 0，表示对应的页框大小为 4KB，采用二级页表，至少需要依次查找页目录表、页表和对应的数据，需要 3 次访存。

015F 123DH 的页目录号为 5H，对应的页表项中 Page_Size 为 1，表示对应的页框大小为 4MB，采用一级页表，至少需要依次查找页表和对应的数据，需要 2 次访存。

2) 根据第一问的分析，015F 123DH 对应的页号为 05H，采用一级页表，分配的是 4MB 的页

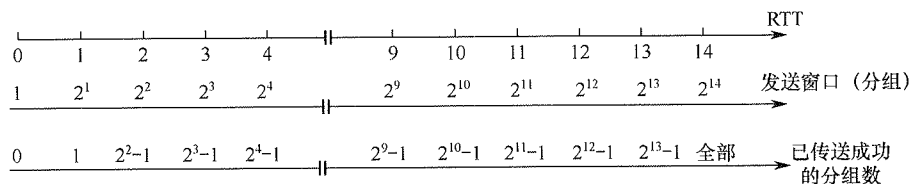


面, 对应的页框号为 163H, 故物理地址为 58DF 123DH。

- 3) 00D4 0866H 的页目录号为 3H, 对应的页表项中 Page_Size 为 0, 根据 FIFO 置换算法, 替换页目录号为 2H 所对应的页框, 将 254H 号页框分配给 3H 号页面, 则对应的物理地址为 9514 0866H。
- 4) 2EBC A234H 对应的页目录号为 BAH, 则该表项在页目录表中的偏移量为 $BAH \ll 2$, 为 2E8H, 故该表项的物理地址为 $7F65\ 4000H + 2E8H = 7F65\ 42E8H$ 。

47. 【解析】

- 1) 在本题中, $K = 2^{10}$, $M = 2^{20}$; 分组长度 = $1KB = 2^{10}B$; 发送窗口大小 = $1MB = 2^{10}KB = 2^{10}$ 个分组。在下图中, 最上面 RTT 坐标的数字 i 表示第 i 个 RTT 结束的时刻。例如, 数字 1 表示收到了对第 1 个分组的确认, 于是发送窗口就增大到 2, 最下面的数字表示已发送成功的分组数。可见, 发送窗口大小 = $2^{10}KB = 1MB$, 经过 10 个 RTT。



发送窗口以及已传送成功的分组数与 RTT 的关系

- 2) 由图可知, 当第 10 个 RTT 结束时, 已发送成功的分组数是 $2^{10} - 1$ 。分析如下: 当第 10 个 RTT 结束时, 发送窗口为 1MB, 已发送成功的数据量约为 1MB (准确地说是 $1MB - 1KB$, 因此还需发送 $9MB + 1KB$)。每经过一个 RTT, 发送窗口就加倍。因此, 当第 11 个 RTT 结束时, 又发送成功了 1MB; 当第 12 个 RTT 结束时, 又发送成功了 2MB; 当第 13 个 RTT 结束时, 又发送成功了 4MB, 此时与 9MB 还相差 2MB, 还要经过一个 RTT; 在第 14 个 RTT 中, 剩下的全部数据约 2MB (准确地说是 $2MB + 1KB$) 都发送完毕, 因此 10MB 文件发送成功需要 14 个 RTT。当第 14 个 RTT 开始时, 发送窗口是 $2^{13}KB = 2^{23}B$ 。选用窗口扩大选项后, 窗口的最大值是 $(2^{30} - 1)B$, 因此 TCP 扩大的窗口是够用的。
- 3) 14 个 RTT 占用的时间是 $14 \times 50ms = 0.7s$, 有效吞吐率 = $10MB \div 0.7s = 10 \times 2^{20} \times 8bit \div 0.7s = 119.8 \times 10^6 b/s = 119.8Mb/s$ (在本题中, 要注意表示文件大小时 $M = 2^{20}$, 表示速率时 $M = 10^6$)。链路带宽的利用率 = $119.8Mb/s \div 1000Mb/s = 11.98\%$ 。



关注公众号【考研小舟】
免费考研资料&无水印PDF



黑白&彩印都是5分/页, 满4.9全国包邮
下单备注: 考研小舟, 送草稿纸



全国硕士研究生入学统一考试

计算机科学与技术学科联考

计算机专业基础综合考试模拟试卷(二) 参考答案

一、单项选择题(第 01~40 题)

- | | | | | | | | |
|-------|-------|-------|-------|-------|-------|-------|-------|
| 01. B | 02. C | 03. B | 04. B | 05. C | 06. D | 07. B | 08. B |
| 09. C | 10. A | 11. B | 12. D | 13. C | 14. B | 15. B | 16. B |
| 17. C | 18. A | 19. B | 20. D | 21. A | 22. A | 23. B | 24. C |
| 25. B | 26. C | 27. A | 28. A | 29. A | 30. C | 31. C | 32. A |
| 33. B | 34. D | 35. C | 36. D | 37. B | 38. A | 39. C | 40. A |

01. B. 【解析】 本题考查时间复杂度。

本算法通过递归来计算二叉树的最大深度。因为通过递归遍历了二叉树中的所有结点, 所以该算法的时间复杂度与该二叉树中的结点数成线性关系, 时间复杂度为 $O(n)$ 。

02. C. 【解析】 本题考查栈的操作。

一棵二叉树的前序序列和中序序列分别对应一种合法的进栈次序和出栈次序, 又由于前序序列和中序序列能唯一地确定一棵二叉树, 因此本题等价于让 5 个字母“o o o p s”按顺序进栈, 求有多少种不同的出栈顺序仍可得到“o o o p s”。分析如下: 进栈序列为“o o o p s”, 出栈序列为“o o o p s”, 最后两个字符 p s 相同, 意味着“o o o”序列进栈后全部出栈。“o o o”全部进栈再出栈, 有 1 种; 前两个字符“o o”进栈再出栈, 有 2 种; 前一个字符“o”进栈再出栈, 有 2 种, 因此共有 $1 + 2 + 2 = 5$ 种。

【另解】 n 个数 $(1, 2, 3, \dots, n)$ 依次进栈, 可能有 $C_{2n}^n / (n+1) = (2n)! / [(n! \times n!) / (n+1)]$ 个不同的出栈序列, 因此“o o o”对应 5 种不同的出栈序列。

03. B. 【解析】 本题考查二叉树的性质。

本题从正面分析较为困难, 可采用逆向法, 从大到小逐个分析选项。假设第 7 层有 64 个结点, 则第 6 层至少有 32 个结点, 以此类推, 上层依次至少有 16, 8, 4, 2, 1 个结点, 结点总数至少是 $1 + 2 + 4 + 8 + 16 + 32 + 64 = 127$, 选项 D 排除。假设第 7 层有 49 个结点, 则第 6 层至少有 25 个结点, 以此类推, 上层依次至少有 13, 7, 4, 2, 1 个结点, 结点总数至少是 $1 + 2 + 4 + 7 + 13 + 25 + 49 = 101$, 选项 C 排除。假设第 7 层有 48 个结点, 则第 6 层至少有 24 个结点, 以此类推, 上层依次至少有 12, 6, 3, 2, 1 个结点, 结点总数至少是 $1 + 2 + 3 + 6 + 12 + 24 + 48 = 96 < 100$, 因此一棵有 100 个结点的二叉树, 第 7 层最多可以有 48 个结点, 选项 B 正确。

04. B. 【解析】 本题考查完全二叉树的性质。

深度为 7 的满二叉树的叶结点数为 $2^6 = 64$, 深度为 8 的满二叉树的叶结点数为 $2^7 = 128$ 。因此,

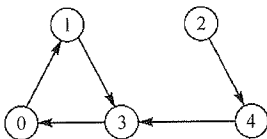


若完全二叉树有 64 个叶结点, 则其最大深度为 8(在深度为 7 的满二叉树的基础上增加 1 个结点)。

05. C. 【解析】本题考查二叉树的性质。

选项 A 正确, 该结点的中序后继结点是其右子树的最左结点, 因此没有左孩子, 中序前驱结点是其左子树的最右结点, 因此没有右孩子。选项 B 正确, 根据题意, x 一定是 y 的左子树中最左下的一个结点。选项 C 错误, 从中序线索树的最小值结点处不断查找后继结点, 可以遍历完树中的所有结点。选项 D 正确, 因为 x 是一个叶结点, x 要么是 y 的右孩子 (y 在 x 的左边, y 的数值比 x 的小), 要么是 y 的左孩子 (y 在 x 的右边, y 的数值比 x 的大)。又因为 x 没有左子树和右子树, 所以 y 即是 x 的前驱或后继结点, 即 y 的数值要么是树中大于 x 的值的的最小关键字, 要么是树中小于 x 的值的最大关键字。

06. D. 【解析】本题考查图的邻接矩阵和连通性。



这是有向图, 环中的元素必须互相可达, 而 0, 1, 3 就是一个环, 选项 I 错误。一个强连通分量中的每个顶点都需要与其他顶点互相可达, 而 4 不能到达 2, 所以 2 和 4 分属两个强连通分量, 有 0-1-3, 4, 2 共三个强连通分量, 选项 II 错误。因为有环, 所以不存在拓扑序列, 选项 III 错误。因此, 选择选项 D。

07. B. 【解析】本题考查关键路径。

先对该图进行拓扑排序, 拓扑排序的一个合法序列为 A, B, C, D, E, F。按照该顺序列出下表, 得到各个活动的最早开始时间和最迟开始时间。

	A	B	C	D	E	F
Ve	0	5	5	7	10	13
VI	0	11	5	9	10	13

然后, 分别计算各条边的时间余量如下 (计算方法为, 边终点的最迟开始时间 - 边起点的最早开始时间 - 边的权值):

AC: $5 - 0 - 5 = 0$; AB: $11 - 0 - 5 = 6$; AD: $9 - 0 - 7 = 2$; CD: $9 - 5 - 1 = 3$;

CE: $10 - 5 - 5 = 0$; BF: $13 - 5 - 2 = 6$; DF: $13 - 7 - 4 = 2$; EF: $13 - 10 - 3 = 0$ 。

因此, 时间余量最大的活动的的时间余量是 6。

08. B. 【解析】本题考查红黑树的性质。

一棵红黑树可以是一棵全黑的满二叉树, 选项 A 显然错误。根据红黑树“不红红”的性质可知, 选项 B 正确。平衡二叉树的平衡要求比红黑树更严格, 因此平衡二叉树的高度更低, 查找效率更优, 选项 C 和选项 D 错误。

09. C. 【解析】本题考查 B 树的删除操作。

删除关键字 71 后, 发生了兄弟不够借的情况, 此时结点 55 和 60 合并, 需要继续向上调整,



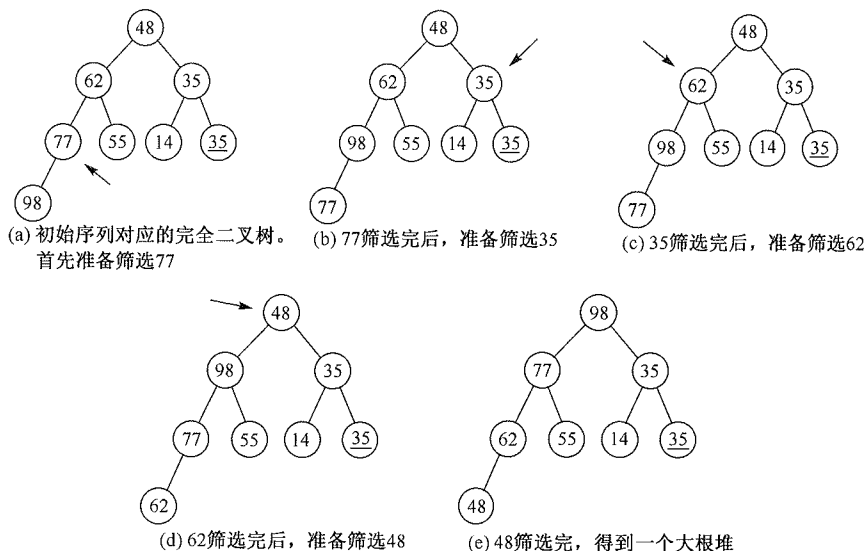
结点 47 和 20 合并，合并后的树形如选项 C 所示。

10. A. 【解析】本题考查快速排序的特点。

若待排序序列初始有序（默认升序），最后一个元素是最大值。划分时，所有其他元素都小于基准，导致左子序列包含所有元素（ $n-1$ 个），右子序列为空。递归处理左子序列时，同样选择子序列的最大值为基准，划分极不平衡，时间复杂度为 $O(n^2)$ ，此时为最坏情况。

11. B. 【解析】本题考查初始堆的构造过程。

首先对以第 $\lfloor n/2 \rfloor$ 个结点为根的子树筛选，使该子树成为堆，然后向前依次对以各结点为根的子树进行筛选，直到筛选到根结点。序列 {48, 62, 35, 77, 55, 14, 35, 98} 建立初始堆的过程如下图所示。



如图所示，(a)调整结点 77，交换 1 次；(b)调整结点 35，不交换；(c)调整结点 62，交换 2 次；(d)调整结点 48，交换 3 次；(e)为大根堆。因此，上述序列建初始堆共交换元素 6 次。

12. D. 【解析】本题考查影响 CPU 执行时间的因素。

更快速的处理器可以加快指令的执行速度，选项 I 正确。增加同类处理器的个数，可以提高程序的并行执行能力，选项 II 正确。优化后的代码可以更有效地利用处理器的资源，选项 III 正确。内存访问通常是程序执行速度的重要瓶颈，因此选项 IV 可以显著提升程序的执行速度。

13. C. 【解析】本题考查算术移位和 OF 标志位。

$[x/2 + 2y]_{\text{补}} = [x]_{\text{补}} \gg 1 + [y]_{\text{补}} \ll 1 = 0100\ 0100 \gg 1 + 1101\ 1100 \ll 1 = 0010\ 0010 + 1011\ 1000 = 1101\ 1010 = \text{DAH}$ 。 x 右移，没有溢出或精度损失； y 为负数，左移后符号位仍为 1，没有溢出；从最后一步加法操作来看，一个正数和一个负数相加，必然不会溢出，因此 OF 位为 0。

14. B. 【解析】本题考查 IEEE 754 单精度浮点数的对阶和舍入。

在 IEEE 754 单精度浮点数的加减运算中，若对阶操作得到的两个阶码之差的绝对值 $|\Delta E|$ 等于 24，则表示阶小的那个数的尾数右移 24 位，虽然尾数加减运算的结果的前 24 位直接取阶大的那个



数的相应位,但由于可以保留附加位,阶小的那个数右移后的尾数可能会在舍入时向前面一位进1。例如, $1.00\dots01\times2^1 + 1.10\dots00\times2^{-23} = 1.00\dots01\times2^1 + 0.00\dots0011\times2^1 = 1.00\dots0111\times2^1$, 其中最后两位为保留的附加位,最终需要根据这两位进行舍入,若舍入后的结果为 $1.00\dots10\times2^1$, 则运算结果并不等于阶大的那个数。若 $|\Delta E|$ 等于 25, 则保留的附加位中最左边第一位一定是 0, 采用就近舍入时, 这些附加位被完全丢弃。因此, 当 $|\Delta E|$ 大于或等于 25 时, 运算结果直接取阶大的那个数。

▲注意: IEEE 754 提供了以下 4 种可选的舍入模式。

- 1) 就近舍入: 舍入为最近的可表示数。当运算结果是两个可表示数的非中间值时, 实际上是“0 舍 1 入”方式; 当运算结果正好在两个可表示数的中间时, 则将结果选为偶数。
- 2) 正向舍入: 朝数轴 $+\infty$ 方向舍入, 即取右边最近的可表示数。
- 3) 负向舍入: 朝数轴 $-\infty$ 方向舍入, 即取左边最近的可表示数。
- 4) 截断法: 直接截取所需位数, 丢弃后面的所有位, 这种舍入处理最简单。对正数或负数来说, 都是取更接近原点的那个可表示数, 是一种趋向原点的舍入。

15. B。【解析】本题考查 DRAM 芯片的性质和扩展方式。

64M×8 位 DRAM 芯片结构是三维的, $64M = 2^{26}$, 共有 $2^{13} = 8192$ 行、 $2^{13} = 8192$ 列, 每个行列的交界处都包括 8bit。在 DRAM 芯片的行缓冲中, 包含一行中的所有数据, 因此一个 DRAM 芯片的行缓冲大小为 $8192\times8\text{bit} = 8\text{KB}$, 8 个芯片的行缓冲总大小为 64KB, 该内存条经过位扩展且支持突发传输方式, 因此必然采用了多模块交叉编址, DRAM 芯片采用了行列地址复用技术, 因此每增加 1 个引脚, 容量至少增加 4 倍。选项 I、II、III 均正确。存储器总线宽度为 64 位, 因此数据寄存器 (MDR) 的宽度也是 64 位, 选项 IV 错误。

16. B。【解析】本题考查存储芯片的扩展。

RAM 区的地址范围为 0000 1000 0000 0000 0000~1111 1111 1111 1111, 因此 RAM 区的大小为 $31\times32\text{KB}$, 有 $(31\times32\text{KB})/16\text{KB} = 62$ 。

17. C。【解析】本题考查 Cache 缺失时间的计算。

Cache1 比 Cache2 的缺失率更高, 但 Cache1 的单次缺失开销可能小于 Cache2 的单次缺失开销, 因此总缺失开销不一定比 Cache2 的大。提高 Cache 的关联度通常能降低 Cache 的缺失率, 但也不是绝对的。下面举例说明: 2 路组相联 Cache 的组数是直接映射 Cache 的行数的一半, 因此可以找到一个地址序列 A, B, C, 使得 A 映射到某个 Cache 行, B 和 C 同时映射到另一个 Cache 行, 且 A, B, C 映射到同一 Cache 组。因此, 若访问的地址序列为 A, B, C, A, B, C, A, B, C, ..., 则直接映射的命中情况为 miss/miss/miss/hit/miss/miss/hit/miss/miss/..., 命中率为 33.3%; 对于 2 路组相联映射, 因为 A, B, C 映射到同一组, 每组只有 2 行, 采用 LRU 替换算法, 所以每个数据刚调出 Cache 就又被访问到, 每次都是 miss, 命中率为 0。选项 C 正确。Cache 缺失所引起的时间开销和 Cache 替换算法也有关系。

18. A。【解析】本题考查各种机器指令的功能。

条件跳转指令根据条件是否满足来决定是否跳转, 不一定会改变程序的执行顺序, 其余 5 类指令一定会改变程序的执行顺序。无条件跳转指令会跳转到转移目标地址; 过程调用指令会使程序跳转到相应的被调用函数; 过程返回指令会让程序返回到调用过程; 自陷指令会让程



序转到相应的内核函数；中断返回指令会让中断处理程序返回到被中断过程。

19. B. 【解析】本题考查控制冒险的产生原因。

各类转移指令（包括调用、返回指令等）的执行，以及异常和中断的出现，都会改变指令执行顺序，因此都可能引发控制冒险，在上述指令序列中，第 4 条分支指令和第 6 条无条件跳转指令都有可能引起分支控制冒险，因此共有 2 条指令会产生分支控制冒险。

20. D. 【解析】本题考查多处理器系统的特性。

对于多处理器系统，可以根据存储访问时间是否一致，分为一致性内存访问（UMA，Uniform Memory Access）和非一致性内存访问（NUMA，Non-Uniform Memory Access）两类。UMA 结构是指每个处理器对所有存储单元的访问时间是一致的；NUMA 结构是指处理器对不同存储单元的访问时间可能不一致，访问时间与存储单元的位置有关，选项 D 错误。

21. A. 【解析】本题考查总线的定时方式。

在异步定时方式中，没有统一的时钟，也没有固定的时间间隔，完全依靠传输双方相互制约的“握手”信号来实现定时控制。而异步传输方式一般用在工作速度差异较大的设备之间，I/O 接口和打印机之间的速度差异较大，应采用异步传输方式来提高效率。异步定时方式能保证两个工作速度相差很大的部件或设备之间可靠地进行信息交换。

▲注意：在工作速度不同的设备之间进行数据传输时，应选用异步传输方式，虽然采用同步控制也可进行数据传输，但是不能发挥快速设备的高速性能，因为速度快的设备总要等待速度慢的设备。

22. A. 【解析】本题考查中断允许触发器。

开中断和关中断对应中断允许触发器；中断屏蔽对应中断屏蔽寄存器；中断请求对应中断请求寄存器；中断服务程序的入口地址对应中断向量寄存器。

23. B. 【解析】本题考查系统调用。

系统调用是操作系统内核提供给用户程序请求操作系统服务的接口。无论是 Windows、Linux 还是 UNIX 操作系统，只要在同一台计算机上，就拥有相同的指令集体系结构，它们在底层都需要通过相同的机制来执行系统调用，以便与硬件进行交互。当硬件改变时，系统调用指令一般来说也会相应地改变，因此选择选项 B。

24. C. 【解析】本题考查进程的状态。

执行 I/O 指令、系统调用、修改页表都需要切换到内核态，只要通用寄存器中存放的变量是普通用户进程可以访问到的，通用寄存器清零就不需要切换到内核态执行。

25. B. 【解析】本题考查进程的状态与转换。

若正在执行的进程的时间片用完，则“主动”调用程序转入就绪态。进程的阻塞和唤醒是由 block 和 wakeup 原语实现的，block 原语是由被阻塞进程自我调用实现的，而 wakeup 原语则是由一个与被唤醒进程相合作或其他相关的进程调用实现的，选项 I 和 II 正确。I/O 操作结束不会直接导致一个进程从就绪态变为运行态，只是当有等待该设备的进程时，I/O 操作



结束时会把该进程从阻塞态变为就绪态，选项 III 错误。一个进程时间片到了后，将会从运行态变为就绪态，选项 IV 错误。只有当运行中的进程请求某个资源或等待某个事件时，才会转入阻塞态，因此不可能直接从就绪态转到阻塞态，选项 V 正确。

26. C。【解析】本题考查多线程的特点。

线程最直观的理解就是“轻量级实体”，引入线程后，线程成为 CPU 独立调度的基本单位，进程是资源拥有的基本单位。引入多线程是为了更好地并发执行，键盘属于慢速外设，它无法并发执行（整个系统只有一个键盘），而且键盘采用人工操作，速度很慢，因此完全可以使用一个线程来处理整个系统的键盘输入。符合多线程系统的特长的任务应该符合一个特点，即可以切割成多个互不相干的子操作，由此可知，选项 A 中矩阵乘法运算得到的矩阵上的每个元素的操作都可作为一个子操作分割开；选项 B 中 Web 服务器要应对多个用户提出的 HTTP 服务请求，当然也符合多线程系统的特长；选项 D 已说明用不同线程来处理用户的操作。

27. A。【解析】本题考查时间片轮转调度算法。

选择时间片大小时，一般应考虑如下三个因素：系统对响应时间的要求、就绪队列中进程的数量、系统的处理能力。各个进程所需的运行时间是无法事先预知的。

28. A。【解析】本题考查进程的前驱图。

在执行 $S3:c=a-b$ 之前，必须先计算出 a 和 b 的值，因此 $S1$ 和 $S2$ 必须在 $S3$ 之前执行。在执行 $S4:w=c+1$ 之前，必须先计算出来 c 的值，因此 $S4$ 必须在 $S3$ 之后执行。

29. A。【解析】本题考查缺页中断的计算。

进程的工作集是 2 个页框，其中一个页框始终被程序代码占用，因此可供数据使用的内存空间只有 1 个页框。在虚空间中以行为主序存放，每页存放 128 个数组元素，因此每行占一页。程序 1 访问数组的方式为先行后列，每次访问都针对不同的行，因此每次都产生缺页中断，共 128×128 次。程序 2 访问数组的方式是先列后行，每次访问不同的行时会产生缺页中断，共 128 次。

30. C。【解析】本题考查文件共享。

引入索引节点前，文件地址等信息都存放在各自的目录项中，而文件各自的目录项彼此独立，因此文件增加的部分不能被共享；引入索引节点后，文件的地址信息存放在被共享的索引节点中，因此文件增加的部分也能被共享，选项 A 错误。count 不表示本索引节点被打开的次数，而表示链接到本索引节点的目录项的数量，选项 B 错误。采用符号链接时，只有文件的拥有者才拥有指向其索引节点的指针，其他用户只有该文件的路径名，选项 D 错误。

31. C。【解析】本题考查 I/O 控制方式的功能。

在程序直接控制方式下，设备驱动程序完成用户程序的 I/O 请求后才结束，这种情况下用户进程在 I/O 过程中不会被阻塞，在内核态下进行 I/O 处理。在中断控制方式下，设备驱动程序启动第一次 I/O 操作后，将调出其他进程执行，而当前用户进程则被阻塞。在 DMA 控制方式下，设备驱动程序对 DMA 控制器初始化后，发送“启动 DMA 传输”命令，外设开始进行 I/O 操作并将在外设和主存之间传输数据，同时 CPU 执行处理器调度程序，转至其他进程执行，当前用户



进程被阻塞。综上所述, 选项 II、III 正确。

32. A. 【解析】本题考查磁盘高速缓存的概念。

磁盘高速缓存是指在内存中为磁盘盘块设置的一个缓冲区, 缓冲区中保存了某些磁盘块的副本, 选项 A 错误。当出现访问磁盘的请求时, 先查看磁盘高速缓存, 看所请求的盘块内容是否已在磁盘高速缓存中, 若在, 则可直接从磁盘高速缓存中获取; 若不在, 则需要启动磁盘将所需的盘块内容读入。设计磁盘高速缓存时, 需要考虑: ① 如何将磁盘高速缓存中的数据传输给请求进程; ② 采用什么样的置换策略; ③ 已修改的盘块数据何时写回磁盘。

33. B. 【解析】本题考查分组交换的时延计算方法。

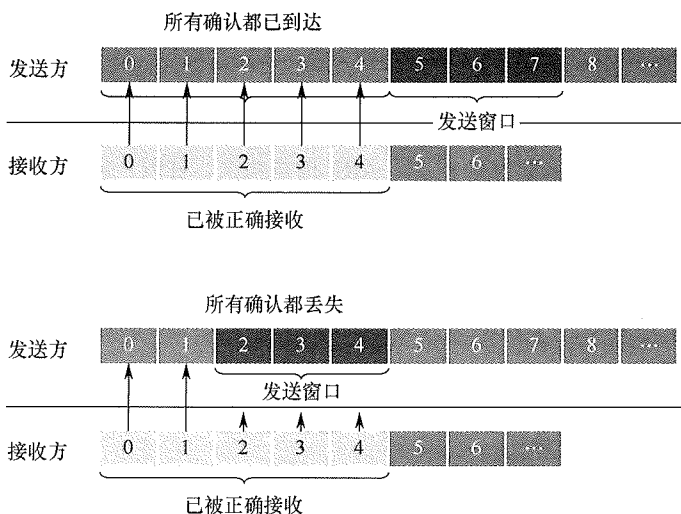
各设备的发送速率为 s , 分组的数据载荷长度为 x , 总时延 $D = [(x + 20)/s] \times (0.2\text{MB}/x) + (x + 20)/s$ 。前半部分是分组数量和一个分组的发送时延的乘积, 得到所有分组的总发送时延; 后半部分是第 2 个转发结点转发一个分组的发送时延, 两者相加得到总时延。要求 x 的值使得 D 最小, 可以求 D 对 x 的导数并令导数值为 0, 得到 $x \approx 2048\text{B}$ 。

34. D. 【解析】本题考查香农公式概念的理解。

在实际传输环境中, 信噪比不可能做到任意大。信号传输功率是受限的, 任何电子设备的噪声也不可能做到任意小 (任何电子设备都有其固有的噪声)。因此, 信噪比不可能做到任意大, 选项 A 错误。对于一定的信噪比, 码元的传输率越小, 传输每个码元所用的时间越长, 相对来说抗噪声能力就越强, 因此判决错误的概率会减小, 选项 B 错误。若增大信噪比, 则码元的传输率就可提高而不至于使判决错误的概率增大, 选项 C 错误。

35. C. 【解析】本题考查 TCP 的滑动窗口机制。

接收方下一个期望收到的序号是 5, 表明序号到 4 为止的分组都已收到, 假定所有确认都已到达发送方, 这时发送窗口最靠前, 范围是 [5, 7]; 假定所有确认都丢失, 发送方都没有收到这些确认, 这时发送窗口最靠后, 范围是 [2, 4]。因此, 发送窗口可能是 [2, 4], [3, 5], [4, 6], [5, 7] 中的任意一个, 两种边界情况如下图所示。

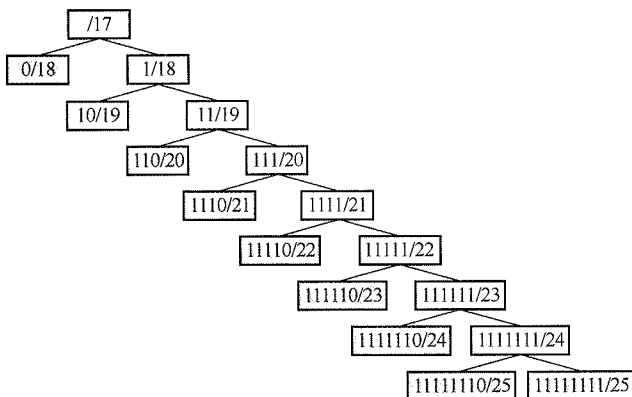


36. D. 【解析】本题考查对 CSMA/CD 协议最短帧长的理解。

CSMA/CD 协议要求最短帧长的发送时延必须大于或等于一个数据帧的往返时延 RTT，主机 H1 发送数据帧的时间为 $64 \times 8 \text{B} / (100 \text{Mb/s}) = 5.12 \mu\text{s}$ ，因此从主机 H1 到主机 H2 的单程时延最大为 $5.12 \mu\text{s} \div 2 = 2.56 \mu\text{s}$ ，这部分时延由两部分组成，即通过集线器引起的时延和链路上的传播时延。通过单个集线器的时延为 $0.48 \mu\text{s}$ ，通过两个集线器的时延则为 $0.96 \mu\text{s}$ 。这样，链路上的最大单程传播时延就为 $2.56 \mu\text{s} - 0.96 \mu\text{s} = 1.6 \mu\text{s}$ ，因此最远距离为 $1.6 \mu\text{s} \times 200 \text{m} / \mu\text{s} = 320 \text{m}$ 。

37. B. 【解析】本题考查 IP 地址的划分方法。

将某个“/17”地址块划分为 9（不多也不少）个子块，且要求可能的最小子块。因此，刚开始划分出的 IP 地址空间要尽可能大。可以画出如下图所示的二叉树来划分 IP 地址空间，由图可知最小的 IP 地址空间的网络前缀占 25 位，包含的 IP 地址数量为 $2^7 = 128$ 。



38. A. 【解析】本题考查移动 IP。

固定主机 B 发送的 IP 数据报的源 IP 地址为主机 B 的 IP 地址，目的 IP 地址为移动主机 A 的永久 IP 地址，归属代理收到该 IP 数据报后，会将目的 IP 地址改变为外地代理的转交地址。

39. C. 【解析】本题考查 IP 数据报的总长度字段。

IP 数据报的总长度字段占 2B，即一个 IP 数据报的最大长度为 65535B，IP 首部的最小长度为固定首部的 20B，因此 TCP 报文段的总长度最大为 65515B。TCP 报文段的总长度显然有一个最小值，即当 TCP 报文段的数据字段的长度为 0 时，TCP 报文段的总长度为 20B。

40. A. 【解析】本题考查 DHCP 发现报文、ARP 请求报文、HTTP 报文和 IGMP 报文。

DHCP 发现报文封装在广播 IP 数据报中，使用广播 IP 地址 255.255.255.255，之后封装在以太网广播帧中，使用广播 MAC 地址 FF-FF-FF-FF-FF-FF。ARP 请求报文直接封装在以太网广播帧中，使用广播 MAC 地址 FF-FF-FF-FF-FF-FF。HTTP 请求报文封装在单播 IP 数据报中，之后封装在以太网单播帧中。IGMP 报文封装在多播 IP 数据报中，之后封装在以太网多播帧中，使用多播 MAC 地址。

二、综合应用题

41. 【解析】

1) 类比满二叉树，满二叉树的第 i 层有 2^{i-1} 个结点，各层结点数量的排列是以 2 为公比的等



比数列，满 k 叉树的各层结点数量的排列是以 k 为公比的等比数列，因此满 k 叉树的第 i 层有 k^{i-1} 个结点。

- 2) 不难发现，在满 k 叉树中编号为 p 的结点的第 $k-1$ 个孩子的编号一定是 pk ，因此若编号为 p 的结点不是叶结点，则其 k 个孩子的结点编号在顺序区间 $[pk+2-k, pk+1]$ 内，其第 i 个孩子的编号为 $pk-k+i+1$ 。
- 3) 由上述分析可知， $p=1$ 时为根结点，无父结点；否则，编号为 p 的结点的父结点的编号为 $\lfloor (p+k-2)/k \rfloor$ 。
- 4) 根据满 k 叉树的性质，若某个结点不是其父结点的最后一个孩子，则其一定有右兄弟。假设结点 p 不是根结点，其父结点的编号为 q ，则根据 2) 中的分析不难得出，若结点 p 是结点 q 的最后一个孩子，则 $p=kq+1$ ，因此当 $(p-1)\%k \neq 0$ 时，表示结点 p 有右兄弟，其右兄弟的编号为 $p+1$ 。若结点 p 是根结点，则显然没有右兄弟，此时 $(p-1)\%k = 0$ 。

42. 【解析】

- 1) 借助快速排序的划分思想，用 i 和 j 分别依次遍历数组中下标为偶数和奇数的整数。当 $A[i]$ 为奇数且 $A[j]$ 为偶数时，交换它们，使得偶数交换到偶数下标的位置，奇数交换到奇数下标的位置。如此继续，直至遍历完数组中的所有元素。

- 2) 算法的实现如下：

```
void rearrangeSort(Datalist &L){
    int i=0,j=1;           //i 是偶数位, j 是奇数位
    int tmp;
    while(i<L.length&& j<L.length){
        while(i<L.length&&L.data[i]%2==0) i=i+2; //偶数通过, 找奇数
        while(j<L.length&&L.data[j]%2==1) j=j+2; //奇数通过, 找偶数
        if(i<L.length&&j<L.length){ //交换奇偶数
            tmp=L.data[i]
            L.data[i]=L.data[j];
            L.data[j]=tmp;
            i=i+2;j=j+2;
        }
    }
}
```

- 3) 该算法只需要扫描一次顺序表，因此时间复杂度为 $O(n)$ 。

43. 【解析】

- 1) $x=68=0100\ 0100B=44H$, $y=80=0101\ 0000B=50H$ ，故寄存器 A 和 B 中的内容分别是 44H 和 50H。
- 2) $x+y=0100\ 0100+0101\ 0000=(0)1001\ 0100=94H$ ，故寄存器 C 中的内容为 94H，对应的真值为 148，运算结果正确。加法器最高位的进位 Cout 为 0，结果不为 0，零标志 $ZF=0$ ；进位标志 $CF=Cout=0$ 。
- 3) $x-y=x+[-y]_{补}=0100\ 0100+1011\ 0000=(0)1111\ 0100=F4H$ ，故寄存器 D 中的内容为 F4H，对应的真值为 244。由于相减的结果为负数，因此运算结果不正确。加法器最高位的进位 Cout 为 0，结果不为 0，零标志 $ZF=0$ ，借位标志 $CF=Cout\oplus 1=0\oplus 1=1$ 。
- 4) 当无符号整数相加时，若加法器最高位进位 $Cout=1$ ，则表示结果大于最大可表示的数；当无符号整数相减时，若加法器最高位进位 $Cout=1$ ，则表示被减数大于减数，反之表示被减数小于减数。因此，当无符号整数相加时， $CF=Cout$ ，表示进位；当无符号整数相减

时, 将最高位进位 Cout 取反来作为借位标志 CF, 即 $CF = \overline{Cout}$, $CF = 1$ 表示有借位。

- 5) 无符号整数一般用来表示地址 (指针) 信息, 当两个地址相加的结果大于最大地址而丢掉最高位进位时, 相当于取模运算, 因此通常不需要判断其运算结果是否溢出。

44. 【解析】

- 1) 根据题中汇编指令的格式可知, 该指令系统中的立即数只能占 16 位, 因此无法在一条指令中直接给出一个 32 位的地址, 只能通过其他的方法。
- 2) 低位。高位。“或”操作。因为指令中的立即数为 16 位, 所以一个 32 位的地址无法作为立即数送到寄存器中。一种实现方案是, 通过将地址 A 的高 16 位和低 16 位分别作为两条指令的立即数, 用“或”操作将它们合并到一个 32 位寄存器中, 这样才能还原得到 32 位的地址, 因此第 3 条取数指令 lw 的偏移量为 0。
- 3) 另一种实现方案是, 取数指令 lw 的偏移量是 A 的低位部分 A_lower, 由于 lw 指令计算主存地址时对偏移量采用的是符号扩展, 因此要使高 16 位的最终结果为 A_upper, 必须对 A_upper 做如下调整: 若 A_lower 的最高位 (视为低 16 位的符号位) 是 0, 则 A_upper_adjusted=A_upper, 这样 A_lower 符号扩展后的高 16 位为全 0, 与高 16 位 A_upper 相加后, 高 16 位还是 A_upper; 若 A_lower 的最高位是 1, 则 A_lower 符号扩展后的高 16 位为全 1, 此时 A_upper_adjusted 应满足 $FFFFH + A_upper_adjusted = A_upper$ 。而 $FFFFH + A_upper + 1 = A_upper$ (最高位的进位被丢弃), 因此 $A_upper_adjusted = A_upper + 1$ 。

45. 【解析】

- 本题中有 4 组同步关系: ① 零件 A 的数量不能大于 1 个; ② 零件 B 的数量不能大于 3 个; ③ 零件 A 的数量必须要有 1 个才能开始装配; ④ 零件 B 的数量必须要有 3 个才能开始装配。

```
Semaphore empty_a = 1;           //保证零件 A 的数量不超过 1 个
Semaphore empty_b = 3;           //保证零件 B 的数量不超过 3 个
Semaphore full_a = 0;            //有零件 a 才能开始装配
Semaphore full_b = 0;            //有零件 b 才能开始装配
```

生产零件 A:

```
while(true){
    P(empty_a);
    生产 A;
    V(full_a);
}
```

生产零件 B:

```
while(true){
    P(empty_b);
    生产 B;
    V(full_b);
}
```

装配生产:

```
while(true){
    P(full_a);
    P(full_b);
    P(full_b);
    P(full_b);
}
```



```

装配;
V(empty_a);
V(empty_b);
V(empty_b);
    V(empty_b);
}

```

46. 【解析】

- 1) $4096\text{B} = 4\text{KB}$ ，故语句①中每次访问的都是不同页面的第一个字节，因此全部发生缺页中断，执行语句①需要的总时间 $t = 1048 \times (100\text{ns} + 10\text{ms} + 100\text{ns} + 100\text{ns}) \approx 10.48\text{s}$ 。
- 2) 分配给该进程的页面数量为 $2\text{MB}/4\text{KB} = 512$ ，根据 LRU 算法，执行完语句①后，内存中的页面序号是后 512 个（数组 data 占 1048 个页面），即第 536~1047 号。语句②：访问字节属于第 1 页，未命中，发生缺页中断，执行时间 $t = 100\text{ns} + 10\text{ms} + 100\text{ns} + 100\text{ns} \approx 10\text{ms}$ ；语句③：访问字节属于第 513 页，未命中，发生缺页中断，执行时间 $t = 100\text{ns} + 10\text{ms} + 100\text{ns} + 100\text{ns} \approx 10\text{ms}$ ；语句④：访问字节属于第 1 页，命中，执行时间 $t = 100\text{ns} + 100\text{ns} = 200\text{ns}$ ；语句⑤：访问字节数据属于第 769 页，命中，执行时间 $t = 100\text{ns} + 100\text{ns}$ 。
- 3) 语句②执行后会将第 1 页调入内存，语句③执行时将页面 1 换出的概率是 $1/512$ ，语句④再访问第 1 页会发生缺页中断，因此语句④执行时间大于 1ms 的概率是 $1/512$ 。

47. 【解析】

- 1) 部门 1 的 IP 地址：211.72.32.0/27。部门 2 的 IP 地址：211.72.32.32/27。
接口 D1 的 IP 地址：211.72.32.1/27。接口 D2 的 IP 地址：211.72.32.33/27。
- 2) 部门 1 的网络配置的子网掩码为 255.255.255.224，默认网关为 211.72.32.1。
- 3) 路由器 R1 中目的网络为部门 1、部门 2 和 Internet 的路由表项如下：

目的网络	子网掩码	下一条 IP 地址	接口
211.72.32.0	255.255.255.224	直连	D1
211.72.32.32	255.255.255.224	直连	D2
0.0.0.0	0.0.0.0	211.72.33.6	L2

路由器 R4 中目的网络为部门 1、部门 2 和 Internet 的路由表项如下：

目的网络	子网掩码	下一条 IP 地址	接口
211.72.32.0	255.255.255.192	211.72.33.22	L10
0.0.0.0	0.0.0.0	211.72.33.17	L9

- 4) R1、R3、R5 一段时间没有收到 R2 的信息，认为 R2 不可达，将到达 R2 的跳数置为 16（表示不可达），R4 也从 R5 或 R3 处获得此消息，收敛后，经过 R2 的路由被其他路由替代。
路由器 R1 的路由表收敛后，目的网络为 Internet 的路由表项如下：

目的网络	子网掩码	下一跳 IP	接口
0.0.0.0	0.0.0.0	211.72.33.2	L1



全国硕士研究生入学统一考试

计算机科学与技术学科联考

计算机专业基础综合考试模拟试卷（三）参考答案

一、单项选择题（第 01~40 题）

01.	A	02.	A	03.	C	04.	A	05.	D	06.	B	07.	C	08.	C
09.	D	10.	D	11.	D	12.	C	13.	C	14.	B	15.	C	16.	C
17.	D	18.	D	19.	D	20.	A	21.	D	22.	D	23.	C	24.	A
25.	B	26.	D	27.	C	28.	B	29.	C	30.	C	31.	A	32.	C
33.	D	34.	B	35.	B	36.	D	37.	D	38.	A	39.	C	40.	B

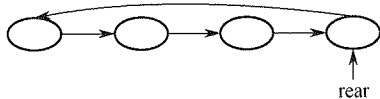
01. A. 【解析】本题考查时间复杂度。

在程序中，执行频率最高的语句为 $i=i*3$ 。设该基本语句共执行了 k 次，根据循环结束条件，有 $n > 2 \times 3^k \geq n/3$ ，由此可得算法的时间复杂度为 $O(\log_3 n) = O(\log_{10} n) = O(\log_2 n)$ 。

▲注意：题中 $k = \log_3 n$ ，又因 $\log_3 n = \log_{10} n / \log_{10} 3$ ，即 k 的数量级为 $\log_{10} n$ ，由此可知，当时时间复杂度为对数级别时，底数的改变对整个时间复杂度没有影响，可一律忽略底数写为 $O(\log_2 n)$ 。

02. A. 【解析】本题考查链式队列的操作。

题中所述的链式队列 Q 如下图所示。出队时，要先找到队头结点指向的结点，再令队尾结点指向这个结点，即 $x=Q.rear \rightarrow next; Q.rear \rightarrow next=Q.rear \rightarrow next \rightarrow next$ ，时间复杂度为 $O(1)$ 。入队时，需要在 $rear$ 和 $rear \rightarrow next$ 之间插入新结点，再令 $rear$ 指向新尾结点(s)，即 $s \rightarrow next=Q.rear \rightarrow next; Q.rear \rightarrow next=s; Q.rear=s$ ，时间复杂度为 $O(1)$ 。



03. C. 【解析】本题考查完全二叉树顺序存储的性质。

根据顺序存储的完全二叉树子结点与父结点之间的倍数关系推导。 k 号结点的祖先为 $[k/2]$ ，计算两个结点 i 和 j 共同的祖先算法可归结如下：

- 1) 若 $i=j$ ，则执行步骤 2)，否则共同父结点为 i （或 j ）。
- 2) 取 $\max\{i, j\}$ 执行操作（以 i 为例）， $i = [i/2]$ ，然后跳回步骤 1)。

根据算法即可算出答案为 2。

04. A. 【解析】本题考查二叉树的存储特点。

当采用三叉链表存储时，每个结点占 $d+12$ 字节，共占用 $n(d+12)$ 字节。当采用顺序存储时，



每个结点占 d 字节, 需存储 k 个结点, 共占用 kd 字节。若要求顺序存储更节省空间, 则要求 $kd < n(d+12)$, 即 $d < 12n/(k-n)$ 。

05. D. 【解析】本题考查树和二叉树的转换。

根据树转化为二叉树的“左孩子右兄弟”规则可知, 在树 T 中 x 是其双亲的右孩子, 也就是说, 在树 T 中 x 一定有左兄弟, 选项 D 正确。

06. B. 【解析】本题考查无向完全图的性质。

n 个顶点的无向完全图共有 $n(n-1)/2$ 条边。若 $n+1$ 个顶点和 $n(n-1)/2$ 条边构成非连通图, 则只有可能是 n 个顶点构成完全图且第 $n+1$ 个顶点构成一个孤立顶点; 若增加一条边, 则在任何情况下都是连通的。因此, 若 $n+1$ 个顶点构成无向非连通图, 边数 e 必满足 $e \leq n(n-1)/2$, 将 $e=36$ 代入, 有 $n \geq 9$, 则 $n+1 \geq 10$, 所以顶点数至少为 10。

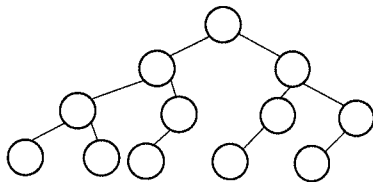
07. C. 【解析】本题考查深度优先遍历。

深度优先遍历是指找到新访问结点后, 就从新结点开始找新访问结点, 若未找到, 则回溯到上一个找到的新访问结点继续查找。从顶点 1 出发, 下一个新访问结点是 3, 从 3 开始, 找到 4, 从 4 开始, 没有新结点, 回溯到 3, 找到新访问结点 5, 从 5 开始, 找到 2, 从 2 开始没有新结点, 回溯到 5, 没有新结点, 回溯到 3, 没有新结点, 回溯到 1, 没有新结点, 访问结束。因此, 得到的顶点序列为 1, 3, 4, 5, 2。

▲注意: 当一个图只给出了相应的图形时, 不管它采用哪种遍历方式, 遍历序列一般都不是唯一的, 但在给定存储结构(邻接矩阵或邻接表等)后, 相应的遍历序列一般都是唯一的。

08. C. 【解析】本题考查折半查找。

对应的折半查找判定树如下图所示: 第一层有 1 个结点, 第二层有 2 个结点, 第三层有 4 个结点, 第四层有 5 个结点, 因此 $ASL_{\text{查找成功}} = (1 \times 1 + 2 \times 2 + 4 \times 3 + 5 \times 4) / 12 = 37/12$ 。



09. D. 【解析】本题考查散列表的应用。

$49 \% 11 = 5$, 与元素 38 冲突; 利用二次探测法计算下一个散列地址 $(49 + 1^2) \% 11 = 6$, 与元素 61 冲突; 继续计算下一个散列地址 $(49 - 1^2) \% 11 = 4$, 与元素 15 冲突; $(49 + 2^2) \% 11 = 9$, 无冲突; 因此, 关键码值为 49 的散列地址为 9。

10. D. 【解析】本题考查堆排序的执行过程。

筛选法最初建的堆为 {8, 17, 23, 52, 25, 72, 68, 71, 60}, 输出 8 后重建的堆为 {17, 25, 23, 52, 60, 72, 68, 71}, 输出 17 后重建的堆为 {23, 25, 68, 52, 60, 72, 71}。建议读者在解题时画草图。

11. D. 【解析】本题考查败者树的特点。

败者树是一棵完全二叉树，归并段的数量（路数）就是叶结点的数量，对有 6 个叶结点的完全二叉树来说，树的深度为 4。

12. C. 【解析】本题考查根据时钟周期 CPI 来计算程序的执行速度。

M1 的时钟频率为 800MHz，M2 的时钟频率为 400MHz。若在 M1 和 M2 上都用 C1 编译器，则 M1 上的 CPI 为 $2 \times 30\% + 3 \times 50\% + 4 \times 20\% = 2.9$ ，M2 上的 CPI 为 $1 \times 30\% + 2 \times 50\% + 1.5 \times 20\% = 1.6$ ，因此在 M1 上一条指令的平均执行时间为 $2.9\text{ns} \times 1000 / 800 = 2.9\text{ns} \times 1.25 = 3.625\text{ns}$ ，在 M2 上一条指令的平均执行时间为 $1.6\text{ns} \times 1000 / 400 = 1.6\text{ns} \times 2.5 = 4\text{ns}$ ，M1 和 M2 的性能之比为 $4 / 3.625 \approx 1.1$ 。若在 M1 和 M2 上都用 C2 编译器，则 M1 上的 CPI 为 $2 \times 30\% + 3 \times 20\% + 4 \times 50\% = 3.2$ ，M2 上的 CPI 为 $1 \times 30\% + 2 \times 20\% + 1.5 \times 50\% = 1.45$ ，因此在 M1 上一条指令的平均执行时间为 $3.2\text{ns} \times 1000 / 800 = 3.2\text{ns} \times 1.25 = 4\text{ns}$ ，在 M2 上一条指令的平均执行时间为 $1.45\text{ns} \times 1000 / 400 = 1.45\text{ns} \times 2.5 = 3.625\text{ns}$ ，M2 和 M1 的性能之比为 $4 / 3.625 \approx 1.1$ 。

13. C. 【解析】本题考查补码减法与标志位。

-3 的机器码为 FFFD，以小端方式存放，所以地址码部分应该为 FDFF，机器码为 2DFDFF，计算 $\text{ax-imm} = 7 - (-3) = 10$ 为正数，SF 标志位应该为 0。

14. B. 【解析】本题考查 IEEE 754 浮点数的表示和舍入方法。

十进制数 0.1 转换成二进制数为 $0.00011[0011]... = +1.1001100110011... \times 2^{-4}$ ，float 型采用 IEEE 754 单精度浮点数表示。符号位为 0；阶码 $= 127 - 4 = 123 = 01111011\text{B}$ ；尾数采用就近舍入，尾数的小数部分为 0.1001100110011001101，因此 float 型变量 x 的机器表示为 0 0111011001100110011001101，转换成十六进制数为 3D3C CCCC_H。

15. C. 【解析】本题考查 I/O 控制方式。

在程序查询方式下，CPU 不断轮询 I/O 设备的状态，完全依赖 CPU，效率最低。中断方式虽然比程序查询方式更高效，但仍需 CPU 介入处理中断，仍以 CPU 为中心。在 DMA 方式下，I/O 设备直接与内存交换数据，仅在传输开始和结束时通知 CPU，是真正以存储器为中心的工作方式。无条件查询方式类似于程序查询方式，CPU 直接读写 I/O 设备，不涉及存储器中心化。

16. C. 【解析】本题考查磁盘的相关性质。

传输时间是扇区大小、旋转速度、磁道上位密度的函数，选项 C 错误。早期磁盘的每个磁道所存储的信息量是一样的，但现代磁盘允许采用固定位密度的方式，这样外圈的磁道所含的信息量会大于内圈的磁道，选项 A 正确。选项 B 和 D 显然正确。

17. D. 【解析】本题考查 Cache 容量的计算。

主存块大小为 64B，因此块内偏移占 6 位；采用 8 路组相联，每组共有 8 块，共 $(32\text{KB}/64\text{B})/8 = 64 = 2^6$ 组，因此组号占 6 位，标记占 $32 - 6 - 6 = 20$ 位；由于采用回写方式，因此需要 1 位脏位（随机替换策略不需要额外标记位）；最后加上 1 位有效位，共 22 位。因此实际每个 Cache 行的大小为 $64 \times 8 + 22 = 534$ 位；L1 data Cache 和 L1 code Cache 均有 $32\text{KB}/64\text{B} = 512$ 行，因此 L1 Cache 共需 1024 行。综上所述，L1 Cache 总容量至少有 $534 \text{ 位} \times 1024 = 534\text{K 位}$ 。



18. D. 【解析】本题考查数据传输指令。

传输指令实现部件之间的数据传输，这里的部件一定是能够存储信息的部件，选项中单独出现的 CPU 实际是指 CPU 中的寄存器。对于选项 A，出/入栈指令 (push/pop) 实现的是 CPU 中的通用寄存器和栈顶之间的数据传输。对于选项 B，访存指令 (load/store) 实现的是 CPU 中的通用寄存器和存储单元之间的数据传输。对于选项 C，I/O 指令 (in/out) 实现的是 CPU 中的通用寄存器和 I/O 端口之间的数据传输。对于选项 D，寄存器传输指令 (move) 实现的是通用寄存器和通用寄存器之间的数据传输，若说成是 CPU 和寄存器之间的数据传输，则意味着 CPU 中还有其他非寄存器部件能存储数据，这种描述显然是错误的。

19. D. 【解析】本题考查运算器的组成。

数据高速缓存是专门存放数据的 Cache，不属于运算器。

▲注意：运算器应包括算术逻辑单元、暂存寄存器、累加器、通用寄存器组、程序状态字寄存器、移位器等，控制器应包括指令部件、时序部件、微操作信号发生器（控制单元）、中断控制逻辑等，指令部件包括程序计数器（PC）、指令寄存器（IR）和指令译码器（ID）。

20. A. 【解析】微指令字长为 24 位，其具体格式如下表所示。

3 位	4 位	4 位	2 位	3 位	8 位
				判断测试字段	下地址字段
操作控制字段				顺序控制字段	

因为下地址字段有 8 位，所以控制存储器的容量为 $256 \times 24\text{bit}$ 。

▲注意：这里说到外部条件有 3 个，有的同学可能觉得可用 2 位字段来表示，然后地址位就是 9 位，因此答案应该是 $512 \times 24\text{bit}$ 。然而，这是不对的，因为题目并未说这三个外部条件是互斥的，也就是说，这三个外部条件组合起来共有 $2^3 = 8$ 种可能，所以不可能用 2 位字段来表示。

▲注意：控制存储器中存放的是微程序，微程序的数量取决于机器指令的条数，与微指令的数量无关。

21. D. 【解析】本题考查总线上数据的传输方式。

目前常见的内存条大多采用 DDR、DDR2、DDR3 等 SDRAM 芯片技术。与这种内存条相连的存储器总线，每个时钟周期总是在上升沿和下降沿各传输一次数据，因此一个时钟周期内可以并行传输两次数据，选项 D 错误。其他选项的描述均正确。

22. D. 【解析】本题考查 DMA 方式。

每传输一个数据块就要向 CPU 发出中断请求， $4\text{KB} \div (8\text{Mb/s}) = 4 \times 1024\text{B} / (8 \times 10^6\text{b/s}) = 4096\mu\text{s}$ 。

23. C. 【解析】本题考查操作系统的运行模式。

外部设备发出中断信号后，CPU 不一定能马上响应中断，因此 CPU 不一定能立即切换到内核态。在虚拟内存管理中进行地址转换时，若页号大于页表长度，则表示发生越界异常，需要马上执行相应的异常处理程序，CPU 需要切换到内核态；若要访问的页面不在内存，则表示发生缺页异常，CPU 需要切换到内核态。当 CPU 响应并处理中断时，并不一定发生了进程切换，进程切换需要 CPU 执行相应的进程调度程序。综上所述，选项 II 和 III 正确。



24. A. 【解析】本题考查进程的状态。

等待态也就是阻塞态，当正在运行的进程需要等待某个事件时，会由运行态变为阻塞态。P操作的作用相当于申请资源，当P操作未得到相应的资源时，进程就进入阻塞态。选项B、C都是从运行态变为就绪态。选项D执行V操作可能改变其他进程的状态，但与本进程状态的转变没有直接关系。

25. B. 【解析】本题考查虚拟文件系统的概念。

VFS是操作系统内核中的一个抽象层，其核心目的是屏蔽不同文件系统（如EXT4、NTFS、FAT32等）的实现细节，向上层应用和系统调用提供统一的访问接口（如open()、read()、write()）。

26. D. 【解析】本题考查进程调度的时间计算。

在0~10s的时段内，只有3~3.5s间CPU未运行程序，因此CPU的利用率为 $9.5/10 = 95\%$ 。进程A的周转时间为 $3s - 0s = 3s$ ，进程D的周转时间为 $3.5s - 0s = 3.5s$ ，进程B的周转时间为 $10s - 4s = 6s$ ，进程C的周转时间为 $8s - 6s = 2s$ ，因此平均周转时间为 $(3s + 3.5s + 6s + 2s)/4 = 3.625s$ 。进程D的等待时间为3s，进程B的等待时间为2s，进程A、C的等待时间都是0s，因此进程D的等待时间最长。进程A的带权周转时间为 $3s/3 = 1s$ ，进程B的带权周转时间为 $6/4 = 1.5s$ ，进程C的带权周转时间为 $2/2 = 1s$ ，进程D的带权周转时间为 $3.5s/0.5 = 7s$ ，因此，进程A、B、C和D的平均带权周转时间为 $(1s + 1.5s + 1s + 7s)/4 = 2.625s$ 。于是，选项I~IV都正确。

27. C. 【解析】本题考查请求和保持条件。

第一种方法的优点是简单、易行且安全，但缺点也很明显：① 资源被严重浪费，严重地降低了资源的利用率；② 会使进程经常发生饥饿现象。因为仅当进程获得其所需的全部资源后才能开始运行，所以可能由于个别资源长期被其他进程占用而致使等待该资源的进程迟迟不能开始运行。第二种方法能使进程更快地完成任务，提高设备的利用率，减小进程发生饥饿的概率，但并未杜绝进程发生饥饿的概率，选项C错误。

28. B. 【解析】本题考查各存储分配方法的特点。

固定式分区存在内部碎片，当程序小于固定式分区大小时，也占用一个完整的内存分区空间，导致分区内部有空间浪费，这种现象称内部碎片。凡涉及页的存储分配管理，每页的长度都一样（对应固定），所以会产生内部碎片，虽然页的碎片较小，但每个进程平均产生半块大小的内部碎片。段式管理中每段的长度都不一样（对应不固定），所以只会产生外部碎片。段页式管理首先被分为若干逻辑段，然后将每段分为若干固定的页，所以其仍然是固定分配的，会产生内部碎片。

29. C. 【解析】本题考查页面缓冲算法。

系统可以在内存中设置空闲页面链表和修改页面链表来降低页面换入换出的频率，使磁盘I/O次数大为减少。① 空闲页面链表实际上是一个空闲物理块链表，是系统掌握的空闲物理块，用于分配给频繁发生缺页的进程，以降低进程的缺页率，当有一个未被修改的页面要换出时，实际上并不将它换出到外存中，而将它们所在的物理块挂在空闲链表的末尾。② 修改页面链表，它是由已修改的页面形成的链表，设置该链表的目的是减少已修改页面换出的次数。选项C的说法显然错误。



30. C. 【解析】本题考查文件共享。

无论是采用符号链接还是采用硬链接,都存在一个共有的问题,即每个共享文件都有几个文件名,换言之,每增加一条链接就增加一个文件名,实质上是每个用户都使用自己的路径名去访问共享文件。当试图遍历整个文件系统时,将多次遍历到该共享文件,因此当要将一个目录中的所有文件都转储到磁带上时,可能对一个共享文件产生多份副本。

31. A. 【解析】本题考查 SPOOLing 技术。

在打印机和磁盘上的输出井之间,内存中的输出缓冲区作为中介,打印机和输出井之间并不存在直连通道,选项 A 错误。输入进程和输出进程都是运行在内核态下的程序,选项 B 正确。输出进程同样是作为进程参与调度的,因此与用户进程是并发执行的,选项 C 正确。输出进程需要沟通磁盘和打印机这两种外设,因此需要至少两种设备驱动程序的支持,选项 D 正确。

32. C. 【解析】本题考查提前读和延迟写的概念。

提前读是指用户对文件进行访问时,经常采用顺序访问方式,在读当前块的同时,可将下一个盘块也提前读入缓冲区,这样,当下次要读该盘块时,便可直接从缓冲区中取得下一盘块的数据,而无需再启动磁盘 I/O,因此选项 A、B 正确。延迟写是指在缓冲区中的数据本应立即写回磁盘,但考虑到该缓冲区中的数据在不久后可能会再次被本进程或其他进程访问(共享资源),因此并不立即写回磁盘,而挂在空闲缓冲区队列的末尾,此时,任何访问该数据的进程都可直接读出其中的数据而不必启动磁盘 I/O,这样,又可进一步减小等效的磁盘 I/O 时间。由此可知,延迟写和随机访问方式之间并没有必然的联系,选项 C 错误。当缓冲区中的数据延迟到必须往磁盘上写时(如缓冲区满),才进行写磁盘操作。

33. D. 【解析】本题考查传播时延和 RTT 的概念。

若站点 A 和站点 B 是直接连接的,则该 RTT 的值就是站点 A 和站点 B 之间的传输媒介的往返传播时延。但是,若站点 A 和站点 B 之间还有一个或多个路由器,则该 RTT 的值还应包括分组所经过的路由器的排队时间和处理时间。若遇到网络拥塞,则位于途中的某个路由器甚至会丢弃这个分组,此时测量出的 RTT 值是无穷大(测不出 RTT 的值),表示站点 B 永远收不到应答分组,可能是站点 A 发送的分组在途中被丢弃而未到达站点 B,也可能是站点 B 发送的应答分组在途中被丢弃而未到达站点 A。因此,该 RTT 的值大于或等于站点 A 和站点 B 之间的传输媒介的往返传播时延。

34. B. 【解析】本题考查差分曼彻斯特编码。

差分曼彻斯特编码的值由上一个码元的后半部分电平与当前码元的前半部分电平是否相同决定,若相同,则为 1,否则为 0,因此对应的比特流是 0001 011。

35. B. 【解析】本题考查截断二进制指数退避算法。

截断二进制指数退避算法从离散的整数集合 $\{0, 1, \dots, (2^k - 1)\}$ 中随机取出一个数,记为 r ,重传所需推迟的时间就是 r 倍的争用期, k 为重传次数。两个站点同时发送数据,碰撞以后就同时执行二进制指数退避算法,第一次重传时,两个站点都会从集合 $\{0, 1\}$ 中随机选取一个整数来确定重传时间,若选取的整数相同,则重传推迟的时间就会相等,下一次仍会碰撞,两个站点分别从集合 $\{0, 1\}$ 中选取相同整数的概率显然为 0.5,若第一次重传失败,进行第二次重



传,则两个站点会从集合{0,1,2,3}中再随机选取一个整数,本次选取相同整数的概率为0.25。不难发现,若将第*i*次重传失败的概率记为 p_i ,则显然有 $p_i=(0.5)^k$, $k=\min[i,10]$ 。第二次重传成功即表示第一次重传失败且第二次重传成功,第一次重传失败的概率为0.5,第二次重传成功的概率为 $1-0.25=0.75$,所以重传第二次才成功的概率为 $0.5\times0.75=0.375$ 。

36. D. 【解析】本题考查 PPP。

PPP 没有编号和确认机制,必须靠上层的协议(有编号和确认机制)来保证数据传输正确无误,选项 D 错误。其余说法均正确。

37. D. 【解析】本题考查 RIP 的过程。

对比表 1 和表 2 发现,R1 到达目的网络 20.0.0.0 的距离为 7,而表 2 中 R2 到达目的网络 20.0.0.0 的距离为 4。由于 $7 > 4 + 1$,此时 R1 经过 R2 到达目的网络 20.0.0.0 的路由距离变短,因此 R1 要根据 R2 提供的数据修改相应路由项的距离值为 5。

R1 到达目的网络 30.0.0.0 的距离为 4,而表 2 中 R2 到达目的网络 30.0.0.0 的距离为 3。由于 $4 = 3 + 1$,显然 R1 经过 R2 到达目的网络 30.0.0.0 并不能得到更短的路由距离,因此 R1 无须进行更新操作,将保持该路由表项原来的参数。

当 R1 收到 R2 发送的报文后,按照如下规律更新路由表信息:

- ① 若 R1 的路由表没有某项路由记录,则 R1 在路由表中增加该项,由于要经过 R2 转发,因此距离值要在 R2 提供的距离值的基础上加 1。
- ② 若 R1 的路由表中的某表项路由记录比 R2 发送的对应项的距离值加 1 还大,则 R1 在路由表中修改该项,距离值根据 R2 提供的距离值加 1。可见,对于路由表距离值为 0 的直连网络,无须进行更新操作,其路由距离保持为 0。

38. A. 【解析】本题考查默认路由的配置。

所有网络都必须使用子网掩码,在路由器的路由表中也必须要有子网掩码一栏。一个网络若不划分子网,则使用默认子网掩码。默认子网掩码中 1 的位置和 IP 地址中的网络号字段 net-id 正好对应。主机地址是一个标准的 A 类地址,其网络地址为 11.0.0.0。选项 I 的网络地址为 11.0.0.0,选项 II 的网络地址为 11.0.0.0,选项 III 的网络地址为 12.0.0.0,选项 IV 的网络地址为 13.0.0.0,因此和主机在同一个网络中的是选项 I 和选项 II。

39. C. 【解析】本题考查 TCP 序号绕回问题。

传输率为 2.5Gb/s,51.2s 内可发送 $2.5\times2^{30}\times51.2/8=2^{34}$ B 的数据,PDU 首部中序号字段的值用来指明 PDU 数据载荷的第 1 个字节的序号。假设序号字段的长度为 n 位,则 $2^n\geq2^{34}$,即 $n\geq34$,这样就能确保出现序号绕回时,网络中之前具有相同序号的 PDU 已从网络中消失。

40. B. 【解析】本题考查 DNS。

DNS 是为了获取站点的 IP 地址,若知道对方站点的 IP 地址,则能正常地发送数据,选项 A 错误。DNS 是基于 UDP 进行传输的,目的是减少开销,选项 C 错误。当一个权限域名服务器还不能给出最后的查询回答时,就告诉发出查询请求的 DNS 客户下一步应当找另一个权限域名服务器。例如,查询 www.abc.xyz.com 时,需要依次查询 xyz.com、abc.xyz.com 的权限域名服务器。



二、综合应用题

41. 【解析】

1) 该图对应的邻接矩阵如下:

$$\begin{bmatrix} \infty & 2 & 3 & \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 5 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 3 & 10 & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & 4 & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty & 3 & \infty \\ \infty & \infty & \infty & \infty & 2 & \infty & \infty & 6 \\ \infty & \infty & \infty & \infty & \infty & \infty & \infty & 1 \\ \infty & \infty & \infty & \infty & \infty & \infty & \infty & \infty \end{bmatrix}$$

2) 只有顶点 V_1 的入度为 0, 由此可得两个拓扑序列: $V_1, V_2, V_3, V_4, V_6, V_5, V_7, V_8$ 和 $V_1, V_3, V_2, V_4, V_6, V_5, V_7, V_8$ 。

3) 关键路径共 3 条, 长为 17, 依次为 $V_1 \rightarrow V_2 \rightarrow V_4 \rightarrow V_6 \rightarrow V_8$, $V_1 \rightarrow V_3 \rightarrow V_5 \rightarrow V_7 \rightarrow V_8$, $V_1 \rightarrow V_2 \rightarrow V_4 \rightarrow V_6 \rightarrow V_5 \rightarrow V_7 \rightarrow V_8$ 。

事 件	V_1	V_2	V_3	V_4	V_5	V_6	V_7	V_8
最早发生时间	0	2	3	7	13	11	16	17
最晚发生时间	0	2	3	7	13	11	16	17

活 动	V_1-V_2	V_1-V_3	V_2-V_4	V_3-V_4	V_3-V_5	V_4-V_6	V_6-V_5	V_5-V_7	V_6-V_8	V_7-V_8
最早开始时间	0	0	2	3	3	7	11	13	11	16
最晚开始时间	0	0	2	4	3	7	11	13	11	16
时间余量	0	0	0	1	0	0	0	0	0	0

4) 顶点 V_1 到其他各顶点的最短路径和距离为 2 ($V_1 \rightarrow V_2$), 3 ($V_1 \rightarrow V_3$), 6 ($V_1 \rightarrow V_3 \rightarrow V_4$), 12 ($V_1 \rightarrow V_3 \rightarrow V_4 \rightarrow V_6 \rightarrow V_5$), 10 ($V_1 \rightarrow V_3 \rightarrow V_4 \rightarrow V_6$), 15 ($V_1 \rightarrow V_3 \rightarrow V_4 \rightarrow V_6 \rightarrow V_5 \rightarrow V_7$), 16 ($V_1 \rightarrow V_3 \rightarrow V_4 \rightarrow V_6 \rightarrow V_5 \rightarrow V_7 \rightarrow V_8$ 或 $V_1 \rightarrow V_3 \rightarrow V_4 \rightarrow V_6 \rightarrow V_8$)。

42. 【解析】

解法 1

1) 算法的基本设计思想:

注意到旋转后的数组实际上可划分成两个排序的子数组, 且前面的子数组的元素都大于或等于后面子数组的元素, 而最小的元素刚好是这两个子数组的分界线。

我们试着用二元查找的思路来寻找这个最小的元素。定义两个指针, 分别指向数组的第一个元素和最后一个元素。按照题目旋转的规则, 第一个元素应该是大于或等于最后一个元素的。再定义一个指针指向中间的元素, 若该中间元素位于前面的递增子数组中, 则它应大于或等于第一个指针指向的元素, 此时最小的元素位于右子数组中, 然后将第一个指针指向该中间元素, 以便在缩小的范围内继续寻找。同样, 若该中间元素位于后面的递增子数组中, 则思路 and 上面的类似。

按照上述思路, 第一个指针总是指向前面递增数组的元素, 而第二个指针总是指向后面递增数组的元素。最后, 第一个指针将指向前面子数组的最后一个元素, 而第二个指针指向后面子数

组的第一个元素，此时两个指针相邻，而第二个指针指向的正好是最小元素。这就是循环结束的条件。

2) 算法的实现如下：

```
int Min(int *numbers,int length){
    if(numbers==0||length<=0)
        return 0;
    int index1=0;                //第一个指针
    int index2=length-1;        //第二个指针
    int indexMid=index1;         //中间指针
    while(numbers[index1]>=numbers[index2]){
        if(index2-index1==1){
            indexMid=index2;
            break;
        }
        indexMid=(index1+index2)/2;
        if(numbers[indexMid]>=numbers[index1])    //在右区间
            index1=indexMid;
        else if(numbers[indexMid]<=numbers[index2]) //在左区间
            index2=indexMid;
    }
    return numbers[indexMid];
}
```

3) 每次都寻找的范围缩小一半，时间复杂度为 $O(\log_2 N)$ 、空间复杂度为 $O(1)$ 。

解法 2

本题的直观解法并不难。从头到尾遍历数组一次，就能找出最小元素，时间复杂度显然是 $O(N)$ 。然而，这个思路未利用输入数组的特性。

43. 【解析】

- 1) Cache 行数为 $4\text{KB}/64\text{B} = 64$ ；采用 4 路组相联，每组有 4 行，共 16 组。主存地址空间大小为 64KB，按字节编址，因此主存地址有 16 位，其中低 6 位为块内地址，中间 4 位为组号（组索引），高 6 位为标记。
- 2) 采用写回策略，因此每个 Cache 行中要有 1 位脏位（dirty bit）；每组有 4 行，因此每行要有 2 位 LRU 置换位。此外，每行还有 6 位标记、1 位有效位和 64B 数据，共有 64 行，所以 Cache 的总容量为 $64 \times (6 + 1 + 1 + 2 + 64 \times 8) = 33408$ 位。
- 3) 块大小为 64B，主存的第 0~4344 号单元对应前 $4345/64 \approx 68$ 块（第 0~67 号块）。CPU 对主存的第 0~67 号块连续访问 16 次，下图给出了访问过程中主存块和 Cache 行之间的映射关系，图中列方向是 Cache 的 16 组，行方向是每组的 4 行。

	第 0 行	第 1 行	第 2 行	第 3 行
第 0 组	0/64/48	16/0/64	32/16	48/32
第 1 组	1/65/49	17/1/65	33/17	49/33
第 2 组	2/66/50	18/2/66	34/18	50/34
第 3 组	3/67/51	19/3/67	35/19	51/35
第 4 组	4	20	36	52
⋮	⋮	⋮	⋮	⋮
第 15 组	15	31	47	63



主存的第 0~15 号块分别对应 Cache 的第 0~15 组，可存放在对应组中的任意一行，假定按顺序存放在第 0 行；第 16~31 号块分别按顺序存放在 Cache 的第 0~15 组的第 1 行；第 32~47 号块分别按顺序存放在 Cache 的第 0~15 组的第 2 行；第 48~63 号块分别按顺序存放在 Cache 的第 0~15 组的第 3 行。因此，访问主存的第 0~63 号块都是没有冲突的，每块都是第一个单元未命中，然后将这一块调入 Cache 对应组的某行，这样该块其余 63 个单元都命中。主存的第 64~67 号块分别对应 Cache 的第 0~3 组，此时这四组的 4 行都被占满，每组都要选择淘汰一个主存块。

采用 LRU 算法，因此将最近最少使用的第 0~3 号块分别从第 0~3 组的第 0 行中淘汰，再将第 64~67 号块分别存放到 Cache 的第 0~3 组的第 0 行，每块也都是第一个单元未命中，调入后，其余 63 个单元都能在 Cache 中找到。综上所述，在第一次循环中，每块都只有第一个单元未命中，其余都命中。在后面的 15 次循环中，Cache 的第 4~15 组的主存块一直未被替换，而第 0~3 组每次都需要替换，因此只有 $68 - 48 = 20$ 行中对应主存块的第一个单元未命中，其余都命中。总访问次数为 $4345 \times 16 = 69520$ ，其中未命中次数为 $68 + 15 \times 20 = 368$ 。命中率为 $(69520 - 368) / 69520 \approx 99.47\%$ ，平均访存时间约为 $20\text{ns} + 200\text{ns} \times (1 - 0.9947) = 20\text{ns} + 1.06\text{ns} = 21.06\text{ns}$ 。

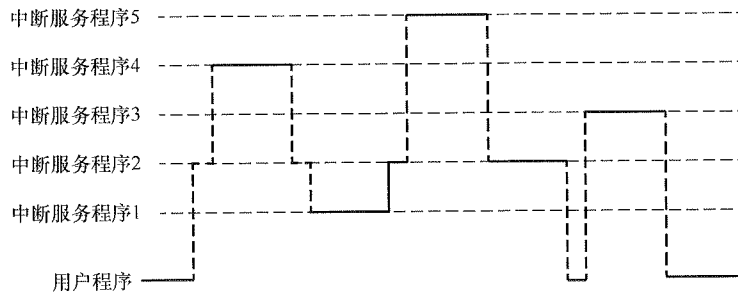
44. 【解析】

- 1) 中断源 1 的处理优先级最高，说明中断源 1 能屏蔽其他所有中断源，屏蔽字为全 1；中断源 3 的处理优先级最低，因此除其自身外，对其他所有中断源都开放，屏蔽字为 00100。以此类推，得到各中断源的中断服务程序中设置的中断屏蔽字如下表所示。

中断处理程序	中断屏蔽字				
	第 1 号	第 2 号	第 3 号	第 4 号	第 5 号
第 1 号	1	1	1	1	1
第 2 号	0	1	1	0	0
第 3 号	0	0	1	0	0
第 4 号	0	1	1	1	1
第 5 号	0	1	1	0	1

- 2) 运行用户程序时，用户程序对所有中断源都开放，因此中断源 2 和 4 同时发出中断请求时，中断判优逻辑（或查询程序）根据响应优先级（ $2 > 4$ ）决定先响应中断源 2。中断源 2 的响应周期结束后，首先保护现场、保护旧屏蔽字、设置新的屏蔽字 01100，然后在执行中断源 2 的处理程序前先开中断，一旦开中断，则马上响应中断源 4，因为中断源 2 的中断屏蔽字中对应中断源 4 的屏蔽位是 0，即对中断源 4 是开放的。中断源 4 的处理结束后，返回中断源 2 的服务程序执行；在处理中断源 2 的过程中，同时发生中断源 1、3、5 的请求，因为中断源 2 对中断源 1 和 5 开放，对中断源 3 屏蔽，所以中断判优逻辑（或查询程序）根据响应优先级（ $1 > 5$ ）决定先响应中断源 1。因为中断源 1 的处理优先级最高，所以在其处理过程中不会响应任何新的中断请求，直到中断源 1 处理结束，然后返回中断源 2；因为中断源 2 对中断源 5 开放，所以返回中断源 2 后又立即转去执行中断源 5 的服务程序，执行完后又回到中断源 2，在中断源 2 的服务程序的执行过程中，虽然有中断源 3 的请求，但由于中断源 2 对中断源 3 不开放，因此中断源 3 得不到响应。直到中断源 2 处理结束并返回到用户程序，才能响应并处理中断源 3。CPU 的运行过程如下图所示。





45. 【解析】

由于不允许两个方向的猴子同时跨越绳索，因此对绳索应该互斥使用。但同一个方向允许多只猴子通过，所以临界区允许多个实例访问。本题的难点是位于南北方向的猴子具有相同的行为，当一方有猴子在绳索上时，同方向的猴子可继续通过，但此时要防止另一方的猴子跨越绳索。类比经典的读者/写者问题。

信号量设置：对绳索应互斥使用，设置互斥信号量 `mutex`，初始值为 1。但同一个方向允许多只猴子通过，所以定义变量 `NmonkeyCount` 和 `SmonkeyCount` 分别表示从北向南和从南向北的猴子数量。因为涉及更新 `NmonkeyCount` 和 `SmonkeyCount`，所以需要对其进行保护。更新 `NmonkeyCount` 和 `SmonkeyCount` 时需要用信号量来保护，所以设置信号量 `Nmutex` 和 `Smutex` 来保护 `NmonkeyCount` 和 `SmonkeyCount`，初始值都为 1。

```
int SmonkeyCount=0;           //从南向北攀越绳索的猴子数量
int NmonkeyCount=0;          //从北向南攀越绳索的猴子数量
semaphore mutex=1;           //绳索互斥信号量
semaphore Smutex=1;          //南方向猴子间的互斥信号量
semaphore Nmutex=1;          //北方向猴子间的互斥信号量
cobegin{
process South_i(i=1,2,3,...){
    while(TRUE){
        p(Smutex);           //互斥访问 SmonkeyCount
        if(SmonkeyCount==0)   //本方第一只猴子需发出绳索使用请求
            p(mutex);
        SmonkeyCount=SmonkeyCount+1; //后续猴子可以进来
        v(Smutex);
        Pass the cordage;
        p(Smutex);           //猴子爬过去后需要更新 SmonkeyCount，互斥
        SmonkeyCount=SmonkeyCount-1; //更新 SmonkeyCount
        if(SmonkeyCount==0)     //若此时后方已无要通过的猴子，则最后一只猴子通过后放开绳索
            v(mutex);
        v(Smutex);
    }
}
process North_j(j=1,2,3,...)
    while(TRUE){
        p(Nmutex);           //互斥访问 NmonkeyCount
        if(NmonkeyCount==0)   //本方第一只猴子需发出绳索使用请求
            p(mutex);
        NmonkeyCount=NmonkeyCount+1; //后续猴子可以进来
        v(Nmutex);
        Pass the cordage;
```



```

p(Nmutex); //猴子爬过去后需要更新 NmonkeyCount, 互斥
NmonkeyCount=NmonkeyCount-1; //更新 NmonkeyCount
if (NmonkeyCount==0) //若此时后方已无要通过的猴子, 则最后一只猴子通过后放开绳索
    v(mutex);
v(Nmutex);
}
} coend

```

▲注意: 有的同学注意到了这种算法会导致饥饿, 但是题目中只要求实现互斥, 并未对饥饿控制提出要求, 而且若还要考虑饥饿, 则必然导致复杂性大大增加, 而一般考试的难度不会这么大。若实在要考虑, 则可以设一个固定的数值代表一次单向的最大通过量, 当一个方向通过那么多猴子后, 看看对方是否要通过, 若有, 则让出绳锁, 若没有, 则继续让这个方向的猴子通过。

46. 【解析】

在混合索引分配方式中, 文件 FCB 的直接地址中保存了分配给文件的前 n 块 (第 0 到 $n-1$ 块) 的物理块号 (n 的值由直接地址项数决定, 本题中为 10); 一次间址中记录了一个一次间址块的块号, 而一次间址块中保存了分配给文件的第 n 块到第 $n+k-1$ 块的块号 (k 的值由盘块大小和盘块号长度决定, 本题中为 170); 二次间址中记录了一个二次间址块的块号, 二次间址块可记录 k 个一次间址块的块号, 这些一次间址块被用来保存分配给文件的第 $n+k$ 块到第 $n+k+k^2-1$ 块的块号; 三次间址中则记录了一个三次间址块的块号, 三次间址块可记录 k 个二次间址块的块号, 这些二次间址块又可记录 k^2 个一次间址块的块号, 而这些一次间址块则被用来保存分配给文件的第 $n+k+k^2$ 块到第 $n+k+k^2+k^3-1$ 块的物理块号。

- 1) 文件系统支持的最大文件长度 = $10 + 170 + 170 \times 170 + 170 \times 170 \times 170 = 4942080$ 块 = $4942080 \times 512\text{B} = 2471040\text{KB}$ 。
- 2) $5000/512$, 商为 9, 余数为 392, 对应的逻辑块号为 9, 块内偏移量为 392。由于 $9 < 10$, 因此可直接从文件 FCB 的第 9 个地址项处得到物理块号, 块内偏移量为 392。
 $15000/512$, 商为 29, 余数为 152, 对应的逻辑块号为 29, 块内偏移量为 152。由于 $10 \leq 29 < 10 + 170$, 而 $29 - 10 = 19$, 因此可从文件 FCB 的第 10 个地址项中得到一次间址块的地址, 读入该一次间址块并从它的第 19 项中获得相应的物理块号, 块内偏移量为 152。
 $150000/512$, 商为 292, 余数为 496, 对应的逻辑块号为 292, 块内偏移量为 496。由于 $10 + 170 \leq 292 < 10 + 170 + 170 \times 170$, 而 $292 - (10 + 170) = 112$, $112/170$ 的商为 0, 余数为 112, 因此可从文件 FCB 的第 11 个地址项中得到二次间址块的地址, 读入该二次间址块并从它的第 0 项中得到一个一次间址块的地址, 再读入该一次间址块并从它的第 112 项中获得相应的物理块号, 块内偏移量为 496。
- 3) 由于文件的 FCB 已在内存中, 为访问文件中某个位置的内容, 最少需要访问 1 次磁盘 (即可通过直接地址项读文件盘块), 最多需要访问 4 次磁盘 (第 1 次是读三次间址块, 第 2 次是读二次间址块, 第 3 次是读一次间址块, 第 4 次是读文件盘块)。

47. 【解析】

- 1) PC2 所在的网络是 211.73.1.144/29, M1 所在的网络是 211.73.1.152/29, 两者处在不同的网络中, 而中间需要用路由器隔离, 因此设备 X1 是路由器。
- 2) PC2 收到的数据帧由 X1 直接转发而来, 因此源 MAC 地址是接口 D5 的 MAC 地址。在 M1



收到的数据帧中，地址 1 是接收地址（接口 D1 的 MAC 地址），地址 2 是发送地址（接口 D2 的 MAC 地址），地址 3 是源地址（注意不是发送地址），即接口 D4 的 MAC 地址。

3) PC1 所在的网络是 211.73.1.128/28，可使用的第一个 IP 地址是 211.73.1.129/28，因此分配给默认网关的 IP 地址是 211.73.1.129/28。主机号占 4 位，除去全 0、全 1、默认网关和 PC1 的 IP 地址，还剩 12 个空闲的 IP 地址可供分配。

4) R1 的路由表如下所示。

目的网络	子网掩码（点分十进制表示）	接口
211.73.1.144	255.255.255.240	D8
211.73.1.128	255.255.255.240	D10
0.0.0.0	0.0.0.0	D9



关注公众号【考研小舟】
免费考研资料&无水印PDF



黑白&彩印都是5分/页，满4.9全国包邮
下单备注：考研小舟，送草稿纸



全国硕士研究生入学统一考试

计算机科学与技术学科联考

计算机专业基础综合考试模拟试卷（四）参考答案

一、单项选择题（第 01~40 题）

01.	A	02.	C	03.	A	04.	C	05.	C	06.	B	07.	B	08.	D
09.	C	10.	A	11.	C	12.	B	13.	A	14.	A	15.	D	16.	B
17.	B	18.	C	19.	C	20.	D	21.	C	22.	A	23.	B	24.	A
25.	B	26.	A	27.	B	28.	B	29.	D	30.	C	31.	D	32.	C
33.	D	34.	B	35.	A	36.	A	37.	B	38.	C	39.	A	40.	C

01. A. 【解析】本题考查时间复杂度。

将算法中基本运算的执行次数的数量级作为时间复杂度。基本运算是 $i=i/2$ ；设其执行次数为 k ，则 $(n \times n)/(2^k) = 1$ ，得 $k = \log_2 n^2$ ，因此 $k = \log_2 n^2 = 2\log_2 n$ ，即 k 的数量级为 $\log_2 n$ ，因此时间复杂度为 $O(\log_2 n)$ 。

02. C. 【解析】本题考查稀疏矩阵的压缩存储。

稀疏矩阵有 1000 个非零元素，即存储该矩阵的三元组表有 1000 个表项，每个表项存储一个非零元素的行、列和值，每个表项占用的字节数为 $2 + 2 + 4 = 8$ ，因此用三元组表存储该矩阵时所需的字节数是 8000。

▲注意：稀疏矩阵压缩存储时除了要存储三元组表非零元素的值，还要存储稀疏矩阵的行数和列数。

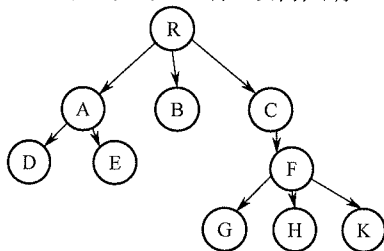
03. A. 【解析】本题考查特殊二叉树的性质。

对于选项 I，最后一层的叶结点数可能为奇数，即倒数第二层上有非叶结点的度为 1。对于选项 II，显然满足。对于选项 III，可能存在非叶结点只有一个孩子结点。对于选项 IV，根据哈夫曼树的构造过程可知所有非叶结点的度均为 2。对于选项 V，可能存在非叶结点只有一个孩子结点。

▲注意：在哈夫曼树中没有度为 1 的结点。

04. C. 【解析】本题考查树的存储。

根据树的双亲表示法，画出树的形态如下，可知该树共有 6 个叶结点，选项 C 错误。



05. C. 【解析】本题考查哈夫曼树的特点。

在构造度为 m 的哈夫曼树的过程中，每次把 m 个子结点合并为一个父结点，每次合并减少 $m-1$ 个结点，从 n 个叶结点减少到最后一个父结点共需 $(n-1)/(m-1)$ 次合并，每次合并增加 1 个非叶结点。

06. B. 【解析】本题考查图的存储的特点。

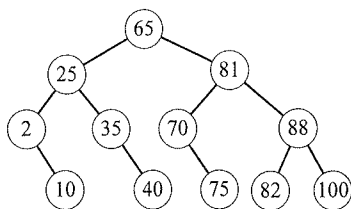
对于有向图，求一个顶点的度时，若采用邻接矩阵存储，则只需遍历该顶点对应的行和列；若采用邻接表存储，则求出度很方便，但求入度时需要遍历整个邻接表。

07. B. 【解析】本题考查 Dijkstra 算法的思想。

执行 Dijkstra 算法时，当前某个轮次选出的顶点都是待求顶点中到源点距离最近的顶点，因此它是按长度递增的顺序求出图的某个顶点到其余顶点的最短路径，选项 B 正确。

08. D. 【解析】本题考查折半查找的查找过程。

有序表长 12，依据折半查找的思想，第一次查找第 $\lfloor (1+12)/2 \rfloor = 6$ 个元素，即 65；第二次查找第 $\lfloor [(6+1)+12]/2 \rfloor = 9$ 个元素，即 81；第三次查找第 $\lfloor [7+(9-1)]/2 \rfloor = 7$ 个元素，即 70；第四次查找第 $\lfloor [(7+1)+8]/2 \rfloor = 8$ 个元素，即 75。比较的元素依次为 65, 81, 70, 75。对应的折半查找判定树如下图所示。



09. C. 【解析】本题考查散列表的性质。

不同的冲突处理方法对应的平均查找长度是不同的，选项 I 错误。散列查找的思想是通过散列函数计算地址，然后比较关键字，以确定是否查找成功，选项 II 正确。平均查找长度与填充因子（表中记录数与表长之比）有关，选项 III 错误。在开放定址的情况下，不能随便删除表中的某个元素（只能标记为删除状态），否则可能导致搜索路径中断，选项 IV 错误。

10. A. 【解析】本题考查堆排序的过程。

堆排序的过程首先是构造初始堆，然后将堆顶元素（最大值或最小值）与最后一个元素交换，此时堆的性质会被破坏，需要从根结点开始进行向下调整操作。如此反复，直到堆中只有一个元素为止。观察发现，每趟排序都从未排序序列中选择一个最大元素放到其最终位置，符合大顶堆的性质，初始序列本身就是一个大顶堆。冒泡排序虽然也可形成全局有序序列，但是题中的排序过程显然不满足冒泡排序的过程。若是快速排序，则第二趟以 25 为基，排序的结果应该是 21, 15, 25, 47, 84，所以并不是快速排序。

11. C. 【解析】本题考查外部排序如何判断添加虚段的数目。

虚段产生的原因是初始归并段不足以构成严格 m 叉树，需添加长度为 0 的“虚段”。按照哈

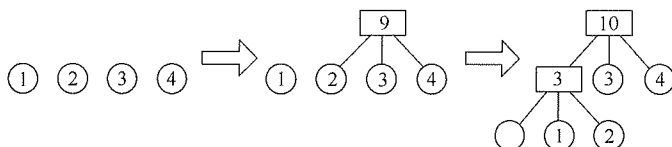


夫曼原则，权为 0 的叶子应该离树根最远，所以虚段一般都在最后一层，作为叶结点。度为 0 的结点有 n_0 个，度为 m 的结点有 n_m 个，则对严格 m 叉树有 $n_0 = (m-1)n_m + 1$ ，由此得 $n_m = (n_0 - 1)/(m-1)$ 。

1) 若 $(n_0 - 1) \% (m-1) = 0$ ，则说明这 n_0 个叶结点（初始归并段）正好可以构成严格 m 叉树。

2) 若 $(n_0 - 1) \% (m-1) = u > 0$ ，则说明这 n_0 个叶结点中有 u 个多余，不能包含在 m 叉归并树中。为构造包含所有 n_0 个初始归并段的 m 叉归并树，应在原有 n_m 个内结点的基础上增加一个内结点。它在归并树中代替一个叶结点位置，被代替的叶结点加上刚才多出的 u 个叶结点，再加上 $m-u-1$ 个虚段，就可建立严格 m 叉树， $5 - (17 \% 4) - 1 = 3$ ，选择选项 C。

举一个最简单的例子，如下图所示。



12. B. 【解析】本题考查根据时钟频率、指令条数和 CPI 来计算程序执行时间。

程序执行时间 = (指令条数 × CPI) / 主频 = $1.2 \times 4 \times 10^9 / (2\text{GHz}) = 2.4\text{s}$ ，所占百分比为 $(2.4/4) \times 100\% = 60\%$ 。下表中给出了几种经常考查的性能指标。

CPU 时钟周期	通常为节拍脉冲或周期 T ，即主频的倒数，它是 CPU 中最小的时间单位
主 频	机器内部主时钟的频率，主频的倒数是 CPU 时钟周期 CPU 时钟周期 = $1/\text{主频}$ ，主频常以 MHz 为单位，1Hz 表示每秒一次
CPI	执行一条指令所需的时钟周期数
CPU 执行时间	运行一个程序所花的时间 CPU 执行时间 = CPU 时钟周期数 / 主频 = (指令条数 × CPI) / 主频
MIPS	每秒执行多少百万条指令， $\text{MIPS} = \text{指令条数} / (\text{执行时间} \times 10^6) = \text{主频} / \text{CPI}$
MFLOPS	每秒执行多少百万次浮点运算， $\text{MFLOPS} = \text{浮点操作次数} / (\text{执行时间} \times 10^6)$

13. A. 【解析】本题考查 C 语言的强制类型转换。

unsigned short 型变量是 16 位无符号整数，us 的机器数为 8000H，在转换为 int 型时，根据 C 语言的整数类型转换规则，短字长向长字长转换时，根据短字长的数据判断是做符号扩展还是做零扩展，由于 unsigned short 型变量是无符号整数，因此做零扩展，转换为 int 型后，真值不变。

14. A. 【解析】本题考查 IEEE 754 单精度浮点数的表示。

IEEE 754 规格化单精度浮点数的阶码范围为 $-126 \sim 127$ ($1 - 127 \sim 254 - 127$)，尾数为 $1.f$ 。最接近 0 的负数的绝对值部分应最小，而又因为 IEEE 754 标准规格化，尾数取 1.0 ；阶码取最小数 -127 ，所以最接近 0 的负数为 $-1.0 \times 2^{-126} = -2^{-126}$ 。

15. D. 【解析】本题考察固态硬盘。

固态硬盘是 ROM，可以随机访问，读很快，但可能需要先擦除才能写，所以写的速度慢很多，选项 A 正确，选项 B 正确。固态硬盘重复写同一个块可能要反复擦写，因此会缩短寿命，选项 C 正确。磨损均衡机制是为了尽量使每个块平均磨损，延长固态硬盘的读写寿命，选项 D 错误。



16. B. 【解析】本题考查 Cache 和辅存的区别。

Cache—主存和主存—辅存这两个层次有以下几个方面的相同点：① 都基于程序访问的局部性原理，将一块相邻的局部信息从慢速存储器复制到快速存储器；② 都必须考虑慢速存储器和快速存储器之间的映射问题；③ 当需要在快速存储器中装入新块而对应位置已满时，都需要考虑将哪一块从快速存储器中替换出来；④ 当在快速存储器中找不到信息时，都要从慢速存储器中将该信息所在的块装入快速存储器。在 CPU 和主存之间加入 Cache 的目的是加快 CPU 访问信息的速度；而在主存和辅存之间采用虚拟存储器机制是为了使程序员写程序时不受内存容量的限制，即增加系统的存储容量，选项 B 正确。

17. B. 【解析】本题考查指令的地址码字段。

缓冲存储器（如 Cache）用来存放最近使用的数据，其内容和调度是由硬件或操作系统完成的，因此不能作为指令的地址码，若操作数是从 Cache 调入的，则只有一种可能，即当操作数在内存中时，正好 Cache 中有它的映像，可以直接从 Cache 中调入操作数，但是不能直接指定某个 Cache 为操作数地址。控制存储器采用 ROM 结构，存放的是微程序，它对软件开发人员是透明的，显然不能作为指令的地址码。CPU 不能直接访问外存单元，若所需的数据存放在外存单元中，则需要先调入主存单元，而指令中只能使用主存地址。综上所述，操作数可以指定的地位只有数据寄存器和主存单元。

18. C. 【解析】本题考查指令周期。

每个指令周期中，比如取指，CPU 一定会访问内存，选项 A 正确。每个指令周期一定大于或等于一个 CPU 时钟周期，选项 B 正确。由于开中断，当前程序在每条指令执行结束时都可能被外部中断打断，选项 D 正确。空操作指令的指令周期中，PC 的内容会改变，选项 C 错误。

19. C. 本题考查指令流水线。

▲注意：流水线的划分方式不是唯一的。本题中第二条、第三条指令会产生 RAW 数据冲突，第三条指令要等待第二条指令写入 ecx 后才能执行，即第三条指令的 ID 在第二条指令的 WB 之后，需要推迟三个阶段，即需要加入三条 nop 指令，流水线的执行情况如下表所示。

IF	ID	EX	MEM	WB					
	IF	ID	EX	MEM	WB				
		nop							
			nop						
				nop					
					IF	ID	EX	MEM	WB

因此，选择选项 C。

20. D. 【解析】本题考查总线的分类与特点。

地址、控制和状态信息都是单向传输的，数据信息是双向传输的。

21. C. 【解析】本题考查中断向量。

因为中断向量就是中断服务程序的入口地址，所以需要找到指定的中断向量，而中断向量是



保存在中断向量表中的。因为 0800H 是中断向量表的地址, 所以 0800H 的内容就是中断向量。

22. A. 【解析】本题考查 CPU 查询中断和设备发起中断的区别。

CPU 总是在一条指令执行结束之后和取下一条指令之前去查询有无中断请求。若此时处于开中断状态, 且有未被屏蔽的中断请求, 则 CPU 执行中断隐指令, 进入中断响应周期。

23. B. 【解析】本题考查操作系统的体系结构。

CPU 时钟频率受限于信号在介质上的传输时间, 若不断提高 CPU 的时钟频率, 则势必导致元器件体积进一步缩小, 而元器件体积缩小是有极限的, 从而导致 CPU 时钟频率的提高也是有极限的, 选项 A 正确。为了使多个处理机协调工作, 系统必须为此付出一定的开销, 因此利用 n 个处理机运行所获得的加速比并不能达到一台处理机的 n 倍, 选项 B 错误。 n 个处理机的系统可以共享一些硬件资源, 因此更加节省投资, 当其中一个处理机出问题时, 可让其他处理机继续处理, 因此多处理机系统更加可靠, 选项 C、D 正确。

24. A. 【解析】本题考查进程的内存管理。

由于内存的共享, 不同进程对应的页表中可能包含内容相同的页表项, 选项 A 正确。当内存采用单一连续分配方式时, 虚拟地址空间等于物理地址空间, 选项 B 错误。页面越小, 可以减少页内碎片, 提高内存利用率, 注意不是外部碎片, 选项 C 错误。分段大小可以不同, 目的是反映程序的逻辑结构并有利于段的共享, 而不是消除外部碎片, 选项 D 错误。

25. B. 【解析】本题考查线程的实现方式。

要注意掌握进程与线程的区别和联系, 以及在具体执行中线程与进程扮演的角色和线程的属性。在多线程模型中, 进程依然是资源分配的基本单元, 而线程是最基本的 CPU 执行单元, 它们共享进程的逻辑地址空间, 但各个线程有自己的栈空间。因此, 选项 I 正确、II 错误。在一对一线程模型中, 一个进程中的每个用户级线程都映射到一个内核级线程, 一个线程被阻塞不影响该进程的其他线程运行状态, 因此选项 III 正确、IV 错误。假如选项 IV 正确, 则凡是遇到等待 I/O 输出的线程都被撤销, 这显然是不合理的, 某个进程被阻塞只会将该进程加入阻塞队列, 当它得到等待的资源时, 就会回到就绪队列。

26. A. 【解析】本题考查调度算法的性质。

基于时间片的调度算法在执行过程中, 进程的执行是以时间片为单位的。多级反馈队列调度算法在各个队列内以 FCFS 原则依次执行时间片, 在底层队列中按照时间片轮转法执行。另外, 没有单独的抢占式调度算法这种说法, 一般都是说某种调度算法是抢占型的或是非抢占型的。

▲注意: 抢占式调度一般指进程的调度算法, 因为所谓的抢占即抢占 CPU, 而作业调度和中级调度并没有抢占的对象, 所以一般也谈不上抢占式调度。

27. B. 【解析】本题考查生产者-消费者问题。

当缓冲区中有空位时, 产品直接放入指针 in 所指向的位置, 因此指针 in 指向的是循环队列中队尾元素的下一个元素; 当消费者需要取产品时, 直接从指针 out 指向的位置取出产品, 因此指针 out 指向的是队首元素。当 $in == out$ 时, 循环队列为空, 缓冲区中 item 的数量为 0; 当 $(in+1) \% BUFFER_SIZE == out$ 时, 缓冲区中 item 的数量为 $BUFFER_SIZE-1$,



此时即循环队列为满的情况。

28. B. 【解析】本题考查首次适应算法的内存分配。

作业 1、2、3 进入主存后，主存的分配情况如图 1 所示（灰色表示空闲空间）。作业 1、3 释放后，主存的分配情况如图 2 所示。作业 4、5 进入系统后，主存的分配情况如图 3 所示。

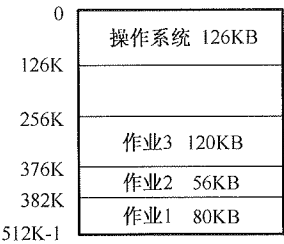


图 1

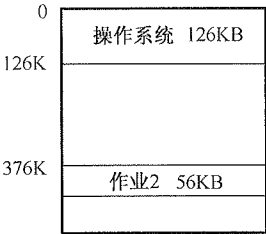


图 2

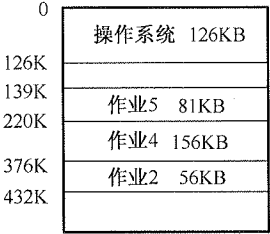


图 3

29. D. 【解析】本题考查页面置换算法。

LRU:

访问串	5	4	3	2	4	3	1	4	3	2	1	5
内 存	5	5	5	2	2	2	1	1	1	2	2	2
		4	4	4	4	4	4	4	4	4	1	1
			3	3	3	3	3	3	3	3	3	5
缺 页	○	○	○	○			○			○	○	○

共缺页 8 次。

FIFO:

访问串	5	4	3	2	4	3	1	4	3	2	1	5
内 存	5	5	5	2	2	2	2	2	3	3	3	5
		4	4	4	4	4	1	1	1	2	2	2
			3	3	3	3	3	4	4	4	1	1
缺 页	○	○	○	○			○	○	○	○	○	○

共缺页 10 次。

分别缺页 8、10 次，选择选项 D。

30. C. 【解析】本题考查磁盘逻辑格式化。

逻辑格式化将初始文件系统数据结构存储到磁盘上，这些数据结构包括空闲空间和已分配空间，一个初始为空的目录，以及建立根目录、初始化保存空闲磁盘块信息的数据结构。

31. D. 【解析】本题考查磁盘的缓冲区。

本题需分情况讨论：若 $T_2 > T_1$ ，即 CPU 处理数据比数据传输慢，则磁盘将数据传输到缓冲区，再传输到用户区进行处理，第一次需要花的时间为 $T_1 + T_2$ ，剩余数据可视为 CPU 进行连续处理，共花费时间 $(n-1)T_2$ ，读入并处理所用的总时间为 $T_1 + nT_2$ 。若 $T_2 < T_1$ ，即 CPU 处理数据比数据传输快，则此时除了第一次可视为 I/O 连续输入，磁盘将数据传输到缓冲区，与缓冲区中的数据传输到用户区及 CPU 处理数据，两者可视为并行执行，那么花费时间主要取决于



磁盘将数据传输到缓冲区所用的时间 T_1 ，前 $n-1$ 次所花的时间共为 $(n-1)T_1$ ，而最后一次的时间 T_1 完成后，还要花时间从缓冲区传输到用户区，且 CPU 还要处理，即要加上时间 T_2 ，读入并处理所用的总时间为 $nT_1 + T_2$ 。综上所述，总时间为 $(n-1)\max(T_1, T_2) + T_1 + T_2$ 。

32. C. 【解析】本题考查磁盘的地址结构。

每个柱面有 12 条磁道，每条磁道有 16 个扇区，因此一个柱面上有 192 个扇区。 $5687/192 = 29$ 余 119，因此柱面号为 29； $119/16 = 7$ 余 7，因此磁道号为 7、扇区号为 7。

33. D. 【解析】本题考查 TCP/IP 体系结构的原则和特点。

上层协议可以选择不同性质的服务，如数据链路层可以使用面向连接的 PPP，但网络层使用的是无连接的 IP，选项 A 错误。上层协议可以选择不同性质的服务，如网络层使用无连接的 IP，但传输层可以使用面向连接的 TCP，选项 B 错误。某些层次可能只提供一种服务，如网络层都使用 IP，它只提供无连接的服务，选项 C 错误，选项 D 正确。

34. B 【解析】本题考查物理层的功能。

物理层考虑的是怎样才能在连接各种计算机的传输媒介上传输比特流，而不是具体的传输媒介。物理层的任务是透明地传输比特流，也就是说，发送方发送 1（或 0）时，接收方应当收到 1（或 0）而非 0（或 1）。因此，物理层要考虑用多大的电压代表“1”或“0”，以及接收方如何识别出发送方所发送的比特。物理层还要确定连接电缆的插头应当有多少根引脚以及各引脚应如何连接。传递信息所用的一些传输媒介并不属于物理层的范畴，而在物理层协议的下面，因此也有人将传输媒介当作第 0 层。

35. A. 【解析】本题考查停止-等待协议和信道利用率计算。

停止-等待协议每发出一帧，需要收到该帧的确认后才能发送下一帧，确认帧的长度忽略不计，一个发送周期 = 发送时间 + 往返传播时间，信道利用率 = 发送时间 / (发送时间 + 往返传播时间) = 50%，所以发送时间 = 往返传播时间 = $RTT = 2 \times 200\text{ms} = 400\text{ms}$ ，帧长 = 发送时间 $\times$ 传输率 = $400\text{ms} \times 4000\text{b/s} = 1600\text{bit} = 200\text{B}$ 。

36. A. 【解析】本题考查 VLAN。

虚拟局域网（VLAN）是将一种物理 LAN 逻辑上划分成多个广播域的通信技术。一个 VLAN 是一个广播域。不同 VLAN 由路由器（三层交换机）划分，属于不同网段，可通过三层交换机通信。二层交换机只能分割冲突域而不能分割广播域，一个 VLAN 可包含多个二层交换机分割的区域，因此不一定链接同一个交换机。

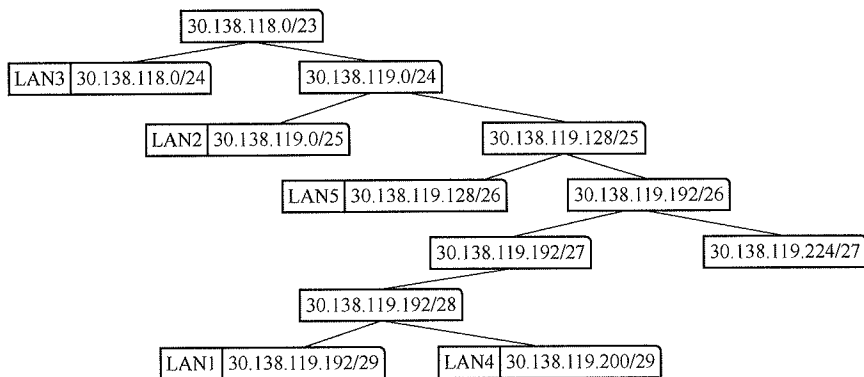
▲注意：408 考试中所说的交换机都指二层交换机。

37. B. 【解析】本题考查 IP 数据报的分片。

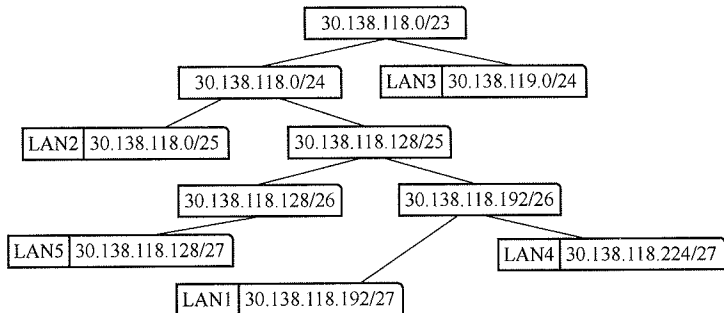
第二个局域网所能传输的最长数据帧中的数据部分只有 1200bit，可见每个 IP 数据报的最大长度是 1200bit，其数据部分（TCP 报文段）最多为 IP 数据报的总长度 - IP 数据报的首部 = $1200\text{bit} - 160\text{bit} = 1040\text{bit}$ ，因此 TCP 交给 IP 的 3200bit 数据必须划分成 4 个 IP 数据报，加上首部长度后有 $3200\text{bit} + 160\text{bit} \times 4 = 3840\text{bit}$ ，因此第二个局域网要向上传输的数据大小为 3840bit。

38. C。【解析】本题考查 IP 地址的划分。

分配网络前缀时，应先分配地址数量较多的前缀，题目没有说 LAN1 有几个主机，但至少需要 3 个地址以供 3 个路由器的接口使用。分配 IP 地址时，要特别注意每一组分配到的 IP 地址空间不能有重叠，如下图所示，可以用画二叉树的方式来验证两种划分方式，经过验证，两种方案都合理。



第一组分配方案



第二组分配方案

39. A。【解析】本题考查对 UDP 检验和的理解。

UDP 的上层用户可以关闭检验和功能（在 UDP 的传输过程中不使用检验和），这样做的好处是可以提高 UDP 的传输速度（但要牺牲一些可靠性）。若发送方决定不使用检验和，则发送方的检验和的值应全置为 0。这表示该数值不是计算出来的，而是发送方关闭了检验和的检错功能，选项 I 正确。UDP 规定：若检验和的计算结果刚好全是 0，则人为地将其置为全 1（以区分关闭检验和），选项 II 正确。按照 UDP 检验和的计算方法，检验和的计算值不可能刚好是全 1，选项 III 错误。

40. C。【解析】本题考查 HTTP 和网络协议三要素。

网络协议三要素中的语义规定了要发出何种控制信息、完成何种动作以及做出何种响应。

二、综合应用题

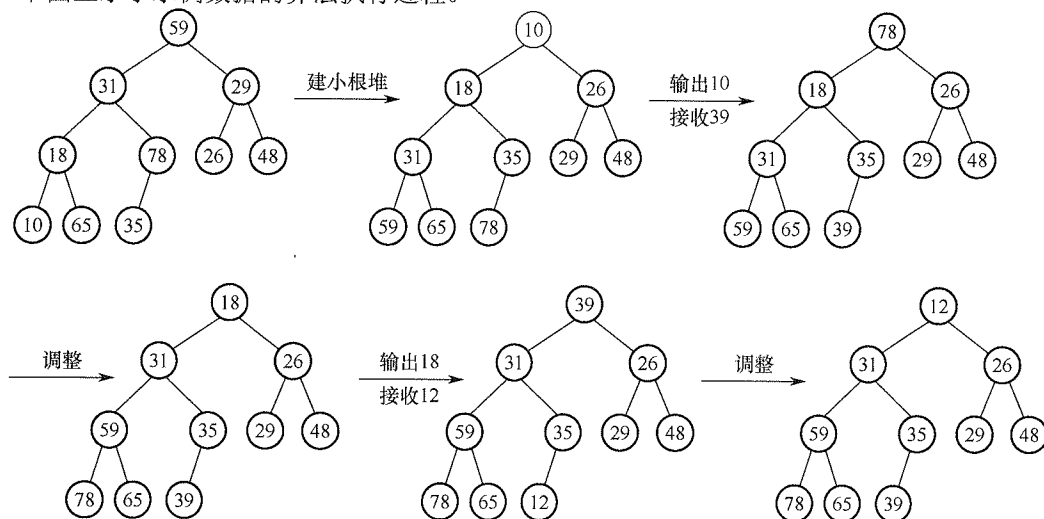
41. 【解析】

1) 采用小根堆的方式筛选最小元素。当要接收新元素时，将堆顶元素输出，并将堆底元素送



入堆顶，然后将新元素插入堆底，最后重新将其调整为小根堆。

2) 下图显示了示例数据的算法执行过程。



42. 【解析】

- 1) 若两棵二叉树都是空树，则它们是镜像相似的；若一棵树是空树，另一棵树不是空树，则它们一定不是镜像相似的。否则，要递归地判断 T_1 的左子树与 T_2 的右子树是否镜像相似，并且判断 T_1 的右子树与 T_2 的左子树是否镜像相似。若相似，则两棵二叉树是镜像相似的；否则，两棵二叉树不是镜像相似的。
- 2) 算法的实现如下：

```
bool mirror(BiTree T1, BiTree T2){
    if (T1==NULL&&T2==NULL) //若两棵树都是空树，则它们是镜像相似的
        return true;
    if (T1==NULL&&T2!=NULL || T1!=NULL&&T2==NULL)
        return false;    //一棵树为空树，一棵树不为空树，则它们一定不是镜像相似的

    if (mirror(T1->lchild, T2->rchild) && mirror(T1->rchild, T2->lchild))
        return true;    //递归地判断左、右子树
    else
        return false;
}
```

43. 【解析】

- 1) $0xFFFF\ FFFF - 0x7FFF\ FFFF = 0xFFFF\ FFFF + 0x8000\ 0001 = 0x8000\ 0000$ ，对应进位 $C = 1$ ， $Sub = 1$ ，因此 $CF = C \oplus Sub = 0$ 。由于负数 - 负数的结果仍为负数且非零，因此 $OF = 0$ 、 $ZF = 0$ 、 $SF = 1$ 。
- 2) $b[5] = -1$ ， $a[5]$ 是最大的正数，因此 $if(b[5] < a[5])$ 成立， $b[5] = 1$ 。
- 3) $-15.25 = -1.11101B \times 2^3$ ，符号位为 1，阶码为 $3 + 127 = 1000\ 0010B$ ，尾数部分为 $111\ 0100\ 0000\ 0000\ 0000\ 0000$ ，对应的机器数为 $0xC174\ 0000$ ，由于采用小端方式存储，因此存放的顺序依次为 $0x00, 0x00, 0x74, 0xC1$ 。



44. 【解析】

- 1) 总线 A 在传输数据时以每个时钟周期 2 个字的速度进行, 因此其最大数据传输率为 $32\text{bit} \times 2 \times 100\text{MHz} = 6400\text{Mb/s} = 800\text{MB/s}$ 。总线 B 在传输数据时以每个时钟周期 1 个字的速度进行, 因此其最大数据传输率为 $32\text{bit} \times 100\text{MHz} = 3200\text{Mb/s} = 400\text{MB/s}$ 。
- 2) 总线 A 虽然每个时钟周期传输 2 个字, 但在单个字的总线事务中每次只需传输 1 个字, 每个总线事务占 $1 + 2 + 1 = 4$ 个时钟周期, 因此连续进行单个字的存储器读总线事务时, 数据传输率为 $4 \times 100\text{MB/s} \div 4 = 100\text{MB/s}$ 。总线 B 每个时钟周期传输 1 个字, 总线 B 的单个字的存储器读总线事务占 $1 + 2 + 1 = 4$ 个时钟周期, 因此连续进行单个字的存储器读总线事务时, 数据传输率也为 100MB/s 。
- 3) 总线 A 的单个字的存储器写总线事务和单个字的存储器读总线事务的情况一样, 数据传输率都是 100MB/s 。总线 B 的单个字的存储器写总线事务占 $1 + 2 = 3$ 个时钟周期, 因此连续进行单个字的存储器写总线事务时, 数据传输率为 $4 \times 100/3\text{MB/s} = 133.3\text{MB/s}$ 。
- 4) 总线 A 进行存储器读或写 8 个字所用时间都为 $1 + 2 + 8/2 = 7$ 个时钟周期, 因此连续进行多个 8 个字突发传输总线事务时, 数据传输率为 $8 \times 4 \times 100/7\text{MB/s} = 457\text{MB/s}$ 。总线 B 的存储器读事务和存储器写事务所用时间不等, 突发读 8 个字所用的时间为 $1 + 2 + 8 = 11$ 个时钟周期, 突发写 8 个字所用的时间为 $1 + 2 + 7 = 10$ 个时钟周期, 因此当 60% 是读操作总线事务、40% 是写操作总线事务时, 数据传输率为 $8 \times 4 \times 100/11 \times 60\% + 8 \times 4 \times 100/10 \times 40\% \approx 303\text{MB/s}$ 。

45. 【解析】

本题将进程调度与内存分配结合起来, 思考的关键是: 只有处于就绪态的进程才能参与 CPU 的竞争, 也就是说, 一个进程必须获得 CPU 之外的全部所需资源才能被调度。因此, 做题时只要将资源分配情况和进程的状态列出来, 就能清楚地判断出哪个进程在进程调度时可以获得 CPU。

- 1) 在整个过程中, 内存、打印机的使用情况和进程的状态信息如下表所示。

时间段	已分配分区 (进程, 始址, 大小)	空闲分区 (始址, 大小)	打印机 R	进程状态
0~4	(1, 0, 15K)	(15K, 85K)	R→P ₁	P ₁ 执行
4~8	(1, 0, 15K) (2, 15K, 30K)	(45K, 55K)	R→P ₁ P ₂ →R	P ₁ 执行, P ₂ 阻塞
8~10	(2, 15K, 30K)	(0, 15K) (45K, 55K)	R→P ₂	P ₁ 完成, P ₂ 执行
10~11	(2, 15K, 30K)	(0, 15K) (45K, 55K)	R→P ₂	P ₁ 完成, P ₂ 执行, P ₃ 后备
11~12	(2, 15K, 30K) (4, 45K, 20K)	(0, 15K) (65K, 35K)	R→P ₂ P ₄ →R	P ₁ 完成, P ₂ 执行, P ₃ 后备, P ₄ 阻塞
12~16	(4, 45K, 20K)	(0, 45K) (65K, 35K)	R→P ₄	P ₁ 完成, P ₂ 完成, P ₃ 后备, P ₄ 执行
16~32	(4, 45K, 20K) (5, 0, 10K)	(10K, 35K) (65K, 35K)	R→P ₄ P ₅ →R	P ₁ 完成, P ₂ 完成, P ₃ 后备, P ₄ 执行, P ₅ 阻塞
32~33	(5, 0, 10K) (3, 10K, 60K)	(70K, 30K)	R→P ₅	P ₁ 完成, P ₂ 完成, P ₃ 执行, P ₄ 完成, P ₅ 就绪
33~47	(5, 0, 10K)	(10K, 90K)	R→P ₅	P ₁ 完成, P ₂ 完成, P ₃ 完成, P ₄ 完成, P ₅ 执行
47~		(0K, 100K)	空闲	P ₁ , P ₂ , P ₃ , P ₄ , P ₅ 全完成



可以看出,选中进程的次序为 P_1, P_2, P_4, P_3, P_5 。

2) 从上述分析不难看出,在时刻 47,所有进程执行完毕。

46. 【解析】

- 1) 磁盘块大小为 4KB, file1.txt 占 5MB, 即占 $5\text{MB}/4\text{KB} = 1280$ 块, 其中 10 个直接地址项指向 10 个磁盘块, 1 个一级间接索引地址项指向 $4\text{KB}/4\text{B} = 1024$ 个磁盘块, 1 个二级间接索引地址项指向 1024×1024 个磁盘块。因此, 分三种情况讨论: ① 若读取的数据在文件的前 40KB, 则只需 1 次读磁盘块操作; ② 若读取的数据在文件的 40KB 和 $(1024+10) \times 4\text{KB} = 4136\text{KB}$ 之间, 则需要先读取 1 次一级间接索引块, 再读取 1 次对应的数据块, 共需 2 次读磁盘块操作; ③ 同理, 若读取的数据在文件的 4136KB 和 5MB 之间, 则需 3 次读磁盘块操作。
- 2) 插入 12KB 的数据, 即向文件中插入 3 个磁盘块, 如果原文件长度小于或等于 28KB (7 个磁盘块), 就不需要修改一级间接索引, 只需修改直接索引项并写回索引节点所在的磁盘块, 加上写回 3 个插入的磁盘块, 共需 4 次写磁盘块操作。如果原文件长度大于 28KB 且小于 4124KB (10+1021 个磁盘块), 就需要修改索引节点中的一级间接索引项所指向的磁盘块, 并写回索引节点所在的磁盘块 (文件大小发生变化) 和一级间接索引项所指向的磁盘块, 加上写回 3 个插入的磁盘块, 共需 5 次写磁盘操作。
- 3) 对文件进行打开操作时, 即 `open()` 系统调用, 传递的参数是文件名; 对文件进行关闭操作时, 即 `close()` 系统调用, 此时已得到文件描述符 `fd`, 传递的参数是文件描述符 `fd`。

47. 【解析】

- 1) B 的 IP 地址是 B 发送的 IP 分组首部中的源地址, 即 106.39.41.16 (6A27 2910)。A 的端口号是 TCP 首部中的目的端口号 C121H。IP 分组总长度为 05ACH $\times$ 1B, IP 首部长度为 $5 \times 4\text{B}$, TCP 首部长度为 $5 \times 4\text{B}$, TCP 数据长度 = IP 总长度 - IP 首部长度 - TCP 首部长度 = 1412B。
- 2) 根据 IP 分组前 64 字节内容及 TCP 首部结构可知, 序号为 080AH, 确认号为 020CH, 报文段数据的初始序号为 0, 按字节编号, 被正确接收和确认后, A 收到的数据字节数 = 序号 + 数据报文长度 = 3470B, B 收到的数据字节数 = 确认号 - 1 = 523B。
- 3) 每收到一个报文段的确认, `cwnd` 加 1, 因此 `cwnd` = 5。B 检测到报文段超时会执行慢开始算法, `cwnd` = 1, 每个报文段最多搭载 1460 字节的应用层数据, 因此最多可发送 1460B。



全国硕士研究生入学统一考试

计算机科学与技术学科联考

计算机专业基础综合考试模拟试卷（五）参考答案

一、单项选择题（第 01~40 题）

01. B 02. B 03. A 04. A 05. D 06. A 07. C 08. B
09. D 10. C 11. B 12. B 13. D 14. A 15. D 16. D
17. B 18. A 19. C 20. B 21. C 22. C 23. D 24. D
25. D 26. C 27. D 28. D 29. D 30. A 31. C 32. C
33. B 34. B 35. A 36. B 37. B 38. C 39. C 40. C

01. B。【解析】本题考查快速排序的特点。

快速排序要求存储的元素具有随机访问的特性，而单链表、静态链表和双链表都不具有随机访问的特性，因此不适用于快速排序，只有数组适合。

02. B。【解析】本题考查共享栈。

题干所说“该共享栈在一端非空时，栈指针指向当前元素的下一位置”，因此当两端非空且栈满时，S1 的栈顶指针应大于 S2 的栈顶指针，且满足 $S1.top - S2.top = 1$ 。

03. A。【解析】本题考查栈的应用。设中间计算结果 $S1 = C/D$, $S2 = (B + C/D)$ ，扫描过程见下表。

扫描字符	运算数栈（扫描后）	运算符栈（扫描后）	说 明
A	A		‘A’ 入栈
-	A	-	‘-’ 入栈
(	A	-(	‘(’ 入栈
B	AB	-(	‘B’ 入栈
+	AB	-(+	‘+’ 入栈
C	ABC	-(+	‘C’ 入栈
/	ABC	-(+ /	‘/’ 入栈
D	ABCD	-(+ /	‘D’ 入栈
	ABS1	-(+	计算 S1
)	ABS1	-(+)	‘)’ 入栈
	AS2	-	计算 S2
×	AS2	- ×	‘×’ 入栈
E	AS2E	- ×	‘E’ 入栈

扫描到 E 时，运算符栈中的内容依次是“-×”。



04. A. 【解析】本题考查完全二叉树的性质。

根结点在第 0 层，代入结点编号进行验证，不难发现每个结点的编号和所在层次的关系为： $\text{层次} = \lfloor \log_2(\text{编号} + 1) \rfloor$ ，因此选择选项 A。

05. D. 【解析】本题考查几种特殊二叉树的性质。

对于选项 A，即满二叉树，设层数为 h ，则 $2^h - 1 = n$ ，求出 h ，叶结点都在最后一层上，即叶结点数为 2^{h-1} 。对于选项 B，即完全二叉树，度为 1 的结点数为 0 或 1， $N = 2N_0 + N_1 + 1$ ，则 $N_0 = \lfloor (n+1)/2 \rfloor$ 。对于选项 C，即哈夫曼树，只有度为 2 和 0 的结点， $N_0 = N_2 + 1$ ， $N_0 + N_2 = n$ ，即 $N_0 = (n+1)/2$ ，可得叶结点数。对于选项 D，无法求出叶结点数。

06. A. 【解析】本题考查 Kruskal 算法。

Kruskal 算法需要按边的权值排序，最小堆能高效动态地获取最小边，选项 A 正确。并查集用于管理顶点的连通性，不存储边，选项 D 错误。选项 B 和 C 与算法的实现无关。

07. C. 【解析】本题考查图的算法。

拓扑排序使用邻接表时的复杂度是 $O(m+n)$ ，序列每次选出一个顶点后，都要更新从该顶点出发的边，而采用邻接矩阵时，每次都会搜索所有顶点，复杂度变为 $O(n^2)$ ，因此邻接表的效率更高。广度优先搜索和深度优先搜索都使用邻接表。普里姆算法每次选取顶点后，也要更新从该顶点出发的边，所以邻接表的操作效率更高。选项 I、II、III、IV 正确，因此选择选项 C。

08. B. 【解析】本题考查排序二叉树的查找。

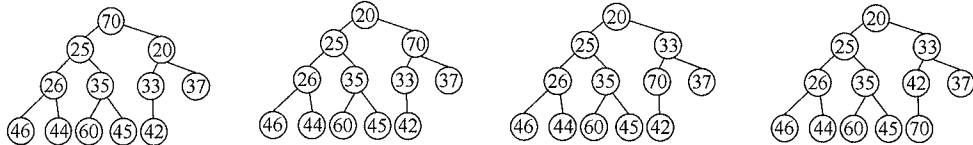
排序二叉树的查找原则是查找范围要越来越小。例如，选项 B 查找 52，首先查找到 95，比 52 大，此时查找范围是 $(-\infty, 90)$ ；之后查找到 59，比 52 大，范围变为 $(-\infty, 59)$ ；之后查找到 84，不在查找范围 $(-\infty, 59)$ 上，所以矛盾不会出现。

09. D. 【解析】本题考查散列表的应用。

装填因子定义为表中记录数/散列表的长度，若装填因子 a 可以大于或等于 1，则说明表中的记录数可以大于散列表的长度，而这只能是采用链地址法处理冲突的情况。

10. C. 【解析】本题考查堆的调整过程。

堆的调整过程如下图所示，可知 70 最后的位置为 C。



11. B. 【解析】本题考查外部排序的特点。

使用置换-选择排序是为了增加初始归并段的长度来减少初始归并段的数量，选项 A 错误。败者树的作用是减少元素之间的比较次数，而不是增大归并路数，选项 C 错误。最佳归并树是严格 k 叉树，并不一定是二叉哈夫曼树，选项 D 错误。

12. B. 【解析】本题考查控制器的功能。

数据和指令通过总线从内存传至 CPU，但传输的是指令还是数据总线本身是无法判断的，所以通过总线无法区分数据和指令，而主存能通过总线和指令周期区分地址和非地址数据。运算器是对数据进行逻辑运算的部件，控制存储器是存放微指令的部件，二者均无区分数据和指令的功能。

▲注意：在控制器的控制下，计算机在不同的阶段对存储器进行读写操作时，取出的代码有不同的用处。在取指阶段读出的二进制代码是指令，在执行阶段读出的二进制代码则是数据。

13. D. 【解析】本题考查补码和浮点数运算的特点。

定点补码运算，符号位参与运算，选项 I 显然错误。浮点数由阶码和尾数组成，进行浮点数运算时，阶码和尾数都要参与，选项 II 正确。进行乘除运算时，阶码显然只进行加、减操作，选项 III 正确。浮点数的正负由尾数的符号决定，而阶码决定浮点数的表示范围，当阶码为负数时，浮点数小于 1，选项 IV 错误。浮点数做加、减运算时，尾数进行的是加、减运算，选项 V 错误。

14. A. 【解析】本题考查不同类型数据转换的精度问题。

float 型数据的范围是 $-3.4E+38 \sim 3.4E+38$ ，比 int 型数据的大，所以不会溢出，但有可能因为精度问题出现舍入，选项 A 错误。double 型数据是双精度浮点数，其范围 $> \text{float}$ 型数据 $> \text{int}$ 型数据，选项 B 正确。double 型数据转换成 float 型数据，范围变小，精度也变小，可能会溢出，也可能舍入，选项 C 正确。double 型数据转换成 int 型数据，只要不是整数就会舍入，选项 D 正确。

15. D. 【解析】本题考查磁盘阵列 (RAID)。

磁盘阵列 (RAID) 是将相同的数据存储在多个硬盘上不同地方的方法。通过将数据放在多个硬盘上，输入/输出操作能以平衡的方式交叠，因此能改良性能。此外，因为多个硬盘增加了平均故障间隔时间，所以存储冗余数据也增加了容错。RAID 不是减少冗余，而是增加冗余 (RAID0 不增加冗余)。

16. D. 【解析】本题考查 Cache 映射。

对于全相联的 Cache，物理地址格式为：标记、块内地址。32 位 = 4B。块内地址 2 位，主存 32 位，所以标记 30 位。Cache 有 32K 块，每块需要的控制位：30 (标记位) + 1 (有效位) + 1 (脏位) = 32 位。数据加控制位共 $32K \times (32 \text{ 位} + 32 \text{ 位}) = 2048K \text{ 位}$ 。

17. B. 【解析】本题考查 FIFO 算法。

FIFO 算法是指淘汰先进入的，易知替换顺序如下表所示。

走 向	0	1	2	4	2	3	0	2	1	3	2	3	0	1	4
c			2	2	2	2	0	0	0	3	3	3	3	3	3
b		1	1	1	1	3	3	3	1	1	1	1	1	1	4
a	0	0	0	4	4	4	4	2	2	2	2	2	0	0	0
命中否					√						√	√		√	

表中除了标注为命中的，其余均未命中，所以命中率为 $4/15 = 26.7\%$ 。



18. A. 【解析】本题考查基址寻址和变址寻址的区别。

两者的有效地址都加上了对应寄存器的内容，都扩大了指令的寻址范围，选项 I 正确。变址寻址适合处理数组、编写循环程序，选项 II 正确。基址寻址有利于多道程序设计，选项 III 正确。基址寄存器的内容由操作系统或管理程序确定，在执行过程中其内容不变，而变址寄存器的内容由用户确定，在执行过程中其内容可变，选项 IV 和 V 错误。

▲注意：基址寻址和变址寻址的真实地址 EA 都是形式地址 A 加上一个寄存器中的内容。

19. C. 【解析】本题考查部件的“透明性”。

所谓透明，是指那些不属于自己管的部分。在计算机系统中，下层机器级的概念性结构功能特性，对上层机器语言的程序员来说是透明的。汇编程序员在编程时，不需要考虑指令缓冲器、移位器、乘法器和先行进位链等部件。移位器、乘法器和先行进位链属于运算器的设计。

▲注意：在计算机中，客观存在的事物或属性从某个角度看不到，称为“透明”。这与日常生活中的“透明”正好相反，日常生活中的透明就是要公开，让大家看得到。常考的关于透明性的计算机部件有移位器、指令缓冲器、时标发生器、条件寄存器、乘法器、主存地址寄存器等。

20. B. 【解析】本题考查硬件多线程的原理。

在支持硬件多线程的 CPU 中，必须为每个线程提供单独的通用寄存器组、单独的程序计数器等，线程的切换只需激活选中的寄存器，从而省略了与存储器数据交换的环节，大大减少了线程切换的开销，而不是多个线程共享核内的通用寄存器组和 PC，选项 B 错误。

21. C. 【解析】本题考查总线猝发传输。

25ns 是 5 个周期，根据猝发传输，第一个周期传输地址和命令，后 4 个周期传输数据，即传输了 4 个 32 位数据，共 128 位。

22. C. 【解析】本题考查外部中断。

选项 A、B 和 D 都和本条执行的指令有关，是内部中断。而磁盘属于外设，寻道结束时会通过外部中断告知 CPU，选择选项 C。

23. D 【解析】本题考查中断机制。

中断是多道程序得以实现的基础，通过中断，操作系统可在不同的进程之间进行切换，从而实现多任务处理。中断是设备管理的基础，中断可以使 CPU 在等待 I/O 设备完成操作时进行其他任务，中断机制实现了 CPU 和 I/O 设备并行工作，从而提高了 CPU 的利用率。中断也是 I/O 系统中最低的一层，是整个 I/O 系统的基础，因此三个说法均正确。

24. D. 【解析】本题考查进程的状态。

系统当前未执行进程调度程序，除非系统当前处于死锁状态，否则总有一个正在运行的进程，其余的进程状态则不能确定，可能处于就绪态，也可能处于等待态，因此选项 A、B、C 都是正确的。若当前没有正在运行的进程，则所有进程一定都处于等待态，不可能有就绪进程。当没有运行进程而就绪队列中又有进程时，操作系统一定会从就绪队列中选取一个进程并将其变成运行进程。

25. D. 【解析】本题考查进程的优先级。

由于 I/O 操作需要及时完成，它没有办法长时间保存所要输入/输出的数据，因此通常 I/O 型作业的优先级要高于计算型作业的优先级，选项 I 错误。系统进程的优先级应高于用户进程的优先级。作业的优先级与长作业、短作业或系统资源要求的多少没有必然的关系。在动态优先级中。随着进程执行时间的增加，其优先级降低；随着作业等待时间的增加，其优先级上升，选项 II、III 错误。对资源要求低的作业应当给予较高的优先级，让其更早完成，释放出占有的资源，以便其他作业顺利进行。若给对资源要求多的作业更高的优先级，则在没有有效手段避免死锁的情况下，对资源要求多的多个作业共同工作容易造成死锁，选项 IV 错误。

26. C. 【解析】本题考查页帧的分配策略。

若指令执行完成前发生缺页错误，应重启指令，因此要有足够的页帧来容纳任何单个指令可能引用的所有页面。一级间接寻址需要访问指令页（存放指令）、间接地址页（存放操作数的地址）和操作数页（存放实际数据），因此至少给一个进程分配 3 个页帧。

27. D. 【解析】本题考查分页存储管理。

增加页面大小一般来说可以减少缺页中断次数，但不存在反比关系，有时增加页面大小反而会引起缺页增加（FIFO 算法与 Belady 异常），选项 I 错误。分页存储管理方案解决了一个作业在主存中可以不连续存放的问题，因此要注意请求分页存储管理和分页存储管理的区别，选项 II 错误。页面减小将导致页表增大，即页表占用的内存增大，也可能导致缺页数量增加，选项 III 错误。虚存大小只取决于虚拟地址的位数，而与辅存和主存的容量没有必然关系，选项 IV 错误。

28. D. 【解析】本题考查内存保护。

在地址变换过程中，可能会因为缺页、操作保护和越界保护而产生中断。首先，当你访问的页内地址超过页长度时就会发生地址越界，而当你访问的页面不在内存中时就会产生缺页中断，而访问权限错误是当你执行的操作与页表中的保护位（如读写位、用户/系统属性位等）不一致时发生的，例如，你对一些代码页执行了写操作，而这些代码页是不允许写操作的，选项 I、II 和 III 正确，但肯定不会发生内存溢出（内容不足）的现象，选项 IV 错误。

29. D. 【解析】本题考查“打开”文件系统调用。

当用户要求对一个文件实施多次读、写或其他操作时，每次都要从检索目录开始。为了避免多次重复地检索目录，在大多数操作系统中都引入了“打开”这个文件系统调用，当用户第一次请求对某文件进行操作时，须先利用“打开”文件系统调用这个操作，当进程用完文件后，若当前系统中还有其他进程正在使用该文件，则不会立即将文件的控制信息写回外存，选项 D 错误。

30. A. 【解析】本题考查混合索引分配。

$420000 \div 1024 = 410$ 余 160，因此物理块号为 $10 + 256 < 410$ ，通过二次间接索引，在第 11 个地址中可得到一次间接地址，由此得到二次间接地址，再找到物理块号，其块内偏移地址为 160。

31. C 【解析】本题考查设备驱动程序的功能。

设备驱动程序用于直接控制计算机与相关硬件进行 I/O 操作，因为网卡是硬件设备，所以需要



要通过设备驱动程序来控制其工作。其他三个选项都是操作系统或应用程序的功能，不需要用到设备驱动程序。

32. C. 【解析】本题考查虚拟设备的概念。

虚拟设备的功能是将一个物理设备变成多个对应的逻辑设备，选项 C 正确。选项 A 出现概念性错误，选项 B 描述的是设置统一的 I/O 接口，选项 D 描述的是虚拟内存的概念。

33. B. 【解析】本题考查奈奎斯特定理和香农定理。

物理层基本考查奈奎斯特和香农两个公式，它们是两种方式计算的上界。若给出了每个码元的离散电平数，则要用奈奎斯特定理；若给出了信噪比，则要用香农定理。若同时使用两个定理，上界就需要取最小值。本题既给出了单个码元的离散电平数，又给出了信噪比，所以奈奎斯特定理和香农定理都要使用：

奈奎斯特定理：速率 $= 2W\log_2 C = 2W \times 2 = 4W$ 。

香农定理：速率 $= W\log_2(1 + S/N) = W\log_2 1001 = 10W$ 。

速率两者取小，即取 $4W$ ，所以 $W = 8/4\text{kHz} = 2\text{kHz}$ ，因此选择选项 B。

34. B. 【解析】本题考查码分多址的计算。

A 的内积：

$$(-1+1-3+1-1-3+1+1) \cdot (-1-1-1+1+1-1+1+1)/8 = (+1-1+3+1-1+3+1+1)/8 = 1。$$

B 的内积：

$$(-1+1-3+1-1-3+1+1) \cdot (-1-1+1-1+1+1+1-1)/8 = (+1-1-3-1-1-3+1-1)/8 = -1。$$

C 的内积：

$$(-1+1-3+1-1-3+1+1) \cdot (-1+1-1+1+1+1-1-1)/8 = (+1+1+3+1-1-3-1-1)/8 = 0。$$

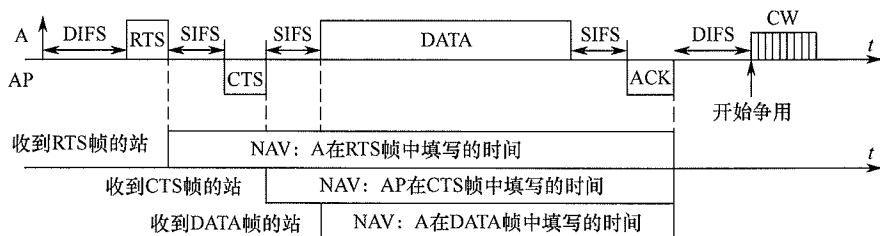
D 的内积：

$$(-1+1-3+1-1-3+1+1) \cdot (-1+1-1-1-1-1+1-1)/8 = (+1+1+3-1+1+3+1-1)/8 = 1。$$

因此，选项 A 和 D 发送了 1，选项 B 发送了 0，而选项 C 未发送数据。

35. A. 【解析】本题考查 CSMA/CA 协议和 NAV 的概念。

如下图所示，当 AP 收到主机 A 发来的 RTS 帧时，会在等待一个 SIFS 的时间后发送相应的 CTS 帧，并在该 CTS 帧中设置 NAV 值，要求其他主机在 $[SIFS + DATA + SIFS + ACK]$ 这段时间内不能打扰 AP 和主机 A 的通信，其中，数据发送时间 $DATA = 1000B \div 50Mb/s = 160\mu s$ ，因此，NAV 值设置为 $SIFS + DATA + SIFS + ACK = 28 + 160 + 28 + 2 = 218\mu s$ 。



36. B. 【解析】本题考查路由表的匹配动作。

经过与路由表比较，发现该目的地址没有与之对应的要达到的网络地址，而在该路由表中有



默认路由, 根据相关规定, 只要目的网络都不匹配, 一律选择默认路由。因此, 下一跳的地址就是默认路由所对应的 IP 地址, 即 192.168.2.66。

37. B. 【解析】本题考查 IP 分组的分片。

对于选项 I, 标识字段在 IP 分组进行分片时, 其值就被复制到所有数据报片的标识字段中, 但其值不变, 选项 I 无变化。对于选项 II 和选项 III, 路由器分片后, 标志字段的 MF、DF 字段均发生相应的变化, 而且由于数据部分长度发生变化, 片偏移字段也发生变化, 因此选项 II、III 均发生变化。对于选项 IV, 总长度字段是首部和数据部分之和的长度, 不是指未分片前的数据报长度, 而是指分片后的每个分片的首部长度与数据长度的总和, 所以 IV 发生变化。对于选项 V, 首部检验和字段需要对整个首部进行检验, 一旦有字段发生变化, 它会发生改变, 所以选项 V 也发生变化。

38. C. 【解析】本题考查 UDP 的首部。

UDP 数据报的首部格式如下图所示。可知, 源端口号为 CB 84, 转换成十进制数为 52100; 目的端口号为 00 0D, 转换成十进制数为 13, 小于 1024, 为熟知端口号; UDP 数据报的总长度为 00 1C, 转换成十进制数为 28, 因此数据载荷长度为 $28 - 8 = 20\text{B}$ 。目的端口号 13 为熟知端口号, 因此该 UDP 数据报是客户端到服务器方向的。

2B	2B	2B	2B
源端口	目的端口	长度	检验和

39. C. 【解析】本题考查 TCP 的重传机制。

在 TCP 的传输过程中, 假设有一个确认报文段丢失, 但若在重传计时器到期之前收到了对更高序号的数据段的确认, 则不会引起重传。若这个确认报文段对应的数据段是本次传输的最后一个数据段, 则一定没有更高序号的确认, 因此一定会引起数据的重传。

40. C. 【解析】本题考查客户端/服务器模型的概念。

客户端是服务请求方, 服务器是服务提供方, 二者的交互由客户端发起。客户端是连接的请求方, 在连接未建立之前, 服务器在端口 80 上监听, 当确立连接后会转到其他端口号, 而客户端的端口号不固定。这时, 客户端必须知道服务器的地址才能发出请求, 很明显服务器事先不需要知道客户端的地址。一旦建立连接, 服务器就能主动发送数据给客户端 (浏览器显示的内容来自服务器), 用于一些消息的通知 (如一些错误的通知)。在客户端/服务器模型中, 默认端口号通常都指服务器, 而客户端的端口号通常是动态分配的。

二、综合应用题

41. 【解析】

1) 平均查找长度为所有元素的查找长度相加再除以元素总数, 计算过程如下:

$$\text{平均查找长度 (ASL)} = (1 + 2 + 3 + 4 + 5 + 6) / 6 = 3.5$$

2) 按照查找概率从大到小排列, 可使顺序查找的平均查找长度最小, 所以元素的排列可以是

$$\{a_6, a_1, a_0, a_5, a_4, a_3\}。$$



- 3) 第 i 个元素的查找长度为 i , 用所有元素的查找长度乘以查找概率再相加, 就可以得到平均查找长度, 计算过程如下:

$$ASL = 1 \times 1/2 + 2 \times 1/2^2 + 3 \times 1/2^3 + \dots + n \times 1/2^n = 2 - (1/2)^n - n(1/2)^{n+1}$$

42. 【解析】

- 1) 算法的基本设计思想:

- ① 在数组尾部从后往前, 找到第一个奇数号元素, 将此元素与其前面的偶数号元素交换。这样, 就形成了两个前后相连且相对顺序不变的奇数号元素“块”。
- ② 暂存①中“块”前面的偶数号元素, 将“块”内奇数号元素依次前移, 然后将暂存的偶数号元素复制到空出来的数组单元中, 形成三个连续的奇数号元素“块”。
- ③ 暂存②中“块”前面的偶数号元素, 将“块”内奇数号元素依次前移, 然后将暂存的偶数号元素复制到空出来的数组单元中, 形成四个连续的奇数号元素“块”。
- ④ 如此继续, 直到前一步的“块”前没有元素为止。

- 2) 算法设计如下:

```
void Bubble_Swap(ElemType A[], int n){
    int i=n, v=1;           //i 为工作指针, 初始假设 n 为奇数, v 为“块”的大小
    ElemType temp;          //辅助变量
    if(n%2==0) i=n-1;       //若 n 为偶数, 则令 i 为 n-1
    while(i>1){              //假设数组从 1 开始存放。当 i=1 时, 气泡浮出水面
        temp=A[i-1];        //将“块”前的偶数号元素暂存
        for(int j=0; j<v; j++) //将大小为 v 的“块”整体前移
            A[i-1+j]=A[i+j]  //从前往后依次前移
        A[i+v-1]=temp;       //暂存的奇数号元素复制到平移后空出的位置
        i=i-2; v++;          //指针向前, 块大小增 1
    } //while
}
```

- 3) 共进行了 $n/2$ 次交换, 每次交换的元素个数为 $1 \sim n/2$, 时间复杂度为 $O(n^2)$ 。虽然时间复杂度为 $O(n^2)$, 但因 n^2 前的系数很小, 实际效率是很高的。算法的空间复杂度为 $O(1)$ 。

43. 【解析】

- 1) Cache 块数为 $512\text{KB}/32\text{B} = 16\text{K}$, 组数为 $16\text{K}/4 = 4\text{K} = 2^{12}$, 组号为 12 位, 块大小为 32B, 即块内部分为 5 位。

标记位为 $32 - 12 - 5 = 15$ 位, 4 路组相联, 所以 LRU 算法的替换位是 2 位。

标记位 15 + 有效位 1 + 算法替换位 2 + 修改位 1 位 (因为是回写法) = 19 位, Cache 控制部分每行至少 19 位。

主存地址 0001 0010 0011 0100 0101 0110 0111 1000, 块内为 1 1000, 组号为 0010 1011 0011, 即 2B3H。

- 2) 因为采取低位交叉工作方式, 且存储器位数 $\times$ 体数 = 总线位数, 应采用同时启动方式。

总线周期 = 存储体周期 = 20ns, 则总线频率 = $1/(20\text{ns}) = 50\text{MHz}$ 。

需要 1 个周期传输地址, 8 个周期传输数据, 共需 $9 \times 20\text{ns} = 180\text{ns}$ 。

44. 【解析】

- 1) 所有指令的每个阶段都要访问主存, 即使没有磁盘访问主存, CPU 也会占满主存周期, 因此主存空闲时间的占比为 0。

- 2) 磁盘的平均数据传输率为 $16 \times 512\text{B}/(8.192/10^3)\text{s} = 1\text{MB/s}$ 。当总线宽度为 8 位时, DMA 控



制器每隔 $1s/(1 \times 10^6) = 1\mu s$ 申请一次数据传送, 在 $1\mu s$ 期间 CPU 共执行 $1/(500/1000) = 2$ 条指令。因此, 每 2 条指令的执行被插入 1 个总线周期用于一次数据传送, 即平均每条指令延长了 $250ns/2 = 125ns$ 。因此, 计算机执行指令的速度降低了 $125/500 = 25\%$ 。

- 3) 总线宽度为 16 位时, DMA 控制器每隔 $2s/(1 \times 10^6) = 2\mu s$ 申请一次数据传送, 在 $2\mu s$ 期间 CPU 共执行 $2/(500/1000) = 4$ 条指令, 因此, 每 4 条指令的执行被插入 1 个总线周期用于一次数据传送, 即平均每条指令延长了 $250ns/4 = 62.5ns$ 。因此, 计算机执行指令的速度降低了 $62.5/500 = 12.5\%$ 。

45. 【解析】

本题是生产者-消费者问题的一个变体。由于每个缓冲区都只写一次, 但要读 n_2 次, 所以可将每个缓冲区视为由 n_2 个格组成, 只有当某个缓冲区的 n_2 个格都空闲时才允许写入, 而且写一次相当于将该缓冲区的 n_2 个格全部写一遍, B_i 进程从缓冲区中取消息时, 只取相应缓冲区的第 i 格。

因此, 可设置下列信号量: mutex, 初始值为 1, 用来实现对缓冲区的互斥访问; empty[i] ($i=1, 2, \dots, n_2$), 初始值均为 m , 对应缓冲池第 i 格中的所有空闲缓冲区; full[i] ($i=1, 2, \dots, n_2$), 初始值均为 0, 对应缓冲池中第 i 格中装有消息的缓冲区。A_i 和 B_j 算法的描述如下:

```
semaphore mutex=1;           //对缓冲区的互斥访问
for(int i=1; i<=n2; i++){
    empty[i]=m;               //缓冲区的第 i 格中的所有空闲缓冲区
    full[i]=0;                //缓冲区的第 i 格中装有消息的缓冲区
}
Ai(){                          //i=1, ..., n1
    int k;
    while(1){
        for(k=1; k<=n2; k++){
            wait(empty[k]);
            wait(mutex);
            将 Ai 的消息放入缓冲区;
            signal(mutex);
            for(k=1; k<=n2; k++){
                signal(full[k]);
            }
        }
    }
}
Bj(){                          //j=1, ..., n2
    while(1){
        wait(full[j]);
        wait(mutex);
        从相应缓冲区的第 j 格当中取出消息;
        signal(mutex);
        signal(empty[j]);
        将消息写入数据区;
    }
}
```

46. 【解析】

地址转换过程一般是首先将逻辑页号取出, 然后查找页表, 得到页框号, 将页框号与页内偏移量相加, 即可得到物理地址。若取不到页框号, 则该页不在内存中, 于是产生缺页中断,



开始请求调页。若内存有足够的物理页面，则可再分配一个新页面；若没有页面，则必须在现有页面中找到一个页面，将新页面与之置换，这个页面可以是系统中的任意一个页面，也可以是本进程中的一个页面，若是系统中的一个页面，则这种置换方式称为全局置换，若是本进程中的页面，则称为局部置换。置换时，为尽可能地减少缺页中断次数，可以应用多种算法，本题使用的是改进的 CLOCK 算法，这种算法必须使用页表中的引用位和修改位，由这两位组成 4 个级别：首先淘汰未被引用和未被修改的页面，其次淘汰未被引用但已被修改的页面，接着淘汰已被引用但未被修改的页面，最后淘汰既被引用又被修改的页面，当页面的引用位和修改位相同时，随机淘汰一个页面。

1) 根据题意，每页为 1024B，地址又是按字节编址的，计算逻辑地址的页号和页内偏移量，合成的物理地址如下表所示。

逻辑地址	逻辑页号	页内偏移量	页框号	物理地址
0793	0	793	4	4889
1197	1	173	3	3245
2099	2	51	—	缺页中断
3320	3	248	1	1272
4188	4	92	—	缺页中断
5332	5	212	5	5332

以逻辑地址 0793 为例，逻辑页号为 $0793/1024 = 0$ ，在页表中存在，页内偏移量为 $0793\%1024 = 793$ ，对应的页框号为 4，所以物理地址为 $4 \times 1024 + 793 = 4889$ 。

2) 第 2 页不在内存中，产生缺页中断，根据改进的 CLOCK 算法，第 3 页为未被引用和未被修改的页面，因此淘汰。新页面进入，页表修改如下：

逻辑页号	存在位	引用位	修改位	页框号	
0	1	1	0	4	
1	1	1	1	3	
2	0→1	0→1	0	—→1	调入
3	1→0	0	0	1→—	淘汰
4	0	0	0	—	
5	1	0	1	5	

因为第 2 页调入是为了使用，所以第 2 页的引用位必须改为 1。

地址转换变为如下表所示。

逻辑地址	逻辑页号	页内偏移量	页框号	物理地址
0793	0	793	4	4889
1197	1	173	3	3245
2099	2	51	1	1075
3320	3	248	—	缺页中断
4188	4	92	—	缺页中断
5332	5	212	5	5332



47. 【解析】

1) 部门 1: $2^6 < 112 < 2^7$, 需要 7 位表示主机号, 又由于部门 1~3 按地址值从小到大分配, 分配的 IP 地址为 160.10.10.0/25。

部门 2: $2^5 < 60 < 2^6$, 需要 6 位表示主机号, 分配的 IP 地址为 160.10.10.128/26。

部门 3: $2^4 < 30 < 2^5$, 需要 5 位表示主机号, 分配的 IP 地址为 160.10.10.192/27。

默认网关可设置为 160.10.10.1/25。

2) 路由器 R1 接口 WAN1 的 IP 地址为 192.10.10.0001 0010, 加下画线的部分为主机号, 因为不能为全 1 或全 0, 因此路由器 R2 接口 WAN2 的 IP 地址只能是 198.10.10.17/30。

3) 要确保企业网络正常访问 Internet, R2 的路由表要有到 R1 的表项, 将部门 1~3 的网络聚合后得到目的网络为 160.10.10.0, 也要有到 Internet 的表项, 目的网络为 0.0.0.0。

目的网络	子网掩码 (点分十进制表示)
160.10.10.0	255.255.255.0
0.0.0.0	0.0.0.0

4) 从路由器 R2 到路由器 R1 为 PPP 连接, 不需要 ARP 报文, 从路由器 R1 到 Web 服务器需要 1 次 ARP 解析。



关注公众号【考研小舟】
免费考研资料&无水印PDF



黑白&彩印都是5分/页, 满4.9全国包邮
下单备注: 考研小舟, 送草稿纸



全国硕士研究生入学统一考试

计算机科学与技术学科联考

计算机专业基础综合考试模拟试卷（六）参考答案

一、单项选择题（第 01~40 题）

01.	C	02.	A	03.	A	04.	B	05.	D	06.	A	07.	A	08.	D
09.	C	10.	C	11.	B	12.	B	13.	B	14.	B	15.	C	16.	C
17.	D	18.	C	19.	D	20.	C	21.	A	22.	A	23.	B	24.	B
25.	B	26.	C	27.	C	28.	D	29.	A	30.	D	31.	A	32.	C
33.	B	34.	B	35.	A	36.	A	37.	B	38.	D	39.	A	40.	A

01. C. 【解析】本题考查静态链表。

根据静态链表结点的类型定义可知，link 字段指出了该结点的下一个结点的下标，因此逻辑上第 $k+1$ 个结点的数据是 $S[S[i].link].data$ 。

02. A. 【解析】本题考查出入栈序列和栈深的关系。

由于栈的容量只有 3，因此第一个出栈元素不可能是 5 或 4，先排除选项 C 和选项 D。接下来分析选项 B，1 入栈后出栈，然后 2, 3, 4, 5 依次入栈，5 出栈，才能得到序列 B，但实现这种出栈序列时，栈的容量至少为 4，仅有选项 A 满足。

03. A. 【解析】本题考查循环队列的性质。

分 $rear > front$ 和 $rear < front$ 两种情况讨论：

① 当 $rear > front$ 时，队列中元素个数为 $rear - front = (rear - front + m) \% m$ 。

② 当 $rear < front$ 时，队列中元素个数为 $m - (front - rear) = (rear - front + m) \% m$ 。

综合①和②可知，选项 A 正确。

【另解】特殊值代入法：对于循环队列，选项 C 和 D 无取 MOD 操作，显然错误，直接排除。

设 $front=0$ 、 $rear=1$ ，则队列中存在一个元素 $A[0]$ ，代入 A、B 两项，显然仅有选项 A 符合。

▲注意：① 不同教材对队尾指针的定义可能不同，有的定义其指向队尾元素，有的定义其指向队尾元素的下一个元素，不同的定义会导致不同的答案（取决于是先移动指针还是先存取元素），考题中通常会特别说明。② 循环队列的队尾指针、队头指针、队列中元素个数，知道其中任何两者均可求出第三者。

04. B. 【解析】本题考查 KMP 算法。

在 KMP 算法中，next 数组指明匹配失败时模式串的匹配指针的指向，next 数组的值只与模式串有关，因此选择选项 B。



05. D. 【解析】本题考查树的性质。

设叶结点数为 n_0 ，分支结点数为 n_k ，则 $n = n_0 + n_k$ 。根据树的度的性质，除根结点外，其余结点都有一个分支进入， $n = kn_k + 1$ ，得 $n_k = (n-1)/k$ ，即 $n_0 = n - n_k = n - (n-1)/k = (nk - n + 1)/k$ 。

06. A. 【解析】本题考查哈夫曼树的性质。

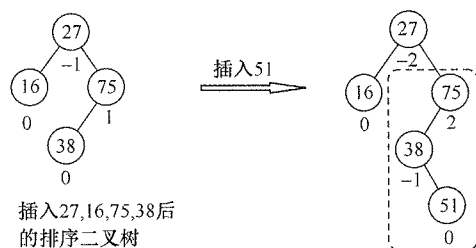
哈夫曼树不一定是完全二叉树，也不一定是排序二叉树，只是所有结点度为 0 和 2 的二叉树，选项 I、II 错误；哈夫曼树最小时只有一个根结点，此时叶结点数为 1，非叶结点数为 0，且每增加一个非叶结点，叶结点数加 1，所以叶结点数等于非叶结点数加 1，选项 III 正确；若哈夫曼树的上层结点的值小于下层结点的值，则这两个结点交换得到的树的最小路径和一定比原树的小，所以不符合哈夫曼树定义，选项 IV 正确。因此，选项 III、IV 正确。

07. A. 【解析】本题考查各种图算法的特点。

对于无向图的每个连通分量，广度优先遍历算法都要执行一次才能遍历完所有结点，因此广度优先遍历算法的调用次数就等于无向图的连通分量数量。

08. D. 【解析】本题考查平衡二叉树的构造。

由题中所给的结点序列构造平衡二叉树的过程如下图所示，插入 51 后，首次出现不平衡子树，虚线框内即为最小不平衡子树，根为 75。



09. C. 【解析】本题考查 B 树的插入操作。

首先考虑从根结点找到插入位置需要进行 h 次读磁盘操作，在最坏情况下，在待插入结点中插入一个关键码后，导致结点分裂，在该层分裂成 2 个结点，因此需要 2 次写磁盘操作；分裂操作逐层向上传导，导致每一层都有结点分裂，因为结点分裂而导致的写磁盘操作共有 $2h$ 次，再加上最后一次结点分裂形成新根结点需要额外的一次写磁盘操作，写磁盘的总次数为 $2h + 1$ 。因此，整个过程中读写磁盘的总次数为 $3h + 1$ 次。

10. C. 【解析】本题考查排序二叉树、大顶堆、小顶堆、平衡二叉树的性质。

排序二叉树中的任意一个结点 x 大于其左孩子，小于其右孩子。从排序二叉树的任意一个结点出发到根结点，只要路径中存在左子树关系，则必不满足题中降序的条件。同理，平衡二叉树也不满足。小顶堆中的任意一个结点 x 均小于左右孩子，因此从任意一个结点到根的路径上的结点序列必然是降序的。大顶堆的情况刚好相反。

▲注意：堆存储在一个连续的数组单元中，它是一棵完全二叉树。

11. B. 【解析】本题考查排序算法的性质。

因为组与组之间已有序，所以对 n/k 个组分别排序即可。基于比较的排序算法，每组的时间下



界为 $O(k\log_2 k)$, 因此全部时间下界为 $O(n\log_2 k)$ 。

12. B. 【解析】本题考查不同进制数之间的转换与算术移位运算。

对于本类题型, 应先将 -1088 转换为 16 位的补码表示, 执行算术右移后, 再转换为十六进制数。 $R1$ 的内容首先为 $[-1088]_{\text{补}} = 1111\ 1011\ 1100\ 0000\text{B} = \text{FBC0H}$ 。算术右移 4 位的结果为 $1111\ 1111\ 1011\ 1100\text{B} = \text{FFBCH}$, 此时 $R1$ 中的内容为 FFBCH 。

13. B. 【解析】本题考查 IEEE 754 单精度浮点数的对阶。

当 $[\Delta E]_{\text{补}}$ 发生溢出时, 有两种可能的情况: ① x 的阶比 y 的阶大, 此时 $\Delta E \geq 128$; 对阶操作时, y 的尾数至少要右移 128 位, 从而使得 24 位有效位全部丢失, 尾数变为全 0, 再与 x 的尾数相加的结果就是 x 的尾数, 因此“ y 被 x 吃掉”, 结果是 x , 此时, $\text{EXS} = 1$; ② x 的阶比 y 的阶小, 此时 $\Delta E \leq -129$, 说明 y 的阶比 x 的阶至少大 129; 对阶操作时, x 的尾数至少要右移 129 位, 因此“ x 被 y 吃掉”, 结果是 y , 此时, $\text{EXS} = 0$ 。

14. B. 【解析】本题考查磁盘访问时间计算。

磁盘访问时间 = 寻道时间 t_1 + 找扇区时间 t_2 + 传输时间 t_3 。

$t_1 = 5\text{ms}$; $t_2 = \text{转半圈的时间} = 0.5\text{r} \div 6000\text{rpm} = 5\text{ms}$; $t_3 = 512\text{B} \div 4096000\text{B/s} = 0.125\text{ms}$ 。

$T = 5\text{ms} + 5\text{ms} + 0.125\text{ms} = 10.125\text{ms}$ 。

15. C. 【解析】本题考查 Cache 的比较器和匹配过程。

Cache 中比较器的个数取决于 Cache 的关联度, 即一个主存块有可能映射到几个 Cache 行, 在 8 路组相联 Cache 中, 关联度为 8, 因此 Cache 中比较器的个数为 8, 当一个主存块计算出组号后, 需要在本组内同时进行 8 次比较判断 tag 位是否匹配。在直接映射 Cache 中, 关联度为 1, Cache 中比较器的个数为 1, 只需与对应 Cache 行的 tag 位进行 1 次比较。

16. C. 【解析】本题考查 Cache 和 TLB。

TLB 的作用是增大虚拟地址到物理地址的转换效率, TLB 缺失后仍可通过查询页表获得虚拟地址对应的物理地址, Cache 缺失后也可在内存中找到数据, 因此不会导致程序执行出错。

17. D. 【解析】本题考查基址寻址和小端方式存储。

基址寄存器的内容是无符号数, 而形式地址是用补码表示的有符号数, 可以首先把形式地址转换为真值, 然后与基址寄存器的内容相加。下面介绍一种更快的计算方法。我们知道, 若一个数的真值为 x , 其补码实质上就是 $2^n + x$, 假设 FFFF FF00H 的真值是 x , 则 $2^n + x = \text{FFFF FF00H}$, $x = \text{FFFF FF00H} - 2^n$, 故 $\text{C000 0000H} + x = \text{C000 0000H} + \text{FFFF FF00H} - 2^n$, 计算结果是一个地址, 必然是正整数, 因此 $\text{C000 0000H} + \text{FFFF FF00H} - 2^n$ 的结果就相当于 $\text{C000 0000H} + \text{FFFF FF00H}$ 的结果用 32 位表示, 因此操作数的地址为 BFFF FF00H 。由于计算机按字节编址, 采用小端方式存储, 且操作数占 2B, 因此操作数的 MSB 存放的地址是 BFFF FF01H 。

▲注意: 从本题的分析过程不难得出, 以后遇到由基址寻址计算操作数地址时, 都可将基址寄存器的内容与形式地址经符号扩展后的结果直接相加。

18. C. 【解析】本题考查定长指令字和变长指令字的区别。



变长指令字的每条指令从一个字节到多个字节不等，因此下一条指令的地址需要通过一个专门的 PC 增量器来进行计算；定长指令字的每条指令的长度固定，可以简单地实现 PC 的自增功能，不需要专门的 PC 增量器，选项 C 错误。变长指令字每次可以按照最长的指令长度来读取，然后根据指令中特定定位的规定对各个字段进行划分，选项 A、B、D 均正确。

19. D. 【解析】本题考查指令流水线的设计原则。

指令执行过程中的各个子功能都需要包含在某个流水段中，但不同的指令所需的子功能是不同的，为保证指令流水线的正常运行，一般的指令都向最长的指令对齐，因此有些指令的流水段就设置了空操作，选项 D 错误。

20. C. 【解析】本题考查指令流水线中控制信号的传输。

要将特定指令对应的控制信号送到对应的流水段中，只需将译码阶段得到的控制信号以流水线的方式传输，每来一个时钟周期，控制信号就往后一个流水段传输一次，使得控制信号和所控制的操作同步，因此①错误，④正确。在指令流水线 CPU 的数据通路中，取指和指令译码两个阶段的动作是一致的，不需要额外的控制信号，②正确。指令译码后，具体的执行流程与具体的指令对应，③正确。综上所述，选项 C 正确。

21. A. 【解析】本题考查超标量流水线的原理。

流水线中采用更多的流水段数属于超流水线技术，选项 A 错误。超标量技术指的是动态多发射技术，CPU 中必须配置多个不同的功能部件，利用部件的并行性来提高指令吞吐率。

22. A. 【解析】本题考查中断的性能分析。

数据传输率为 50KB/s，传输单位为 16bit = 2B，因此每秒引起的中断次数为 $50\text{KB} \div 2\text{B} = 2.5 \times 10^3$ ，每秒 CPU 用于数据传输的开销为 $2.5 \times 10^3 \times 100 = 2.5 \times 10^5$ 个时钟周期，机器主频为 50MHz，所以 CPU 用于软盘数据传输的时间占整个 CPU 时间的比例是 $2.5 \times 10^5 / 50 \times 10^6 = 5\%$ 。

23. B. 【解析】本题考查用户态与内核态。

设置定时器的初值属于时钟管理的内容，需要在内核态下运行；Trap 指令是用户态到内核态的入口，可以在用户态下运行；内存单元复位属于存储器管理的系统调用服务，用户态下随便控制内存单元的复位是很危险的行为。关闭中断允许位属于中断机制，它们都只能运行在内核态下。

24. B. 【解析】本题考查进程调度算法。

要使得平均等待时间最小，需要采取短作业优先调度算法，这几个进程的执行顺序为 1, 2, 3, 4, 5，等待时间分别为 0, 2, 6, 12, 20，所以平均等待时间为 8，选择选项 B。

25. B. 【解析】本题考查信号量机制的应用。

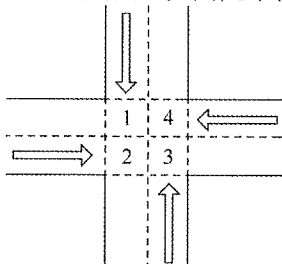
申请资源用 P 操作，执行完后，若 $S < 0$ ，则表示资源申请完毕，需要等待， $|S|$ 表示等待该资源的进程数；释放资源用 V 操作，V 操作后，S 仍然小于或等于 0。在某个时刻，信号量 S 的值为 0，然后对信号量 S 进行 3 次 V 操作，即 $S = S + 3$ ，此时 $S = 3 > 0$ ，表示没有进程在队列中等待。

▲注意：之前对 S 进行了 28 次 P 操作和 18 次 V 操作，但这并不会影响计算结果。



26. C。【解析】本题考查信号量机制的分析。

如下图所示，将十字路口车道的公共区域分为4块，分别为图上的1, 2, 3, 4，直行车辆需要获得该方向上两个邻近的临界资源，如北方来车需要获得两个临界资源 1, 2，南方来车需要获得两个临界资源 3, 4。右转车辆只需要获得一个临界资源，如北方来车右转时需要获得一个临界资源 1，北方来车左转时需要获得三个临界资源 1, 2, 3。综上所述，4 个临界资源便可以很好地保证汽车不相撞（互斥效果）。当然，只用 4 个信号量还是很容易造成死锁，但这不是本题要考虑的问题，因为题目中问到的是至少用几个信号量。



也可用排除法来做该题：在该路口，南北方向的车辆可以同时直行，因此临界资源个数大于或等于 2，排除选项 A；在该路口，4 个方向的车辆都可左转，因此临界资源个数大于或等于 4，排除选项 B。选项 D 通常不会选，因此最终选择选项 C。

27. C。【解析】本题考查计算机动态分区内存分配算法的计算。

对于本类题的解答，一定要画出草图。按照题中的各种分配算法，分配结果如下表所示。

空闲区	100KB	450KB	250KB	300KB	600KB
首次适应算法		212KB 112KB			417KB
邻近适应算法		212KB 112KB			417KB
最佳适应算法		417KB	212KB	112KB	426KB
最坏适应算法		417KB			212KB 112KB

只有最佳适应算法能够完全完成分配任务。

28. D。【解析】本题考查虚拟内存管理的原理。

按需调页适合具有较好局部性的程序。堆栈只在栈顶操作，栈底的元素很久都用不着，显然对数据的访问具有局部性。线性搜索即顺序搜索，显然也有局部性。向量运算就是数组运算，数组是连续存放的，所以数组运算就是邻近数据的运算，也满足局部性。二分搜索先查找中间的那个元素，若未找到，则查找前半部分的中间元素或后半部分的中间元素，以此类推。显然，每次搜索的元素不都是相邻的，二分搜索是跳跃式的搜索，不满足局部性，不适合“按需调页”的环境。

▲注意：要使得按需调页有效，就要抓住按需调页被提出的前提，即程序运行的局部性原理。

29. A。【解析】本题考查文件索引节点的特征。

文件的逻辑结构是从用户视角出发看到的文件组织形式，即文件是由一系列的逻辑记录组成

的,是用户可以直接处理的数据及结构,文件索引节点和逻辑结构没有关系,选项 A 错误。文件可以通过硬链接的方式指向同一个索引节点来实现文件共享;内存索引节点中有一些磁盘索引节点中没有的信息,如共享计数值 count;文件的存取控制权限信息属于文件元数据,是文件索引节点中的内容。

30. D. 【解析】本题考查影响文件访问速度的因素。

选项 A、B 和 C 均是提高文件访问速度的措施。提高 CPU 的利用率和提高文件的访问速度两者之间并没有必然的联系,选项 D 错误。

31. A 【解析】本题考查各种 I/O 控制方式的原理。

程序直接控制 I/O 方式和中断驱动 I/O 方式都是软件控制数据的传输过程,即设备驱动程序负责具体的数据传输,选项 A 错误。DMA 方式由硬件负责具体的数据传输过程,适用于具有 DMA 控制器的系统,用于高速外设的大批量数据传输,选项 B、C、D 均正确。

32. C. 【解析】本题考查缓冲区的计算。

这是单缓冲区,数据输入缓冲区的时间 $T = 80\mu\text{s}$,缓冲区数据传输到 CPU 的时间 $M = 40\mu\text{s}$,CPU 对这块数据处理的时间 $C = 30\mu\text{s}$,处理每块数据的时间 $= \max(C, T) + M = 120\mu\text{s}$ 。

33. B. 【解析】本题考查香农定理的应用。

题干中已说明是有噪声的信道,因此应联想到香农定理,而对无噪声信道则应联想到奈奎斯特定理。首先计算信噪比 $S/N = 0.62/0.02 = 31$;带宽 $W = 3.9\text{MHz} - 3.5\text{MHz} = 0.4\text{MHz}$,由香农定理可知最高数据传输率 $V = W\log_2(1 + S/N) = 0.4 \times \log_2(1 + 31) = 2\text{Mb/s}$ 。

知识补充:奈奎斯特在采样定理和无噪声的基础上提出了奈氏准则: $C_{\max} = f_{\text{采样}} \times \log_2 N = 2f \log_2 N$,其中 f 表示带宽。香农定理给出有噪声信道的容量: $C_{\max} = W\log_2(1 + S/N)$,其中 W 为信道的带宽。它指出,要提高信息的传输率,就应设法提高传输线路的带宽或者提高所传信号的信噪比。

34. B. 【解析】本题考查停止-等待协议和信道利用率的计算。

停止-等待协议每发出一帧,需要收到该帧的 ACK 后才能发送下一帧,确认帧也为 50B (捎带确认说明帧长和数据帧的一样)。数据帧发送时间 = 确认帧发送时间 = $50\text{B} \div 2\text{kb/s} = 200\text{ms}$ 。一个发送周期 = 数据帧发送时间 + 确认帧发生时间 + 传输时间 = $200\text{ms} + 200\text{ms} + 200\text{ms} = 600\text{ms}$ 。信道利用率 = 发送时间 / (发送时间 + 传输时间) = $200\text{ms} \div 600\text{ms} = 33\%$ 。

35. A. 【解析】本题考查 CSMA 协议的各种监听算法。

采用随机的监听延迟时间可以减少冲突的可能性,但缺点也很明显:虽然多个站点有数据要发送,但因为此时所有站点可能都在等待各自的随机延迟时间,而媒介仍然可能处于空闲状态,所以媒介的利用率较低,选项 I 错误。I-坚持 CSMA 的优点是,只要媒介空闲,站点就立即发送;缺点是,若有两个或两个以上的站点有数据要发送,则冲突不可避免,选项 II 错误。按照 p -坚持 CSMA 的规则,若下一个时槽也空闲,则站点同样按概率 p 发送数据。因此,若处理得当,则 p -坚持型监听算法仍然可以缩短网络的空闲时间,选项 III 错误。

CSMA 有三种类型:



- ① 非坚持 CSMA: 一个站点在发送数据帧前, 先对媒介进行检测。若没有其他站点正在发送数据, 则该站点开始发送数据。若媒介被占用, 则该站点不会持续监听媒介, 而要等待一个随机的延迟时间后再监听。
- ② 1-坚持 CSMA: 当一个站点要发送数据帧时, 它会监听媒介, 判断当前时刻是否有其他站点正在传输数据。若媒介忙, 则该站点等待, 直至媒介空闲。一旦该站点检测到媒介空闲, 就立即发送数据帧。若产生冲突, 则等待一个随机的延迟时间再监听。之所以称为 1-坚持, 是因为当一个站点发现媒介空闲时, 它传输数据帧的概率是 1。
- ③ p -坚持 CSMA: 当一个站点要发送数据帧时, 它先检测媒介。若媒介空闲, 则该站点以概率 p 发送数据, 而以概率 $1-p$ 将发送数据帧的任务延迟到下一个时槽。 p -坚持 CSMA 是非坚持 CSMA 和 1-坚持 CSMA 的折中。

36. A. 【解析】本题考查 ARP 的过程。

ARP 请求分组是广播发送的, 但 ARP 响应分组却是普通的单播, 即从一个源地址发送到另一个目的地址。注意, 没有点播这个概念。

单播、广播、组播的优缺点比较如下。

单播的优点: ① 服务器及时响应客户端的请求。② 服务器针对每个客户端的不同请求发送不同的数据, 容易实现个性化服务。

单播的缺点: 在客户端数量大、每个客户端流量大的流媒介应用中, 服务器不堪重负。

广播的优点: ① 网络设备简单, 维护简单, 布网成本低廉。② 服务器不用向每个客户端单独发送数据, 因此服务器流量负载极低。

广播的缺点: ① 无法针对每个客户端的要求和时间及时提供个性化服务。② 网络允许服务器提供数据的带宽有限, 客户端的最大带宽 = 服务总带宽。③ 广播禁止在 Internet 宽带网上传输。

组播的优点: ① 需要相同数据流的客户端加入相同的组, 以共享一条数据流, 节省了服务器的负载。具备广播所具备的优点。② 组播根据接收者的需要对数据流进行复制转发, 因此服务器的服务总带宽不受客户端接入端带宽的限制。③ 组播和单播一样允许在 Internet 宽带网上传输。

组播的缺点: 与单播相比没有纠错机制, 发生丢包、错包后, 难以弥补。

37. B 【解析】本题考查 IP 数据报的首部检验和。

计算首部检验和不采用 CRC 检验码的主要原因是, 可以缩短路由器进行检验的时间, 选项 B 正确。虽然 IP 数据报首部中某些字段 (如标志、生存时间、首部检验和等) 的数值在路由器转发过程中可能发生变化, 但 IP 数据报的首部长度在转发过程中是保持不变的。

38. D. 【解析】本题考查伪首部的特性。

所谓伪首部, 是指它并不是 UDP 数据报或 TCP 报文段的真正首部, 只是在计算检验和时, 临时添加到 UDP 数据报或 TCP 报文段的前面, 得到一个临时的 UDP 数据报或 TCP 报文段。检验和就是按照这个临时的 UDP 数据报或 TCP 报文段计算的。伪首部既不向下传输, 又不向上递交, 仅用于计算传输层的检验和。

39. A. 【解析】本题考查 TCP 连接释放的过程。

根据 TCP 连接释放“4 次挥手”的示意图, 客户 A 发送完第 4 次挥手后, 不立刻关闭连接,

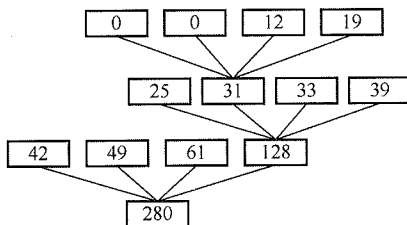


为什么 TIME-WAIT 状态必须等待 2MSL 呢？理由有二：① 为了保证发送的最后一个 ACK 报文段能够到达。这个 ACK 报文段有可能丢失，因此使处在 LAST-ACK 状态的服务器 B 收不到对已发送 FIN+ACK 报文段的确认。B 超时重传这个 FIN+ACK 报文段，而 A 就能在 2MSL 内收到这个重传的 FIN+ACK 报文段。接着 A 重传一次确认，重新启动 2MSL 计时器。最后，A 和 B 都正常进入 CLOSED 状态。② 为了防止旧 TCP 报文段对后续连接产生错误干扰。A 在发送完最后一个 ACK 报文段后，再经过 2MSL，就可使本连接持续的时间内产生的所有报文段都从网络中消失。这样，就可使下一个新连接中不出现这种旧的连接请求报文段。

本地主机的 DNS 地址就是本地域名服务器地址。DNS 客户首先向本地域名服务器进行查询，若本地域名服务器有域名和 IP 地址的对应关系，则通过 UDP 数据报返回给客户；若本地域名服务器没有域名和 IP 地址的对应关系，则让本地域名服务器进行后续查询，不论采用的是递归查询还是迭代查询，之后都先查询根域名服务器。

41. 【解析】

- 1) 形成初始归并段时需要将外存的记录读入内存进行内部排序, 然后每次形成一个长度为 k 的初始归并段, 因此最初会形成 $\lceil n/k \rceil$ 个初始归并段。
- 2) 这样划分初始归并段会使得外部排序过程中有较多的 I/O 次数, 导致外部排序时间较长, 因此可以采用置换-选择排序来增加归并段的长度, 进而减少归并段的个数。
- 3) $(8-1)\%(4-1)=1$, 需要补充 2 个虚段, 最终的最佳归并树如下图所示。



每次归并时都需要进行一次读入和写出, 因此 I/O 总次数为 WPL 值的 2 倍。

因此, 需要 $WPL \times 2 = 878$ 次 I/O 操作来完成归并。

1) 无权有向图的邻接表的定义如下:

```
//图中顶点数目的最大值
//边表结点
//该弧所指向的顶点的位置
//指向下一条弧的指针
```

```
//顶点表结点
//顶点信息
//指向第一条依附该顶点的弧的指针
```



```

}VNode,AdjList[MaxVertexNum];
typedef struct{
    AdjList vertices;           //邻接表
    int vexnum,arcnum;         //图的顶点数和弧数
}ALGraph;                     //ALGraph 是以邻接表存储的图类型

```

- 2) 求顶点 v 的入度时需要检测每个顶点, 查找是否有指向顶点 v 的边, 因此设置一个计数变量 $count$, 通过两个 for 循环遍历整个邻接表的所有边, 每找到在一条指向顶点 v 的边, $count$ 就加 1。

```

int inDegree(ALGraph &G,int v){ //统计有向图 G 中顶点 v 的入度, 通过函数返回
    int i,count=0,ArcNode *p;
    for(i=0;i<G.vexnum;i++){ //逐个顶点检测
        for(p=G.vertices[i].firstarc;p!=NULL;p=p->nextarc)
            //检测每个顶点所指向的边
            if(p->adjvex==v) count++; //终顶点为 v, 累加
    }
    return count;
}

```

43. 【解析】

- 1) ① 页大小为 $256B = 2^8B$, 即页内地址为 8 位。
页号位 $16-8=8$ 位, 即高 8 位为虚拟页号, 低 8 位为页内地址。
- ② TLB 有 $16/4=4$ 组 $=2^2$ 组, 即索引号为 2 位。
 $8-2=6$, 标记为 6 位。高 6 位为标记号, 中间 2 位为索引号, 低 8 位为页号。
- 2) $13-8=5$, 物理页号为 5 位。
高 5 位为物理页号, 低 8 位为页内偏移量。
- 3) Cache 采用直接映射, 块内 $4B = 2^2B$, 块内地址为 2 位。
有 $16=2^4$ 块, 行索引为 4 位; $13-4-2=7$, 标记为 7 位。
高 7 位为标记, 中间 4 位为行索引, 低 2 位为块内地址。
- 4) 先根据逻辑地址 0000 0110 0111 1010B 查询 TLB, 索引号为 10B=2, 标记为 000001B=01H, 有效位为 1, 命中, 页框号为 19H, 物理地址为 1 1001 0111 1010B=197AH。因为 short 型变量的首地址应是 2 的倍数, 所以该变量保存在 197AH~197BH 中。然后, 根据物理地址查找 Cache, 标记为 1100101B=65H, 行索引为 1110B=EH, 有效位为 1, 命中, 块内 10 位是 4AH, 11 位是 2DH, 因为 short 是小端, 所以值为 2D4AH。

44. 【解析】

下面给出的是指令执行阶段的控制信号。执行阶段开始时, 取指阶段已结束, 此时指令的第二个字 (Imm16) 已从存储器中取出并被存放在 MDR 中。

- 1) 当指令功能为 $R[R1] \leftarrow R[R1] + \text{Imm16}$ 时, 在执行阶段不需要访存操作, 因此只需以下 3 个时钟周期:

```

MDRout, Yin
Rlout, add, Zin
Zout, Rlin

```

- 2) 当指令功能为 $R[R1] \leftarrow R[R1] + M[\text{Imm16}]$ 时, 执行阶段需要 1 次访存操作, 因此至少需要



以下 5 个时钟周期:

```
MDRout, MARin
Read1, (Rlout, Yin)
Read2, Rlout, Yin
MDRout, add, Zin
Zout, Rlin
```

其中 Rlout 和 Yin 这两个控制信号可与 Read1 控制信息同时送出, 并在 Read2 操作阶段保持不变, 也可延迟到与 Read2 同时送出。

- 3) 当指令功能为 $R[R1] \leftarrow R[R1] + M[M[Imm16]]$ 时, 执行阶段需要 2 次访存操作, 因此至少需要以下 8 个时钟周期:

```
MDRout, MARin
Read1
Read2
MDRout, MARin
Read1, (Rlout, Yin)
Read2, Rlout, Yin
MDRout, add, Zin
Zout, Rlin
```

对 Rlout 和 Yin 这两个控制信号的处理, 与 2) 中的相同。

45. 【解析】

- 1) 缺页中断是一种特殊的中断, 它与一般中断的区别是: ① 在指令执行期间产生和处理中断信号。CPU 通常在一条指令执行完后检查是否有中断请求, 而缺页中断是在指令执行期间发现所要访问的指令或数据不在内存时产生和处理的; ② 一条指令在执行期间可能产生多次缺页中断。例如, 一条读取数据的多字节指令本身跨越两个页面, 若指令后一部分所在的页面和数据所在的页面均不在内存中, 则该指令的执行至少产生两次缺页中断。
- 2) 每个页面大小为 100B, 页面的访问顺序如下表所示。

10	11	104	170	73	309	185	245	246	434	458	364
0	0	1	1	0	3	1	2	2	4	4	3

采用 FIFO 算法的页面置换情况如下表所示, 共产生缺页中断 6 次。

走 向	0	0	1	1	0	3	1	2	2	4	4	3
块号 1	0	0	1	1	1	3	3	2	2	4	4	3
块号 2			0	0	0	1	1	3	3	2	2	4
淘 汰						0		1		3		2
缺 页	√		√			√		√		√		√

采用 LRU 算法的页面置换情况如下表所示, 共产生缺页中断 7 次。

走 向	0	0	1	1	0	3	1	2	2	4	4	3
块号 1	0	0	1	1	0	3	1	2	2	4	4	3
块号 2			0	0	1	0	3	1	1	2	2	4
淘 汰						1	0	3		1		2
缺 页	√		√			√	√	√		√		√

- 3) 设可接收的最大缺页中断率为 ρ 。若要访问的页面在内存中, 则一次访问的时间是 10ms



(访问内存页表) + 10ms (访问内存) = 20ms。若要访问的页面不在内存中, 则所花的时间为 10ms (访问内存页表) + 25ms (中断处理) + 10ms (访问内存页表) + 10ms (访问内存) = 55ms。平均有效访问时间为 $20\text{ms} \times (1 - \rho) + 55\text{ms} \times \rho$, 要求平均访问时间不超过 22ms, 即 $20\text{ms} \times (1 - \rho) + 55\text{ms} \times \rho \leq 22\text{ms}$, 得到可接收的最大缺页中断率 ρ 为 5.7%。

46. 【解析】

1) SSTF 算法: $95 \rightarrow 80 \rightarrow 58 \rightarrow 40 \rightarrow 38 \rightarrow 35 \rightarrow 28 \rightarrow 143 \rightarrow 160 \rightarrow 204$ 。

磁道移动数: $(95 - 28) + (204 - 28) = 243$ 。

2) SCAN 算法: $95 \rightarrow 143 \rightarrow 160 \rightarrow 204 \rightarrow 80 \rightarrow 58 \rightarrow 40 \rightarrow 38 \rightarrow 35 \rightarrow 28$ 。

磁道移动数: $(204 - 95) + (204 - 28) = 285$ 。

3) C-SCAN 算法: $95 \rightarrow 143 \rightarrow 160 \rightarrow 204 \rightarrow 28 \rightarrow 35 \rightarrow 38 \rightarrow 40 \rightarrow 58 \rightarrow 80$ 。

磁道移动数: $(204 - 95) + (204 - 28) + (80 - 28) = 337$ 。

4) 先来先服务调度算法按照请求的次序进行调度, 比较公平, 不会产生磁头臂黏着现象。对于其他调度算法 (如 SSTF、SCAN、C-SCAN), 若在处理当前磁道请求时, 当前磁道有源源不断的请求到来, 则不论其他磁道有没有产生磁道请求, 都会先处理当前磁道的请求, 因此会产生磁头臂黏着现象。

5) 在磁盘上进行一次读/写操作所花的时间由寻道时间、旋转延迟时间和传输时间决定。其中, 寻道时间是将磁头移动到指定磁道所需的时间, 磁盘调度算法影响的就是这部分时间。磁盘数据的读取通常采用 DMA 方式。

47. 【解析】

1) 解析 URL 的 IP 地址所需的总时间是 $nRTT$, 建立 TCP 连接和获得万维网文档所需的时间为 $2RTTw$, 因此需要的总时间为 $2RTTw + nRTT$ 。

2) 三种条件解析 URL 的 IP 地址所需的时间都是 $nRTT$, 因此只需分析建立 TCP 连接和传输网页元素对象的时间。① 建立 TCP 连接并读取 HTML 文件的时间为 $2RTTw$, 采用非流水线的非持续 HTTP, 因此每传输一个小对象所需的时间都为 $2RTTw$, 传输 3 个小对象所需的时间为 $6RTTw$, 总耗时为 $nRTT + 8RTTw$ 。② 建立 TCP 连接并读取 HTML 文件的时间为 $2RTTw$, 采用非流水线的持续 HTTP, 因此传输 3 个小对象还需要 $3RTTw$, 总耗时为 $nRTT + 5RTTw$ 。③ 建立 TCP 连接并读取 HTML 文件的时间为 $2RTTw$, 采用流水线的持续 HTTP, 因此可在接下来的 $1RTT$ 时间内请求并获得 3 个小对象, 总耗时为 $nRTT + 3RTTw$ 。



全国硕士研究生入学统一考试

计算机科学与技术学科联考

计算机专业基础综合考试模拟试卷（七）参考答案

一、单项选择题（第 01~40 题）

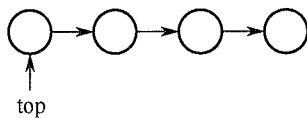
01.	B	02.	A	03.	C	04.	C	05.	D	06.	B	07.	B	08.	A
09.	C	10.	C	11.	C	12.	D	13.	B	14.	A	15.	C	16.	D
17.	C	18.	C	19.	A	20.	C	21.	D	22.	D	23.	D	24.	B
25.	B	26.	D	27.	C	28.	A	29.	D	30.	C	31.	B	32.	B
33.	A	34.	D	35.	C	36.	A	37.	B	38.	C	39.	D	40.	B

01. B. 【解析】本题考查链表的操作。

要保证插入的先后顺序与对应结点在链中的顺序相反，必须使用头插法（尾插法插入的先后顺序与链中的顺序相同），所以排除选项 C、D。在选项 A 中，执行 $L \rightarrow next = p$ 后会导致断链。

02. A. 【解析】本题考查单链表的操作。

不带头结点的单链表 A 作为链式栈的结构如下图所示。



当栈顶结点出栈时，先用 x 保存栈顶元素的值，然后移动栈顶指针。因此，正确的操作应该是 $x = top \rightarrow data; top = top \rightarrow link$ 。

03. C. 【解析】本题考查出栈序列的合法性。

这类题通常采用手动模拟法。A 项：6 入，5 入，5 出，4 入，4 出，3 入，3 出，6 出，2 入，1 入，1 出，2 出；B 项：6 入，5 入，4 入，4 出，5 出，3 入，3 出，2 入，1 入，1 出，2 出，6 出；D 项：6 入，5 入，4 入，3 入，2 入，2 出，3 出，4 出，1 入，1 出，5 出，6 出；C 项：无对应的合法出栈顺序。

技巧：对于已入栈且尚未出栈的序列，要保证先入栈的一定不能在后入栈的前面出栈。选项 C 的 6 在 5 前面入栈，5 没有出栈，6 却出栈了，不合法。其他选项都符合规律。

04. C. 【解析】本题考查串的 next 数组。

1) 设 $next[1] = 0$, $next[2] = 1$ 。



编 号	1	2	3	4	5
S	a	c	a	b	a
next	0	1			

2) 当 $j=3$ 时, $k=\text{next}[j-1]=\text{next}[2]=1$, 观察 $S[2]$ 与 $S[k]$ ($S[1]$) 是否相等, $S[2]=c$, $S[1]=a$, $S[2] \neq S[1]$, 此时 $k=\text{next}[k]=0$, 所以 $\text{next}[j]=1$ 。

↓ $j-1=2$

a c a b a
a c a b a

↑ $k=1$

3) 当 $j=4$ 时, $k=\text{next}[j-1]=\text{next}[3]=1$, 观察 $S[3]$ 与 $S[k]$ ($S[1]$) 是否相等, $S[3]=a$, $S[1]=a$, $S[3]=S[1]$, 所以 $\text{next}[j]=k+1=2$ 。

↓ $j-1=3$

a c a b a
a c a b a

↑ $k=1$

4) 当 $j=5$ 时, $k=\text{next}[j-1]=\text{next}[4]=2$, 观察 $S[4]$ 与 $S[k]$ ($S[2]$) 是否相等, $S[4]=b$, $S[2]=c$, $S[4] \neq S[2]$, 所以 $k=\text{next}[k]=1$ 。

↓ $j-1=4$

a c a b a
a c a b a

↑ $k=2$

5) 此时 $S[k]=S[1]=a$, $S[4] \neq S[1]$, 所以 $k=\text{next}[k]=\text{next}[1]=0$, 所以 $\text{next}[j]=1$ 。

↓ $j-1=4$

a c a b a
a c a b a

↑ $k=1$

可知, next 数组为 01121。

05. D. 【解析】本题考查二叉树的遍历。

解法 1: 对于选项 I, 显然任何遍历都相同。对于选项 II, 根结点无右孩子, 此时前序遍历首先遍历根结点, 中序遍历最后遍历根结点, 所以不相同。对于选项 III, 是一棵左单支树, 前序遍历和中序遍历的序列相反。对于选项 IV, 所有结点只有右子树的右单支树, 前序遍历和中序遍历的序列相同。因此, 选择选项 D。

解法 2: 若树中某棵子树存在左子树, 则中序遍历一定先遍历左子树后才遍历这颗子树本身, 而前序遍历则先遍历这颗子树本身, 所以只要树中某个结点存在左子树, 便是不符合要求的, 因此任何一颗子树都没有左子树的树符合要求, 选项 I 和选项 IV 符合要求。

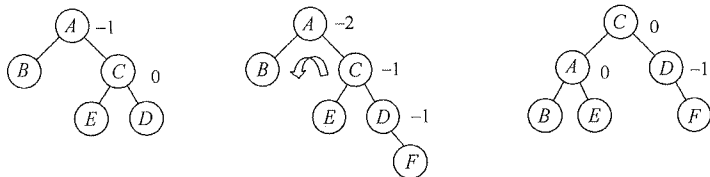
06. B. 【解析】本题考查并查集和图的算法。

Kruskal 算法流程首先将所有的边从小到大排序, 然后使用并查集, 每次选最小的边, 并且该边要保证其两端的两个顶点属于不同的集合, 选择边后, 进行一次并操作, 重复选择边及并操作 $n-1$ 次, 得到最小生成树。

07. B. 【解析】本题考查平衡二叉树的旋转。

由于在结点 A 的右孩子 (R) 的右子树 (R) 上插入新结点 F , A 的平衡因子由 -1 减至 -2 , 因此以 A 为根的子树失去平衡, 需要进行 RR 旋转 (左单旋)。

RR 旋转的过程如下图所示, 将 A 的右孩子 C 向左上旋转代替 A 成为根结点, 将 A 结点向下旋转成为 C 的左子树的根结点, 而 C 原来的左子树 E 则作为 A 的右子树。因此, 调整后的平衡二叉树中平衡因子的绝对值为 1 的分支结点数为 1。



▲注意: 平衡旋转的操作都是插入操作后, 在引起不平衡的最小不平衡子树上进行的, 只要将这个最小不平衡子树调整平衡, 其上级结点将恢复平衡。

08. A. 【解析】本题考查平衡二叉树的调整操作。

插入一个结点后造成了不平衡, 则一定造成了以 a 为根结点的树的平衡因子的增加, 假设插入位置在 a 的左子树上, 又由于 a 的左孩子的平衡因子为 -1 , 则插入位置一定在 a 的左孩子的右子树上 (如果在左子树上, 则不会造成以 a 为根结点的树的平衡因子的增加), 但此时最小不平衡子树却是以 a 的左孩子为根结点的子树, 与题干描述矛盾。故插入位置一定在 a 的右子树上, 既可能在 a 的右孩子的左子树上, 又可能在 a 的右孩子的右子树上, 在 a 的右子树上的任意一个叶结点下插入一个结点都有可能造成以 a 为根结点的子树的失衡。

09. C. 【解析】本题考查 B 树的高度。

磁盘存取次数取决于 B 树的高度, 对有 n 个关键字的 m 阶 B 树, 若让每个结点中的关键字个数最少, 则容纳同样数量关键字的 B 树的高度达到最大。也就是说, 第 1 层至少有 1 个结点; 第 2 层至少有 2 个结点。除根结点外的每个非终端结点至少有 $\lceil m/2 \rceil$ 棵子树, 则第 3 层至少有 $2\lceil m/2 \rceil$ 个结点……第 $h+1$ 层至少有 $2(\lceil m/2 \rceil)^{h-1}$ 个结点, 第 $h+1$ 层是不包含任何信息的叶结点。对关键字个数为 n 的 B 树, 叶结点数即查找不成功的结点数为 $n+1$, 因此有 $n+1 \geq 2(\lceil m/2 \rceil)^{h-1}$, 即 $h \leq \log_{\lceil m/2 \rceil}((n+1)/2) + 1$ 。

10. C. 【解析】本题考查各种内部排序方法的性能。

选择排序在最好、最坏、平均情况下的时间性能均为 $O(n^2)$, 归并排序在最好、最坏、平均情况下的时间性能均为 $O(n \log_2 n)$ 。各种排序方法对应的时间复杂度如下表所示。快速排序在原序列本身有序时达到最坏的时间复杂度, 插入排序在原序列本身有序时达到最好的时间复杂度。

时间复杂度	插入排序	冒泡排序	选择排序	希尔排序	快速排序	堆排序	归并排序
平均情况	$O(n^2)$	$O(n^2)$	$O(n^2)$	—	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$
最好情况	$O(n)$	$O(n)$	$O(n^2)$	—	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$
最坏情况	$O(n^2)$	$O(n^2)$	$O(n^2)$	—	$O(n^2)$	$O(n \log_2 n)$	$O(n \log_2 n)$

11. C. 【解析】本题考查多路平衡归并。



m 路平衡归并是指将 m 个有序表组合为一个新有序表。每经过一趟归并, 剩下的记录数是原来的 $1/m$, 经过 3 趟归并后有 $\lceil 29/m^3 \rceil = 1$, 4 为最小满足条件的数。

▲注意: 本题中 4 和 5 均能满足要求, 但 6 不满足要求。若 $m = 6$, 则只需 2 趟归并便可排好序。因此, 还需要满足 $m^2 < 29$, 即只有 4 和 5 才满足要求。

【另解】画出选项 A、B、C 对应的满树的草图, 然后计算结点数是否能达到或超过 29 个, 若选项 C 能到达, 则选项 D 就不必画草图了, 否则必然选择选项 D。

12. D. 【解析】本题考查数据的存储和排列。

int 型变量的长度为 32 位, 即 4B, 所以 0x804932a 中存放 -10 中的高位或低位的第 2 个字节, -10 的十六进制数为 FFFF FFF6H, 存放内容为 1111 1111, 即选项 D。

▲注意: 本题未说明采用哪种存储方式, 不论是大端方式还是小端方式, 答案均为 1111 1111。

13. B. 【解析】本题考查 IEEE 754 标准的单精度浮点数的表示范围。

最大负数要使得符号位为 1 且绝对值最小。阶码全 1 和全 0 用作特殊用途, 因此能取得的最小阶码 (移码) 的机器数为 0000 0001B; 尾数位为全 0, 因此最大规格化负数为 1 00000001 000000000000000000000000B, 其十六进制数为 80800000H。

14. A. 【解析】本题考查 Cache 命中率的相关计算。

命中率 = $4800 / (4800 + 200) = 0.96$, 因为 Cache 和主存不能同时访问, 所以当 Cache 中没有当前块时, 消耗的时间为 10ns + 50ns, 平均访问时间 = $0.96 \times 10\text{ns} + (1 - 0.96) \times (10\text{ns} + 50\text{ns}) = 12\text{ns}$ 。因此, 效率 = $10\text{ns} / 12\text{ns} = 0.833$ 。

15. C. 【解析】本题考查页式存储器中地址映射的计算。

对于本类题, 要首先将物理地址转换为“物理页号+页内地址”的形式, 然后查找页表, 找出物理页号对应的逻辑页号, 接着将“逻辑页号+页内地址”转换为对应的十进制数。页面大小为 4KB, 即页内地址为 $\log_2 4K = 12$ 位, $32773 = 32768 + 5 = 1000\ 0000\ 0000\ 0000B + 101B = 1000\ 0000\ 0000\ 0101B$, 后 12 位为页内地址, 前 4 位为页号。物理页号为 8, 对应逻辑页号为 3 = 11B。因此, 逻辑地址 = $11\ 0000\ 0000\ 0101B = 3 \times 4K + 5 = 10240 + 2048 + 5 = 12288 + 5 = 12293$ 。

16. D. 【解析】本题考查 Cache 和虚拟存储器的特性。

Cache 失效与虚拟存储器失效的处理方法不同, Cache 完全由硬件实现, 不涉及软件; 虚拟存储器由硬件和 OS 共同完成, 缺页时才发出缺页中断, 选项 I 错误、选项 II 正确、选项 III 错误。虚拟存储器的大小就是虚拟地址 (也称逻辑地址) 空间的大小, 它由虚拟地址的位数决定, 与系统中的磁盘容量和内存容量没有直接关系, 选项 IV 错误。

▲注意: Cache 和虚拟存储器都基于程序访问的局部性原理, 但它们的实现方法和作用均不太相同。Cache 是为了解决 CPU-主存的速度矛盾, 而虚存是为了解决主存容量的不足。

17. C. 【解析】本题考查各种寻址方式。

当采用直接寻址时, 操作数 = (600) = 700, 选项 I 错误; 当采用寄存器间接寻址时, 操作数 = (R) = 700, 选项 II 错误; 当采用立即寻址时, 操作数 = 600, 选项 III 正确; 当采用寄存器寻址时, 操作数 = (R) = 600, 选项 IV 正确。



18. C。【解析】本题考查转移指令的执行。

根据汇编语言指令 $\text{JMP } * -9$ ，即要求转移后的目标地址为 PC 值 -09H ，而相对寻址的转移指令占 2B，取完指令后 $\text{PC} = (\text{PC}) + 2 = 2002\text{H}$ ， $-9 = 1111\ 0111 = \text{F7H}$ ，则跳转完成后 $\text{PC} = 2002\text{H} - 9\text{H} = 2002\text{H} + \text{FFF7H} = 1\text{FF9H}$ 。

19. A。【解析】本题考查 RISC 和 CISC 的特征。

RISC 的单个指令格式简单，完成的功能少，指令的数量也少于 CISC（例如，就寻址而言，CISC 有多种寻址方式，因此指令的数量多于 RISC）。CISC 中不同的指令所花的时间不同，如 load 指令可能需要访存，所花的时间当然远大于一个时钟周期，而在 RISC 中，一个时钟周期通常执行一条微指令。CISC 采用微程序方式，RISC 采用硬布线方式。

20. C。【解析】本题考查微程序方式的工作原理。

执行公共取指令微操作（送至指令寄存器 IR）后，由机器指令的操作码字段形成对应微程序的入口地址。在选项 A 中，机器指令的地址码字段一般不是操作数就是操作数的地址，不可能作为微程序的入口地址；另外，微指令中并不存在操作码和地址码字段，只存在控制字段、判别测试字段和下地址字段，选项 B 和 D 显然错误。

21. D。【解析】本题考查微指令格式的特性。

采用字段直接编码法的微指令长度更短，因此控制存储器的利用率更高，但直接控制法（不译法）的执行速度更快。采用增量计数器法的微指令格式时，没有下地址字段，因此指令长度更短，选项 A、B、C 正确。垂直型微指令相对于水平型微指令包含的微命令更少，指令更短，更规整，微程序更长，因此选项 D 错误。

22. D。【解析】本题考查中断方式的原理。

在中断周期中，关中断由隐指令完成，而不由关中断指令完成，选项 I 错误。中断服务程序的最后一条指令是中断返回指令，选项 II 错误。DMA 接口（DMA 控制器）负责数据的传输工作，不需要 CPU 控制，选项 III 错误。DMA 方式的数据传输完全由硬件（DMA 控制器）实现，选项 IV 错误。

23. D。【解析】本题考查用户态与内核态。

打开定时器属于时钟管理的内容，对时钟的操作必须加以保护，否则，一个用户进程可在时间片还未到之前就将时钟改回去，导致时间片永远不会用完，该用户进程就一直占用 CPU，这显然不合理。从用户态到内核态是通过中断实现的，中断的处理过程很复杂，需要加以保护，但从内核态到用户态则不需要加以保护。读取操作系统内核的数据和指令是静态操作，显然无须加以保护。

24. B。【解析】本题考查进程的同步与互斥。

进程 P_0 和 P_1 写为

P_0 :	① if (turn != -1) turn = 0;	P_1 :	④ if (turn != -1) turn = 1;
	② if (turn != 0) goto retry;		⑤ if (turn != 1) goto retry;
	③ turn = -1;		⑥ turn = -1;

当执行顺序为 1, 2, 4, 5, 3, 6 时， P_0 和 P_1 将全部进入临界区，不能保证进程互斥进入临界区。



有的同学认为这道题会产生饥饿，理由如下：

当 P_0 执行完临界区时，CPU 调度 P_1 执行④。当顺序执行 1, 4, (2, 1, 5, 4), (2, 1, 5, 4), ... 时， P_0 和 P_1 进入无限等待，即出现饥饿现象。

这是对饥饿概念不熟悉的表现。饥饿的定义是：等待时间给进程推进和响应带来明显影响，称为进程饥饿”。当饥饿到一定程度的进程等待到即使完成也无实际意义时，称为饥饿死亡，简称饿死。

产生饥饿的主要原因是：在一个动态系统中，对于每类系统资源，操作系统需要确定一个分配策略，当多个进程同时申请某类资源时，由分配策略确定资源分配给进程的次序。

有时，资源分配策略可能是不公平的，即不能保证等待时间上界的存在。在这种情况下，即使系统没有发生死锁，某些进程也会长时间等待。

在本题中， P_0 和 P_1 只有满足特定的某个序列才能达到饥饿的效果，并不由资源分配策略本身的不公平造成，而这两个进程代码表现出来的策略是公平的，两个进程的地位也是平等的。满足上述条件的特定序列具有特殊性，对进程推进的不确定性而言，是基本不可能出现这种巧合的。否则，几乎所有这类进程就可能产生饥饿。

25. B. 【解析】本题考查银行家算法。

安全性检查一般要用到进程所需的最大资源数减去进程占用的资源数，得到进程为满足进程运行尚需的可能最大资源数，而系统拥有的最大资源数减去已分配的资源数，得到剩余的资源数，比较剩余的资源数是否满足进程运行尚需的可能最大资源数，就可得到当前状态是否安全的结论。因此，并无“满足系统安全的最少资源数”这种说法。

26. D. 【解析】本题考查死锁的检测。

选项 A 不发生死锁，只有一个进程怎么也不发生死锁。选项 B 不发生死锁，两个进程各需要一个资源，而系统中刚好有 2 个资源。选项 C 不发生死锁，3 个进程需要的最多资源数都是 2，系统总资源数是 4，所以总有一个进程得到 2 个资源，运行完毕后释放资源。选项 D 可能发生死锁，2 个进程各自都占有 2 个资源后，系统再无可分配资源。由此，可得出结论：满足 $m \geq n(w-1) + 1$ 时，不发生死锁。

27. C. 【解析】本题考查非连续分配管理方式。

非连续分配允许一个程序分散地装入不相邻的内存分区。动态分区分配和固定分区分配都属于连续分配方式，而非连续分配有分页式分配、分段式分配和段页式分配三种。

28. A. 【解析】本题考查页面置换的相关计算。

当物理块数为 3 时，缺页情况如下表所示。缺页次数为 6，缺页率为 $6/12 = 50\%$ 。

访问串	1	3	2	1	1	3	5	1	3	2	1	5
内 存	1	1	1	1	1	1	1	1	1	1	1	1
		3	3	3	3	3	3	3	3	3	3	5
			2	2	2	2	5	5	5	2	2	2
缺 页	√	√	√				√			√		√

当物理块数为 4 时，缺页情况如下表所示。缺页次数为 4，缺页率为 $4/12 \approx 33\%$ 。



访问串	1	3	2	1	1	3	5	1	3	2	1	5
内 存	1	1	1	1	1	1	1	1	1	1	1	1
		3	3	3	3	3	3	3	3	3	3	3
			2	2	2	2	2	2	2	2	2	2
							5	5	5	5	5	5
缺 页	√	√	√				√					

▲注意：当分配给作业的物理块数为 4 时，作业请求页面序列只有 4 个页面，可以直接得出缺页次数为 4，而不需要按表中所示的那样列出缺页情况。

29. D. 【解析】本题考查文件系统的多个知识点。

文件系统使用文件名进行管理，实现了文件名到物理地址的转换，选项 A 错误。在多级目录结构中，从根目录到任何数据文件都只有唯一一条路径，该路径从树根开始，将全部目录文件名和文件名依次用“/”连接起来，构成该数据文件的路径名。选项 B 的说法不准确，对文件的访问只需通过路径名即可。文件被划分的物理块的大小是固定的，通常和内存管理中的页面大小一致，选项 C 错误。逻辑记录是文件按信息逻辑独立的含义来划分的信息单位，它是对文件进行存取操作的基本单位，选项 D 正确。

30. C. 【解析】本题考查文件控制块和索引节点。

引入索引节点前，每个目录项中存放的是对应文件的 FCB，256 个目录项的目录共需占用 $256 \times 64 / 512 = 32$ 个盘块，因此在该目录中检索一个文件时，平均启动磁盘的次数为 $(1 + 32) / 2 = 16.5$ 次。引入索引节点后，每个目录项中只需存放文件名和索引节点编号，256 个目录项的目录共需占用 $256 \times (6 + 2) / 512 = 4$ 个盘块，因此找到匹配的目录项平均需要启动 $(1 + 4) / 2 = 2.5$ 次磁盘，得到索引节点编号后，还需启动磁盘将对应文件的索引节点读入内存，因此平均需要启动磁盘 3.5 次。可见，平均启动磁盘的次数减少了 13 次。

31. B. 【解析】本题考查设备独立性的定义。

设备独立性是指用户程序独立于具体使用的物理设备的一种特性，引入设备独立性是为了提高设备分配的灵活性和设备的利用率等。

32. B. 【解析】本题考查固态硬盘（SSD）的性质。

固态硬盘基于闪存技术，不属于磁表面存储器，选项 A 错误。闪存翻译层可将 CPU 的逻辑磁盘块读/写请求转化为对底层固态硬盘物理设备的读/写控制信号，选项 B 正确。固态硬盘无论是读还是写，速度都远快于常规硬盘，选项 C 错误。固态硬盘块内的页必须按顺序写入信息，选项 D 错误。

33. A. 【解析】本题考查网络体系结构的原则和特点。

网络体系结构是抽象的，它不包括各层协议及功能的具体实现细节。若规定层的名称和功能，则难以保持网络的灵活性。分层使得各层之间相对独立，各层仅需关注该层需要完成的功能，保持了网络的灵活性和封装性，但网络的体系结构并未规定层的名称和功能必须一致，选项 A 正确。不同的网络体系结构划分出的结构不尽相同，如 OSI 模型与 TCP/IP 模型就不尽相同，选项 B 错误。分层应划分网络的功能，而不将相关的网络功能组合到一层中，选项 C 错误。



分层不涉及具体功能的实现，选项 D 错误。

▲注意：典型的 OSI 模型很好地体现了网络体系结构设计的初衷。

34. D. 【解析】本题考查 BGP 的特点。

BGP 路由在 AS 边界路由器上通过 eBGP 从外部 AS 学习后，可通过 iBGP 通告给同一个 AS 内的其他路由器，以实现内部传播，选项 I 正确。路由器从同一个 AS 内的 iBGP 对等方学习到的路由（如本地生成的或从外部学到的），可经过属性修改（如添加本地 AS 号到 AS_PATH）后，通过 eBGP 通告给不同 AS 内的对等方，选项 II 正确。从一个 iBGP 对等方学习到的路由不会通告给同一 AS 内的其他 iBGP 对等方，这是为了防止 AS 内的路由环路，选项 III 正确。在基本 BGP 实现中，iBGP 要求所有路由器在逻辑上建立全互联对等会话，以确保路由信息一致并防止环路，也就是说，即使路由器之间没有直连的物理链路，只要 IP 可达，iBGP 会话即可建立，选项 IV 正确。

35. C. 【解析】本题考查停止-等待协议的效率分析。

停止-等待协议发送一个分组后，需要收到确认后才能发送下一个分组。发送延迟 = $8\text{bit} \times 100 / (2 \times 1000000\text{b/s}) = 0.0004\text{s}$ ，传播延迟 = $1000\text{m} / (20\text{m/ms}) = 50\text{ms} = 0.05\text{s}$ ，最小间隔 = $0.0004\text{s} + 0.05\text{s} \times 2 = 0.1004\text{s}$ 。因此，数据率 = $8 \times 100\text{bit} / 0.1004\text{s} \approx 8\text{kb/s}$ 。

36. A. 【解析】本题考查交换机的特性。

交换机是二层设备，看不到 IP 地址，选项 D 错误。交换机收到一个帧时，只有源主机的方向是确定的，目的主机往哪个方向转发不一定知道（除非表项中有目的 MAC，此时也不用将目的 MAC 添加到表项中），因此将源 MAC 地址加入表项（如果本来不在其中的话），这就是交换机的自学习，因此选择选项 A。

37. B. 【解析】本题考查子网划分与子网掩码。

不同子网之间需通过路由器相连，子网内的通信则不需要经过路由器转发，因此比较各主机的子网号即可。将子网掩码 255.255.192.0 与主机 129.23.144.16 进行“与”操作，得到该主机网络的地址为 129.23.128.0，再将该子网掩码分别与 4 个候选答案的地址进行“与”操作，只有 129.23.127.222 的网络地址不为 129.23.128.0。因此该主机与 129.23.144.16 不在一个子网中，需要通过路由器转发信息。

▲注意：回答这种题时，要将用到的十进制数转换为二进制数，而不能凭感觉来做选择，否则容易导致错误。

38. C. 【解析】本题考查 IGMP。

IGMP 并非是在互联网范围内对所有多播组成员进行管理的协议，IGMP 不知道 IP 多播组包含的成员数，也不知道这些成员都分布在哪些网络上，选项 A、B 错误。IGMP 的工作过程可分为两个阶段：① 当某个主机加入新的多播组时，该主机应向多播组的多播地址发送一个 IGMP 报文，声明自己要成为该组的成员。本地的多播路由器收到 IGMP 报文后，还要利用多播路由选择协议把这种组成员关系转发给互联网上的其他多播路由器。② 组成员关系是动态的。本地多播路由器要周期性地探测本地局域网上的主机，以便知道这些主机是否还继续是组的成员。只要有一个主机对某个组响应，多播路由器就认为这个组是活跃的，但一个组在经过几次探测

后仍然没有一个主机响应，多播路由器就认为本网络上的主机都已离开这个组，因此不再把这个组的成员关系转发给其他多播路由器。由此可知选项 C 正确、选项 D 错误。

39. D. 【解析】本题考查 TCP 的端口号。

端口号是 16 位无符号整数，范围是 0~65535（共 65536 个），其中 0 通常保留，选项 I 正确。TCP 连接由<源 IP 地址、源端口、目标 IP 地址、目标端口>唯一标识，因此一台主机可建立远超 65536 个并发连接（只要目标 IP 地址或目标端口不同），选项 II 错误。TCP 连接关闭后，端口号会被释放，可重新用于新连接（需等待 TIME WAIT 状态结束，通常为 2MSL），选项 III 正确。通过绑定到不同目标 IP 地址或目标端口，一个本地端口可以建立多个 TCP 连接，例如<192.168.1.1:50000, 1.1.1.1:80>和<192.168.1.1:50000, 2.2.2.2:80>是两个独立的 TCP 连接，选项 IV 正确。

40. B. 【解析】本题考查各种协议的应用。

刚开机时 ARP 表为空，当需要和其他主机通信时，数据链路层需要使用 MAC 地址，因此要用到 ARP。通过校园网访问因特网时，肯定要用到 IP。因为此时访问的是因特网，因特网为公网，所以需要通过 DHCP 分配公网地址。ICMP 主要用于发送 ICMP 差错报告报文和 ICMP 询问报文，因此不一定用到。

二、综合应用题

41. 【解析】

1) 顺序存储。

看上去，堆的每个结点都有左子树和右子树，且经常需要与它们交换，但实际上是采用顺序表保存的，类似于完全二叉树的顺序表保存，结点编号就是其在数组中的位置。

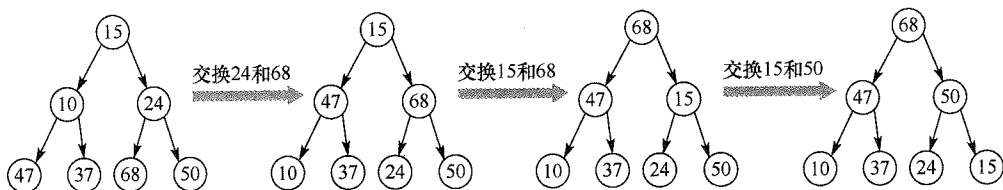
2) 堆的数据结构定义如下：

```
struct Heap {  
    ElementType data[Maxsize]; //存储元素的数组  
    int size; //堆中元素个数  
};
```

其他写法（简单写法）：

```
int data[Maxsize]; //使用 data[0]来保存堆中的元素个数
```

- 3) 堆中的元素有 7 个，从第 $7/2 = 3$ 个开始元素为根的子树开始处理，先处理 3 号元素 24，发现比 68 小，交换 24 和 68；然后处理 2 号元素 10，发现比 47 小，交换 10 和 47；再后处理 1 号元素 15，发现比 68 小，15 和 68 交换。因为 3 号元素与 1 号元素交换，破坏了以 3 号元素为根的子树的堆性质，所以再从 3 号元素向下判断，发现 3 号元素 15 比 50 小，交换 15 和 50，此时不需要再向上判断。建堆过程如下图所示。



42. 【解析】

1) 定义如下:

```
typedef struct node{
    int data;
    struct node *firstchild, *nextbro;
}TNode;
```

2) 结点 p 的度 $= 1 + p$ 的第一个左孩子后面的兄弟数, 对树进行后序遍历, 往 firstchild 域递归, 度 d 不变, 往 nextbro 域递归, 度 $d = d + 1$, 求出每个结点后面的兄弟有多少个, 即可求出所有结点的度, 进而求出树 T 的度。

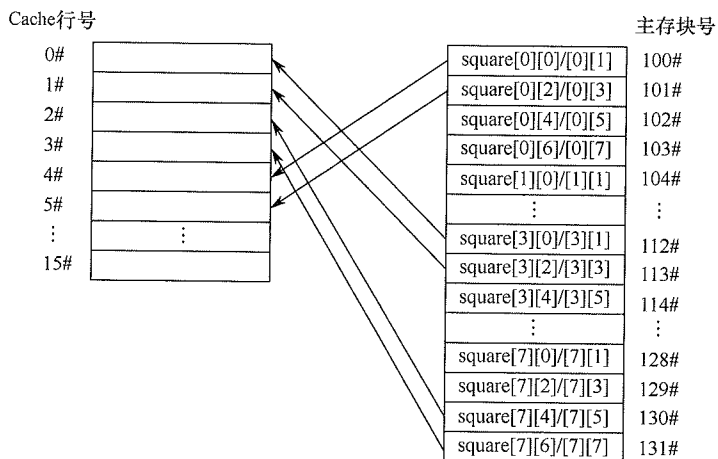
3) 算法的代码如下:

```
int degree=0; //全局变量, 最终输出 degree
int Post(TNode* p){ //后序遍历
    if (p==NULL)
        return 0; //该结点不存在
    Post(p->firstchild);
    int d=1+Post(p->nextbro);
    degree=max(degree, d);
    return d;
}
```

43. 【解析】

1) 在程序段 A、B 和 C 中, 每个数组元素都只被访问一次, 都没有时间局部性; 程序段 A 的访问顺序和存放顺序一致, 空间局部性好; 程序段 B 的访问顺序和存放顺序不一致, 空间局部性不好; 程序段 C 的访问顺序和存放顺序部分一致, 空间局部性的优劣介于程序段 A 和 B 之间。

2) 一个数组元素占 $4 \times 4B = 16B$, 每个块 (或 Cache 行) 能存放 2 个数组元素。8×8 的数组共占 32 个主存块, 正好是 Cache 数据区大小的 2 倍。数组 square 的首地址为 0000 0C80H = 0...0 1100 1000 0000B, 最低 5 位为全 0, 表示主存第 1100100B 块的起始地址, 因此第一个数组元素位于主存的第 100 块。Cache 行数为 $512B/32B = 16$, $100 \bmod 16 = 4$, 所以主存第 100 块映射到的 Cache 行号为 4。主存中的数组元素与 Cache 行的映射关系如下图所示。



3) 程序段 A: 每 2 个数组元素 (共 8 次写操作) 装入一个 Cache 行, 总是第一次不命中, 后面 7 次都命中, 总写操作次数为 $64 \times 4 = 256$, 写不命中次数为 $256 \times 1/8 = 32$, 写缺失率为 12.5%。



程序段 B: 每 2 个数组元素 (共 8 次写操作) 装入一个 Cache 行, 总是只有 1 个数组元素 (共 4 次写操作) 在被淘汰前访问, 且总是第一次不命中, 后面 3 次都命中, 写不命中次数为 $256 \times 1/4 = 64$, 写缺失率为 25%。

程序段 C: 第一个循环共访问 64 次, 每次装入 2 个数组元素, 第一次不命中, 第二次命中; 第二个循环共访问 $64 \times 3 = 192$ 次, 每 2 个数组元素 (共 6 次写操作) 装入一个 Cache 行, 且总是第一次不命中, 后面 5 次都命中, 写不命中次数为 $32 + (3 \times 64) \times 1/6 = 64$, 写缺失率为 25%。

44. 【解析】

1) 因为寄存器 R 是 5 位二进制数, 所以有 $2^5 = 32$ 个通用寄存器。因为一条指令 32 位, 占 4 个地址, 所以编址单位是 $32\text{bit}/4 = 8\text{bit} = 1\text{B}$ 。

2) 方法 1: 可以根据 C 语言代码判断 (比较简单): 第 1 条指令给 for 循环中的 i 赋初始值, 之后的指令都是循环执行的, 所以 loop 指向第 2 条指令 (这条指令的含义是根据 i 的值求出相对于 B 首址的偏移), 地址为 00003004H。

方法 2: 根据机器代码判断 (较难): 查看 bne 指令格式, 得到补码 OFFSET = FFF9H, 对应的有符号数真值为 -7, 因为是字偏移量且字长等于指令长度, 所以相当于往前跳转了 7 条指令 (PC 在取指完自增后, 应该从第 9 条指令开始向前跳转 7 条), 即 $8 + 1 - 7 = 2$, loop 指向第二条指令, 地址为 00003004H。

3) 循环执行 2~8 号共 7 条指令, 循环 5 次, 其中 5 条运算类指令, 1 条访存类指令, 1 条分支跳转类指令。第一条指令是运算类指令, 只执行一次, 整个过程执行 $7 \times 5 + 1 = 36$ 条指令。

$$\text{CPI} = (5 \times (5 \times 4 + 5 + 3) + 4) / 36 = 4$$

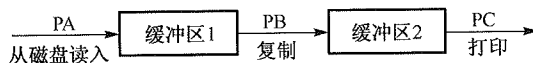
$$\text{IPS} = \text{主频} / \text{CPI} = 25\text{M}, \text{MIPS} = 25$$

$$T = 36 / \text{IPS} = 1.44 \times 10^{-6} \text{s} = 1.44\mu\text{s}$$

4) 指令 1 和 2、2 和 3、3 和 4、4 和 5、6 和 7、7 和 8 都会因为需要使用的寄存器还未被上一条指令写入导致数据相关而阻塞。分支跳转类指令会产生控制相关, 所以指令 8 会产生控制相关。

45. 【解析】

进程 PA、PB、PC 之间的关系为: PA 与 PB 共用一个单缓冲区, PB 又与 PC 共用一个单缓冲区, 其合作方式如下图所示。当缓冲区 1 为空时, 进程 PA 可将一条记录读入; 若缓冲区 1 中有数据且缓冲区 2 为空, 则进程 PB 可将记录从缓冲区 1 复制到缓冲区 2 中; 若缓冲区 2 中有数据, 则进程 PC 可以打印记录。在其他条件下, 相应进程必须等待。事实上, 这是一个生产者-消费者问题。



为遵循这一同步规则, 应设置 4 个信号量 empty1、empty2、full1、full2, 信号量 empty1 和 empty2 分别表示缓冲区 1 和缓冲区 2 是否为空, 其初始值为 1; 信号量 full1 和 full2 分别表示缓冲区 1 和缓冲区 2 是否有记录可供处理, 其初始值为 0。相应的进程描述如下:

```

semaphore empty1=1;    //缓冲区 1 是否为空
semaphore full1=0;     //缓冲区 1 是否有记录可供处理
  
```



```

semaphore empty2=1;      //缓冲区 2 是否为空
semaphore full2=0;       //缓冲区 2 是否有记录可供处理
cobegin{
process PA(){
    while(TRUE){
        从磁盘读入一条记录;
        P(empty1);
        将记录存入缓冲区 1;
        V(full1);
    }
}
process PB(){
    while(TRUE){
        P(full1);
        从缓冲区 1 中取出一条记录;
        V(empty1);
        P(empty2);
        将取出的记录存入缓冲区 2;
        V(full2);
    }
}
process PC(){
    while(TRUE){
        P(full2);
        从缓冲区 2 中取出一条记录;
        V(empty2);
        将取出的记录打印出来;
    }
}
} coend

```

46. 【解析】

- 1) 由图可见，目前系统共有 4 组空闲盘块，第 1 组有 2 块，第 2、3 组分别有 100 块，第 4 组虽记为 100 块，但除去结束标记 0 后实际上只有 99 块，因此空闲盘块总数为 301。
- 2) 第 1 组只有 2 个磁盘块，将第 1 组中的 299 号磁盘块分配给文件 F1 后，需要将 300 号盘块的内容读入栈，作为新的空闲盘块号栈的内容，并将 300 号盘块分配给 F1（其中有用的数据已读入栈），这样就为该文件分配了 2 个磁盘块，最后将新的栈顶 301 号对应的盘块分配给 F1，此时空闲盘块号栈的栈顶盘块号变为 302。
- 3) 回收空闲盘块时，将回收的盘块号存入空闲盘块号栈的顶部，并将栈中的空闲盘块数加 1。回收 700 号盘块后，栈中的空闲盘块数已达 100，表示栈已满，将现有栈中的 100 个空闲盘块号存入新回收的 711 号盘块，并将盘块号 711 作为新栈底，然后继续回收 703, 788, 701 号盘块，回收结束后，空闲盘块号栈中依次保存着盘块号 711, 703, 788, 701，其中盘块号 701 登记在栈顶，空闲盘块号栈的栈顶指针 s_nfree 的值为 4。

47. 【解析】

- 1) 编号为 2, 3, 6 的数据报为主机 A 接收的 IP 数据报，其他均为主机 A 发送的数据报，由主机 A 发送的数据报中的源 IP 地址可知主机 A 的 IP 地址为 c0 a8 00 15。对比编号为 2, 3, 6



的数据报, 可知 2 号数据报来自同一个发送方, 3, 6 号数据报来自同一个发送方, 由 2 号帧的源 IP 地址和目的 IP 地址以及其协议字段 (ICMP) 可知, 该数据报来自不知名的一方 (可能是网络中的某个结点), 而 3, 6 号来自主机 B, 主机 B 的 IP 地址为 c0 a8 00 c0, 1 号数据报的 SYN 值为 1, 3 号数据报的 SYN 值也为 1, 代表这是前两次握手的数据报, 根据 IP 数据报标识字段的值可知, 4 号数据报是主机 A 发送的第二个 IP 数据报, 因此三次握手后应该是编号为 1, 3, 4 的三个数据报。连接建立后, 由主机 A 最后的 4 号确认报文段以及之后发送的 5 号报文段可知, SEQ 字段为 22 68 b9 91, ACK 号为 5b 9f f7 1d, 进而可知主机 A 期望接收对方的下一个报文段的数据中的第一个字节的序号为 5b 9f f7 1d, 也就是说, 若 B 发送数据给 A, 则首字节的编号应该是 5b 9f f7 1d。

- 2) 主机 A 从 4 号报文段携带应用层数据, 所以只需将 4, 5, 7, 8 报文中的数据部分加起来, 观察 4, 5, 7, 8 号报文的头部长度字段, 均为 5, 表示 TCP 头部长度均为 $5 \times 4B = 20B$, 由图可知, 从第三行开始的内容均为要传输的数据, 其和为 $0 + 16 + 16 + 32 = 64B$ 。
- 3) 主机 B 接收到主机 A 的 IP 分组后, 在 8 号报文段的序号字段的基础上加上其发送的数据字节数, 即 $2268B9A1H + 32D = 2268B9C1H$ 。

B 在 6 号报文段中指出自己的窗口字段为 8192B, 说明此时 B 还能接收到这么多数据。之后 A 发送两个报文段。由 7 号和 8 号报文段的序号和确认号可知, 8 号是 7 号的重复发送数据, 所以 B 只需接收 8 号的数据部分, 也就是 32B。因此, 之后 A 还可发送的字节数为 $8192B - 32B = 8160B$ 。



全国硕士研究生入学统一考试

计算机科学与技术学科联考

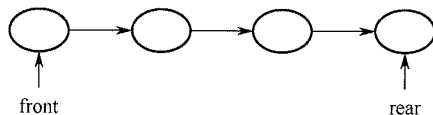
计算机专业基础综合考试模拟试卷（八）参考答案

一、单项选择题（第 01~40 题）

01. D 02. C 03. C 04. C 05. C 06. D 07. B 08. C
09. D 10. D 11. B 12. A 13. A 14. B 15. B 16. D
17. D 18. C 19. B 20. B 21. D 22. A 23. C 24. D
25. B 26. D 27. B 28. A 29. A 30. D 31. C 32. D
33. A 34. B 35. C 36. A 37. B 38. C 39. B 40. D

01. D. 【解析】本题考查链式队列的操作。

题中所述的链式队列 Q 如下图所示。元素在队尾进队，因此要让队尾指针指向新进队的元素，即 $Q.rear \rightarrow link = s$ ，然后让新队尾元素指向空，即 $s \rightarrow link = NULL$ ，最后让队尾指针指向新队尾元素，即 $Q.rear = s$ 。



02. C. 【解析】本题考查中缀转后缀。

采用模拟的方式，对于符号栈，首先“-”入栈，“(”入栈，“×”入栈，到“+”时因为“×”的优先级高于“+”，所以“×”出栈而“+”入栈，然后“+”的优先级低于“/”，所以“/”入栈，此时栈中有“-”“(”“+”和“/”共4个元素，再访问“)”，将“(”后的元素全部出栈，所以栈中最多4个元素。

03. C. 【解析】本题考查对称矩阵的压缩存储。

因为 $B[0]$ 存储的是 $A[1][1]$ ， $B[1]$ 存储的是 $A[1][2]$ ，可知存储的是对称矩阵 A 的上三角矩阵，上三角的前4行共有 $10 + 9 + 8 + 7 = 34$ 个元素，因此 $B[35]$ 存储的是上三角的第5行第2个元素，行下标为5，列下标为6，表示的是5号和6号顶点之间的最短距离。

04. C. 【解析】本题考查 KMP 算法的 $nextval$ 优化。

先根据 $i = j = 2$ 时的失配判断出字符的下标应从0开始，因此 $next[0] = -1$ ，求出模式串 T 的 $nextval$ 数组。

下 标	0	1	2	3	4	5
T	a	a	a	a	b	c
next	-1	0	1	2	3	0
nextval	-1	-1	-1	-1	3	0



$j=2$ 时 $nextval[j]=-1$ ，下次开始比较时， $i++$ ， j 从 0 开始，因此 $i=3$ ， $j=0$ 。

05. C. 【解析】本题考查二叉树的性质。

假设二叉树中度为 2 的结点数为 x ，树的总度数 $+1 =$ 结点数，即 $2x + m + 1 = k + m + x$ ，可知 $x = k - 1$ ，因此二叉树的结点总数为 $2k - 1 + m$ 。除根结点外，每个结点都由一个指针指向，因此指向孩子结点的指针数量是 $2k - 1 + m - 1 = 2k + m - 2$ 。

06. D. 【解析】本题考查深度优先遍历和广度优先遍历。

选项 A，先访问顶点 A 及其三个相邻结点 B 、 C 和 E ， B 无未访问的相邻结点，再访问 C 的相邻结点 F ，后访问 E 的相邻结点 D ，是广度优先遍历。选项 B 和 A 一致，也是广度优先遍历。选项 C，先访问 A ，然后沿着深度一直访问 E 、 D 、 F 和 C ，最后访问 B ，是深度优先遍历。选项 D，先访问 A ，然后沿着深度一直访问 E 和 D ，接着访问 C ，此时不是按照深度遍历的，因此既不是深度优先遍历，又不是广度优先遍历。

07. B. 【解析】本题考查图的遍历的特点。

在 BFS 中，先遍历所有相邻结点，因此生成的树通常会比较“矮”；而在 DFS 中会一直向下延伸直到最深的结点，因此生成的树通常会比较“高”，选项 B 正确。

08. C. 【解析】本题考查平衡二叉树的性质与查找操作。

设 N_h 是深度为 h 的平衡二叉树中含有的最少结点数，则有 $N_0 = 0$ ， $N_1 = 1$ ， $N_2 = 2$ ， $\dots$ ， $N_h = N_{h-1} + N_{h-2} + 1$ ， $N_3 = 4$ ， $N_4 = 7$ ， $N_5 = 12$ ， $N_6 = 20 > 15$ （考生应能画出图形）。也就是说，高度为 6 的平衡二叉树最少有 20 个结点，因此 15 个结点的平衡二叉树的高度为 5，而最小叶结点的层数为 3，选项 D 错误。选项 B 的查找过程不能构成排序二叉树，错误。选项 A 根本就不包含 28 这个值，错误。

09. D. 【解析】本题考查红黑树的性质。

红黑树和 AVL 树查找的时间复杂度都是 $O(\log_2 n)$ ，但关键字一样时红黑树一般比 AVL 树高，所以一般红黑树比 AVL 树查找得慢，选项 A 错误。插入和删除时，红黑树比 AVL 树更快，选项 B 错误。AVL 树对平衡的要求比红黑树高，选项 C 错误。红黑树和 AVL 树的插入、删除操作的时间复杂度都是 $O(\log_2 n)$ ，选项 D 正确。

10. D. 【解析】本题考查散列表的构造过程。

任何散列函数都不可能绝对地避免冲突，因此要采用合理的冲突处理方法，为冲突的关键字寻找下一个“空”位置。将前面各元素分别放入散列表，其中 8, 9, 10 的位置分别存放 25, 26, 8。元素 59 经过散列函数计算应该存入位置 $59 \bmod 17 = 8$ ，发生冲突，采用线性探测再散列，依次比较 9, 10, 11，发现 11 为空，所以将其放入地址 11。各关键字对应的散列地址如下表所示。

关键字	26	25	72	38	8	18	59
散列地址	9	8	4	4	8	1	8



11. B. 【解析】本题考查各种排序算法的特点。
在最好情况下，只有直接插入排序和冒泡排序的时间复杂度为 $O(n)$ ，即线性时间。
12. A. 【解析】本题考查小端存储方式和基址寻址。
先把补码 FF00H 扩充到 32 位，即 FFFF FF00H。
 $C000\ 0000H + FFFF\ FF00H = (1)\ BFFF\ FF00H$ ，所以操作数首地址为 BFFF FF00H，又因为是小端存储方式，LSB 在低位，所以 LSB 存放在 BFFF FF00H。
13. A. 【解析】本题考查强制类型转换及混合运算中的类型提升。
具体计算步骤如下： $a+b=13$ ； $(float)(a+b)=13.000000$ ； $(float)(a+b)/2=6.500000$ ； $(int)x=4$ ； $(int)y=3$ ； $(int)x\%(int)y=1$ ；加号前是 float，加号后是 int，两者的混合运算的结果类型提升为 float 型。因此，表达式的值为 7.500000。
▲注意：
强制类型转换：格式为“TYPE b=(TYPE)a”，执行后，返回一个 TYPE 型的数值。
类型提升：不同类型的数据混合运算时，遵循类型提升的原则，即较低类型转换为较高类型。
14. B. 【解析】本题考查原码乘法的运算过程。
原码一位乘法由被乘数和乘数进行 n 次加法和 n 次移位运算，再加上 1 次异或运算求结果的符号位，因此最多需要 $n+1$ 次 ALU 运算、 n 次移位运算。注意， n 是指去掉符号位后剩下的位数。本题中的原码是 32 位，因此 $n=31$ ，需要 $2n+1=63$ 个时钟周期。
15. B. 【解析】本题考查存储器带宽计算。
每个存储体在一个存储周期可提供 64bit，因为八体低位交叉可并发执行，所以要 $\times 8$ ，有 $64\text{bit} \times 8 / (80\text{ns}) = 6400\text{Mb/s} = 800\text{MB/s}$ 。
16. D. 【解析】本题考查数据的小端存储方式。
数组元素的访问通常采用变址寻址方式，数组起始地址通常是指令中直接给出的形式地址，下标变量存放在变址寄存器。在本题中，数组元素的类型为 float 型，因此每个数组元素占 4B，最后一个元素的下标为 99，其首地址为 $C000\ 1000H + 99 \times 4 = C000\ 1000H + 0000\ 018CH = C000\ 118CH$ ，采用小端存储方式，因此 MSB 所在的地址是 C000 118FH。
17. D. 【解析】本题考查 CALL 指令的执行。
执行子程序调用 CALL 指令时，需要将程序断点即 PC 的内容保存在栈中，然后将 CALL 指令的地址码送入 PC。取出 CALL 指令后，PC 的值加 2 变为 10002H，CALL 指令执行后，程序断点 10002H 进栈，此时 $SP=00FFH$ ，栈顶内容为 1002H。
▲注意：PC 自增的数量取决于指令长度。
18. C. 【解析】本题考查相对寻址。
相对寻址方式的基准地址为 PC 中的内容，计算转移目标地址时，CPU 已从存储器中取出操作码和相对位移量，因此，PC 中已经是转移指令所在地址加 2 的值，即 $200CH + 2 = 200EH$ 。
相对寻址方式的转移目标地址 = PC + 位移量，因此，位移量 = 转移目标地址 - PC =



$1FB0H-200EH = 0001\ 1111\ 1011\ 0000B - 0010\ 0000\ 0000\ 1110B = 00011111\ 1011\ 0000B + 1101\ 1111\ 1111\ 0010B = 1111\ 1111\ 1010\ 0010B$ 。因为位移量占 1B，所以在指令中的位移量为 A2H。运算时位移量扩展 8 位，即 FFA2H，位移量为负数，选项 C 正确。

19. B. 【解析】本题考查指令的寻址方式。

指令 2222H 转换成二进制数为 0010 0010 0010 0010，寻址特征位 $X = 10$ ，因此用变址寄存器 X2 进行变址，位移量 $D = 22H$ ，于是有效地址 $EA = 1122H + 22H = 1144H$ 。

20. B. 【解析】本题考查透明性问题。

选项 A、C、D 中的内容都可直接在汇编语言中出现和改变，而 Cache 是 CPU 内部供 CPU 访问的（为了缓解 CPU 和内存的速度差异），Cache 机制完全由硬件实现，对程序员不可见。

21. D. 【解析】本题考查指令流水线的划分原则。

将最后两个功能部件 E 和 F 合并后，五段式指令流水线的每个部件所花的时间分别为 80ps, 40ps, 50ps, 70ps, 50ps，再加上流水段寄存器延时 20ps，此时耗时最长的是第一段流水段，总延时为 100ps，因此该五段式流水线 CPU 的时钟周期至少是 100ps。

22. A. 【解析】本题考查 DMA 方式中的中断与中断传输方式的区别。

DMA 方式的中断部件用于向 CPU 报告数据传输结束，中断 I/O 方式用于传输数据，DMA 接口中的中断部件仅向 CPU 发出传输结束的信号，而检查数据是否出错则由具体的中断服务程序完成。

23. C. 【解析】本题考查用户态和内核态。

有两种方法：第一种方法是看指令的频率，因为从用户态切换到内核态需要大量的时间，所以仅在内核态下运行的指令很少，算术运算、从内存中取数、将结果送入内存存在每条指令中都可能多次出现，所以是用户态指令，而输入/输出的使用频率较低，更有可能只在内核态下运行；第二种方法是看指令对系统的影响，算术运算、从内存中取数、将结果送入内存一般都只影响到计算的结果，而输入/输出需要使用 I/O 设备，涉及资源使用，有可能影响到其他进程及危害计算机，所以不能在用户态下执行。

24. D. 【解析】本题考查记录型信号量的性质。

执行 V 操作后， $S.value \leq 0$ ，说明在执行 V 操作之前 $S.value < 0$ （此时 $S.value$ 的绝对值是阻塞队列中的进程数），所以阻塞队列中必有进程在等待，因此需要唤醒一个阻塞队列进程，选项 I 正确。由选项 I 的分析可知， $S.value \leq 0$ 就会唤醒。因为可能在执行 V 操作前只有一个进程在阻塞队列中，也就是说 $S.value = -1$ ，执行 V 操作后，唤醒该阻塞进程， $S.value = 0$ ，所以选项 II 错误。 $S.value$ 的值和就绪队列中的进程没有这一层关系，选项 III 错误。而当 $S.value > 0$ 时，说明没有进程在等待该资源，系统自然不做额外的操作，选项 IV 正确。

25. B. 【解析】本题考查死锁定理。

根据死锁定理，首先需要找出既不阻塞又非孤点的进程。对于图 I，由于资源 R_2 已分配了 2



个, 还剩一个空闲的 R_2 , 可以满足进程 P_2 的需求, 因此 P_2 是这样的进程。 P_2 运行结束后, 释放一个资源 R_1 和两个资源 R_2 , 可以满足 P_1 进程的需求, 从而系统的资源分配图可以完全简化, 不处于死锁状态。对于图 I, P_1 需要资源 R_1 , 但是唯一一个资源 R_1 已分配给 P_2 ; 同理, P_2 需要资源 R_4 , 而资源 R_4 也只有一个且已分配给 P_3 ; 而 P_3 还需要一个资源 R_2 , 但两个资源 R_2 都已分配完毕, 所以 P_1, P_2, P_3 都处于阻塞态, 系统中不存在既不阻塞又非孤点的进程, 所以图 II 处于死锁状态。

▲注意: 在进程资源图中, $P \rightarrow R$ 表示进程正在请求资源, $R \rightarrow P$ 表示资源已分配给进程 (资源只能是被动的)。

26. D. 【解析】本题考查系统的安全状态和安全序列。

当 Available 为 (2, 3, 3) 时, 可以满足 P_4, P_5 中任何一个进程的需求。这两个进程结束后释放资源, Available 为 (7, 4, 11) 时满足 P_1, P_2, P_3 中任何一个进程的需求, 因此该时刻系统处于安全状态, 安全序列中只有选项 D 满足条件。

27. B. 【解析】本题考查虚拟页式存储管理中多级页表的计算。

由题中所给的条件, 虚拟地址空间是 2^{48} , 即未完全使用 64 位地址。页面大小为 2^{13} , 即 8KB, 则用于分页的地址线的位数为 $48 - 13 = 35$ 。下面计算每级页表能容纳的最多数目。由题意可知, 每个页面为 8KB, 每个页表项为 8B, 则一页中能容纳的页表项为 $8KB / 8B = 1K$, 即 1024 个页表项, 可占用 10 位地址线来寻址, 剩余的 35 位地址线可分为 $35 / 10 = 3.5$, 向上取整为 4, 因此至少 4 级页表才能完成此虚拟存储的页面映射。

28. A. 【解析】本题考查虚拟存储器的地址转换。

对于这类题, 首先要写出地址结构, 每页 $1KB = 2^{10}B$, 即页内地址为 10 位, 虚拟存在器有 $1024 \text{ 页} = 2^{10} \text{ 页}$, 即虚拟页号为 10 位, 主存为 $64KB = 2^6 \text{ 页}$, 即物理页框号为 6 位。虚拟地址为 $0x00A6F$ 即 $0000000010 \ 1001101111$, 页号为 2, 该页对应的页框号为 4, 物理地址为 $000100 \ 1001101111$, 即 $0x126F$ 。

29. A. 【解析】本题考查多级索引下文件的存放方式。

本题是一个简化的多级索引题。根据题意, 它采用的是三级索引, 则索引表应有三重。由题意可知, 每个盘块为 1024B, 每个索引号占 4B, 因此每个索引块可存放 256 条索引号, 三级索引总共可以管理文件的大小为 $256 \times 256 \times 256 \times 1024B \approx 16GB$ 。

30. D 【解析】本题考查设备驱动程序的功能。

设备驱动程序与 I/O 设备具体的 I/O 控制方式密切相关, 在不同的 I/O 控制方式中, 设备驱动程序所完成的工作内容有很大的不同, 选项 D 错误。选项 A、B、C 均正确。

31. C. 【解析】本题考查磁盘调度算法。

向磁道序号增加的方向移动, 首先排除选项 A、B; 选项 D 是到达端点后先回到最小值, 不扫描磁道, 再往磁道序号增大的方向移动; SCAN 算法是到达端点后往当前方向相反的方向扫描; 选项 C 符合 SCAN 算法的定义, 选择选项 C。



32. D. 【解析】本题考查磁盘调度算法。

SCAN、C-SCAN、LOOK、C-LOOK 算法都以确定的方向移至两端的端点或两端最远的访问点，方向不会随时改变，而最短寻道时间优先算法每次都往最近的访问点移动，方向有可能随时改变，因此选择选项 D。

33. A. 【解析】本题考查网络参考模型的服务访问点。

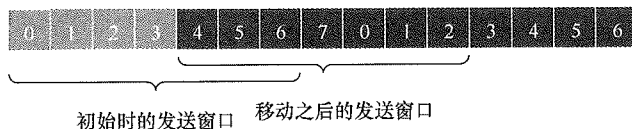
在以太网帧中，有目的地址、源地址、类型、数据部分、FCS 共 5 个字段，其中“类型”字段是数据链路层的服务访问点，它指出数据字段中的数据应交给哪个上层协议，如网络层的 IP。此外，网络层的服务访问点为 IP 数据报的“协议”字段，传输层的服务访问点为“端口号”字段，应用层的服务访问点为“用户界面”。

34. B. 【解析】本题考查物理层设备。

电磁信号在网络传输媒介中进行传递时会衰减而使信号变得越来越弱，还会由于电磁噪声和干扰使信号发生畸变，因此需要在一定的传输媒介距离中使用中继器，以对传输的数据信号整形放大后再传递。放大器常用于远距离模拟信号的传输，但它同时也使噪声放大，引起失真。交换机是链路层的设备，它能将网络分成小的冲突域，为每个用户提供更高的带宽。路由器是网络层的互联设备，可以实现不同网络的互联。中继器的工作原理是信号再生（不是简单的放大），可以延长网络的长度。

35. C. 【解析】本题考查 GBN 协议的理解和计算。

采用 GBN 协议传输数据，使用 3 比特给帧编号，因此数据帧的序号范围为 0~7，发送窗口的最大值为 $2^3 - 1 = 7$ ，GBN 协议默认采用累积确认，发送方收到 1, 3 号帧的确认，表示 0~3 号帧都已被接收方正确接收，4, 5, 6 号帧发生超时，因此需要重传，发送窗口向前移动 4 个数据帧，因此发送窗口内的最后一个数据帧的序号是 2。



36. A. 【解析】本题考查 CSMA/CD 协议的最小帧长。

要在发送的同时进行冲突检测，就要在能检测出冲突的最大时间内数据不能发送完毕，否则冲突检测不能有效地工作。因此，当发送的数据帧太短时，必须进行填充。最小帧长 = 数据传输率 × 争用期。争用期 = 网络中两个站点最大的往返传播时间 $2\tau = 2 \times (1/200000) = 0.00001s$ ；最小帧长 = $1000000000b/s \times 0.00001s = 10000bit$ 。

37. B. 【解析】本题考查 IPv6 分组和 IPv4 分组首部的比较。

IPv6 取消了首部检验和字段，以加快路由器处理 IPv6 分组的速度，选项 I 正确。IPv6 分组的数据载荷中可以包含扩展首部，取代了 IPv4 首部中的可选字段，选项 IV 正确，选项 II 错误。IPv6 虽然取消了标识、标志、片偏移这三个字段，但相关字段已在扩展首部中体现，因此 IPv6 的源主机端仍可分片（注意，中间路由器不允许进行 IPv6 分片）。



38. C. 【解析】本题考查 BGP。

路由器 R_{3c} 使用协议 eBGP 从 AS_4 的 R_{4c} 知道前缀 X。路由器 R_{3a} 使用协议 iBGP 从本自治系统的 R_{3c} 知道前缀 X。路由器 R_{1c} 使用协议 eBGP 从 AS_3 的 R_{3c} 知道前缀 X。路由器 R_{1d} 使用协议 iBGP 从本自治系统的 R_{1c} 知道前缀 X。

39. B. 【解析】本题考查对 TCP 的理解。

在 AS_1 中, 从路由器 R_{1d} 到网关路由器 R_{1c} , 若 AS_1 使用 RIP, 则应选择最短路径到 R_{1c} , 从接口 1 转发需要经过 2 跳, 但从接口 2 转发需要经过 3 跳, 因此应从接口 1 转发。若 AS_1 使用 OSPF, 则应选择代价最小的路径, 从 R_{1d} 到 R_{1c} 有两条路径: $R_{1d} \rightarrow R_{1a} \rightarrow R_{1c}$ 和 $R_{1d} \rightarrow R_{1b} \rightarrow R_{1a} \rightarrow R_{1c}$, 不过根据已知条件无法知道哪条路径的代价更高, 现在 AS_2 和 AS_4 之间有物理连接, 假定路由器 R_{1d} 知道到达前缀 X 可经过 AS_2AS_4 , 也可经过 AS_3AS_4 , 都经过 2 跳, 但在 AS_1 中路由器 R_{1d} 从接口 1 到网关路由器 R_{1c} 需经过 2 跳, 路由器 R_{1d} 到网关路由器 R_{1b} 仅需要经过 1 跳, 因此应从接口 2 转发。

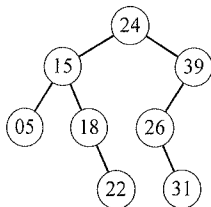
40. D. 【解析】本题考查 TCP 的流量控制方式。

RIP 使用 UDP 进行传输; OSPF 直接使用 IP 进行传输; BGP (不论是 eBGP 还是 iBGP) 都使用 TCP 进行传输。

二、综合应用题

41. 【解析】

1) 将关键字 {24, 15, 39, 26, 18, 31, 05, 22} 依次插入所构成的排序二叉树如下图所示。



前序遍历序列: 24, 15, 05, 18, 22, 39, 26, 31

中序遍历序列: 05, 15, 18, 22, 24, 26, 31, 39

后序遍历序列: 05, 22, 18, 15, 31, 26, 39, 24

2) 各关键字通过散列函数得到的散列地址如下表所示。

关键字	24	15	39	26	18	31	05	22
散列地址	11	2	0	0	5	5	5	9

二次探测法即平方探测法, $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$

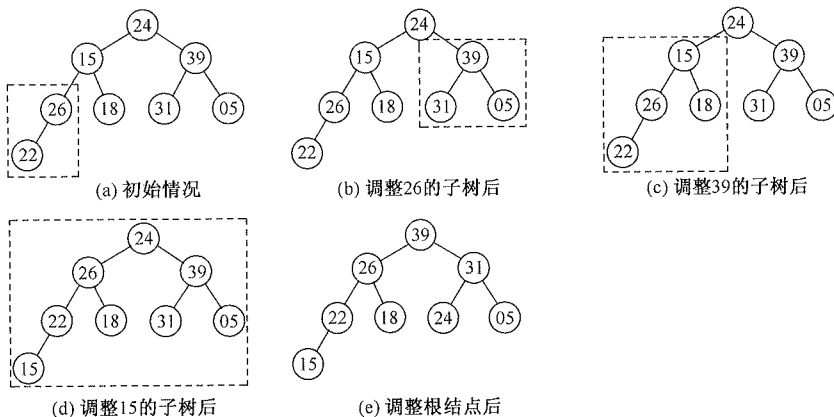
key = 24, 15, 39 均没有冲突, $H_0(26) = 0$ 有冲突, $H_1(26) = 0 + 1 = 1$ 没有冲突; key = 18 没有冲突, $H_0(31) = 5$ 有冲突, $H_1(31) = 5 + 1 = 6$ 没有冲突; $H_0(05) = 5$ 有冲突, $H_1(05) = 5 + 1 = 6$ 有冲突, $H_2(05) = 5 - 1 = 4$ 没有冲突, key = 22 没有冲突。因此, 各个关键字的存储地址如下表所示。

关键字	39	26	15		05	18	31			22		24				
存储地址	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

没有发生冲突的关键字，查找的比较次数为 1。对于发生冲突的关键字，查找的比较次数为冲突次数 + 1，因此等概率下的平均查找长度为

$$ASL = (1 + 1 + 1 + 2 + 1 + 2 + 3 + 1) / 8 = 1.5 \text{ 次}$$

- 3) 首先对以 26 为根的子树进行调整，初始情况如图(a)所示，调整后的结果如图(b)所示；对以 39 为根的子树进行调整，调整后的结果如图(c)所示；对以 15 为根的子树进行调整，调整后的结果如图(d)所示；最后对根结点进行调整，调整后的结果如图(e)所示。



42. 【解析】

思路 1（借助栈，空间复杂度高）：让表的前半部分依次进栈，依次访问后半部分时，从栈中弹出一个元素，进行比较。

思路 2（类似折纸的思想，算法复杂）：找到中间位置的元素，将后半部分的链表就地逆置，然后前半部分从前往后、后半部分从后往前比较，比较结束后再恢复（题中没有说不能改变链，因此可不恢复）。

为了让算法更简单，这里采用思路 1 中的方法，思路 2 中的方法留给有兴趣的读者。

1) 算法的基本设计思想：

- ① 借助辅助栈，让链表的前一半元素依次进栈。注意 n 为奇数时要特殊处理。
- ② 在处理链表的后一半元素时，访问到链表的一个元素后，就从栈中弹出一个元素，对两个元素进行比较，若相等，则将链表中的下一元素与栈中再弹出的元素进行比较，直至链表尾部。
- ③ 若栈是空栈，则得出链表中心对称的结论；否则，当链表中一个元素与栈中弹出的元素不等时，得出链表非中心对称的结论。

2) 算法的实现如下：

```
typedef struct LNode{                                //链表结点的结构定义
    char data;                                       //结点数据
    struct LNode *next;                             //结点链接指针
} *LinkList;
int Str_Sym(LinkList L,int n){
//本算法判断带头结点的单链表是否是中心对称
    Stack s;initstack(s);                            //初始化栈
    LNode *q,*p=L->next;                             //q 指向出栈元素，p 工作指针
```



```

for(int i=1;i<=n/2;i++){                                //前半结点入栈
    push(p);
    p=p->next;
}
if(n%2==1) p=p->next;                                    //若 n 为奇数,则需要特殊处理
while(p!=null){                                          //后半表依次和前半表比较
    q=pop(s);                                            //出栈一个结点
    if(q->data==p->data) p=p->next;                      //相等则继续比较下一个结点
    else break;                                          //不等则跳出循环
}
if(empty(s)) return 1;                                    //若栈空,则说明对称
else return 0;                                           //否则不对称
}

```

3) 算法的时间复杂度为 $O(n)$, 空间复杂度为 $O(n)$ 。

思考: 当长度未知时, 如何操作比较方便?

下面给出两种参考方法:

- ① 先遍历一遍链表, 数出元素个数, 再按参考答案操作。
- ② 同时设立一个栈和一个队列, 直接遍历一遍链表, 让每个元素的值都入栈、入队列, 然后一一出栈、出队列, 比较元素的值是否相同。

43. 【解析】

- 1) 高 16 位为段号, 低 16 位为段内偏移, 则 1 为段号 (对应基地址为 11900H), 0108H 为段内偏移量, 则逻辑地址 00010108H 对应的物理地址为 $11900H + 0108H = 11A08H$ 。
- 2) SP 的当前值为 70FF0H, 减去 4H 后得到 70FECH, 7 为段号, 0FECH 为段内偏移量, 则对应的物理地址为 $13000H + 0FECH = 13FECH$, 因此存储 x 的物理地址为 13FECH。
- 3) 调用 call sin 指令后, PC 自增为 248, 所以逻辑地址 248 被压入栈。由 2) 可知, 每次入栈时 SP 指针先减 4, 因此当前 PC 值入栈后, SP 指针的值为 $70FF0H - 4H - 4H = 70FE8H$, 所以新 SP 指针的值为 70FE8H, 新 PC 值为转移指令的目的地址 360H。

▲注意: 为什么入栈的不是物理地址?

首先, 段式存储器 (页式、段页式也一样) 中的 PC 值一定是逻辑地址; 然后, 取指令时系统才按照逻辑地址根据一定的规则转换为物理地址再去访问内存。因此, 入栈的是 PC 的内容, 当然就是逻辑地址。

- 4) 在执行 call sin 之前, x 被 push 进栈, 执行 sin 函数时, 必须在栈中找到参数 x, x 在栈中的逻辑地址为 $70FE8(SP) + 4 = 70FECH$, 故 mov 指令的目的是将参数 x 的逻辑地址送入 edx 寄存器。mov 指令的源操作数来自栈顶指针 SP 指向的内存区域, 因此寻址方式为堆栈寻址。

44. 【解析】

- 1) 因为最慢部件 (存储器) 的操作时间为 200ps, 因此在不考虑流水线寄存器、多路选择器等延迟的情况下, 五段式流水线处理器的最小时钟周期为 200ps。
- 2) 第 1 条和第 2 条、第 2 条和第 3 条、第 3 条和第 4 条、第 5 条和第 1 条指令之间存在数据相关。



- 3) 采用“转发”技术可以消除第 1 条和第 2 条、第 2 条和第 3 条、第 5 条和第 1 条指令之间的数据相关，但不能消除第 3 条和第 4 条指令之间的 load-use 冒险。需要额外一个时钟的阻塞。
- 4) 由于第 3 条和第 4 条指令之间的 load-use 冒险，8 次循环因此共导致 8 个时钟的阻塞，共有 $8 + 11 = 19$ 次阻塞，且第 1~4 条指令各执行 8 次，第 5、6 条指令各执行 7 次，因此，8 次循环所用的时钟周期数为 $4 \times 8 + 2 \times 7 + 19 = 65$ ，总时间为 $65 \times 200\text{ps} = 13\text{ns}$ 。

45. 【解析】

- 1) 进程最多允许包含 256 个逻辑页，即 2^8 个逻辑页，所以逻辑页号占 8 位，页内偏移量占 $20 - 8 = 12$ 位，逻辑地址的划分如下表所示。因此，每个逻辑页面的大小为 $2^{12}\text{B} = 4096\text{B} = 4\text{KB}$ ，物理页面大小 = 逻辑页面大小 = 4KB。

8 位页号	12 位页内偏移量
-------	-----------

- 2) 初始时，TLB 的内容为空，当访问逻辑地址 00324H 和 01367H 时，发生缺页，缺页处理程序执行完后，TLB 的内容如下表所示。

逻辑页号	页框号
00H	09H
01H	06H

当访问 02C76H 时，发生缺页，缺页处理程序执行完后，根据 LRU 替换算法，逻辑页号为 00H 的页面被替换，TLB 的内容如下表所示。

逻辑页号	页框号
02H	09H
01H	06H

当访问 01A56H 时，逻辑页号为 01H，访问 TLB 就能直接得到页框号，拼接页内偏移量得到物理地址。

当访问 03B33H 时，逻辑页号为 03H，发生缺页，缺页处理程序执行完后，根据 LRU 替换算法，逻辑页号为 02H 的页面被替换，TLB 的内容如下表所示。

逻辑页号	页框号
03H	09H
01H	06H

当访问 04478H 时，逻辑页号为 04H，发生缺页，缺页处理程序执行完后，根据 LRU 替换算法，逻辑页号为 01H 的页面被替换，TLB 的内容如下表所示。

逻辑页号	页框号
03H	09H
04H	06H

综上所述，在这四次访问中共发生 3 次缺页，缺页的逻辑地址分别为 02C76H, 03B33H, 04478H。



- 3) 根据 2) 的分析可知, 逻辑地址 02C76H 对应的页框号为 09H, 因此物理地址为 09C76H。逻辑地址 01A56H 对应的页框号为 06H, 因此物理地址为 06A56H。逻辑地址 03B33H 对应的页框号为 09H, 因此物理地址为 09B33H。逻辑地址 04478H 对应的页框号为 06H, 因此物理地址为 06478H。
- 4) 访问 01367H 时, 首先访问 TLB 需要 B 纳秒, LB 缺失, 访问内存页表需要 A 纳秒, 页表缺失, 执行中断处理程序需要 C 纳秒, 中断处理完成后, 继续访问 TLB 需要 B 纳秒, 得到物理地址后, 访问内存获取所需数据需要 A 纳秒, 故总时间开销为 $(2A + 2B + C)$ 纳秒。
- 访问 01A56H 时, 访问 TLB 直接命中, 花费时间 B 纳秒, 然后根据物理地址访问内存获取所需的数据需要 A 纳秒, 总时间开销为 $(B + A)$ 纳秒。

46. 【解析】

- 1) 链接分配: 块地址占 4B, 即 32 位, 因此能表示的最多块数为 $2^{32} = 4G$, 而每个盘块中存放的文件大小为 4092B, 故链接分配可管理的最大文件为 $4G \times 4092B = 16368GB$ 。
- ▲注意: 有的同学觉得最后一块不用放置索引块, 可以为 4096B, 但一般文件系统块的结构是固定的, 为了多这 4B 的空间, 会多很多额外的消耗, 所以并不会这么做。
- 2) 连续分配: 对大小两个文件都只需在文件控制块 (FCB) 中设两项: 一是首块物理块块号, 二是文件总块数。
- 链接分配: 对大小两个文件都只需在文件控制块 (FCB) 中设两项: 一是首块物理块块号, 二是文件最后一个物理块号。
- 3) 连续分配: 为了读大文件前面和后面的信息, 都需先计算信息在文件中的相对块数, 前面信息的相对逻辑块号为 $5.5K/4K \approx 1$ (从 0 开始编号), 后面信息的相对逻辑块号为 $(16M + 5.5K)/4K = 4097$ 。然而, 计算物理块号 = 文件首块号 + 相对逻辑块号, 最后每块分别只需花一次磁盘 I/O 操作读出该块信息。
- 链接分配: 为了读大文件前面 5.5KB 的信息, 只需先读一次文件头块得到信息所在块的块号, 再读一次第 1 号逻辑块得到所需的信息, 共需要 2 次读盘。为了读大文件 $16M + 5.5K$ 的信息, 逻辑块号为 $(16M + 5.5K)/4092 = 4101$, 要先将该信息所在块前面的块顺序读出, 共花 4101 次磁盘 I/O 操作才能得到信息所在块的块号, 最后花一次 I/O 操作读出该块信息。因此, 共需要 4102 次 I/O 操作才能读取 $(16M + 5.5K)$ 处的信息。

47. 【解析】

- 1) 在 S1 中, $SYN = 1, ACK = 0$; 在 S2 中, $SYN = 1, ACK = 1$; 在 S3 中, $SYN = 1, ACK = 1$ 。TCP 的三次握手中各标志位的值, 需要熟记。
- 2) S4 中搭载的 HTTP 报文头部中的方法是 Get。
- 3) S8 中的数据长度为 1000B, S13 中的数据长度为 500, S14 中的确认号字段值为 6501。
- B 为发送方, A 为接收方。
- S4: A 向 B 发送了 381B 的报文, 接收方的窗口大小为 6500。
- S5: B 向 A 发送了 1000B 的报文, 并对 S4 进行了确认, 希望收到 A 的第 382 个字节。
- S6: A 向 B 发送对 S5 的确认。
- S7: B 向 A 发送了 1000B 的报文, 希望收到 A 的第 382 个字节。
- S8: B 向 A 发送了 XB 的报文, 希望收到 A 的第 382 个字节。



S9: A 向 B 发送了对 S8 的确认, 并通知 B 接收窗口的大小为 3500, 由于确认号是 3001, 说明已收到前 3000B 的报文, 所以 S8 的 X 为 1000。

S10、S11、S12: B 向 A 发送 1000B 的报文。

S13: B 向 A 发送 XB 的报文, 但到目前为止一直没有收到 A 发送的更新窗口大小的报文, S9 这时得知 A 的接收窗口大小为 3500, S10~S12 共发送了 3000B 的报文, 故 S13 只能发送 500B 的报文。

S14: A 对 S13 的确认, A 已收到 6500B, 所以确认号为 6501。

- 4) 如果 B 在报文段 S14 之后仍采用慢开始机制, 那么 $n = 6000$; 如果 B 在报文段 S14 之后改用拥塞避免机制, 那么 $n = 5000$ 。



